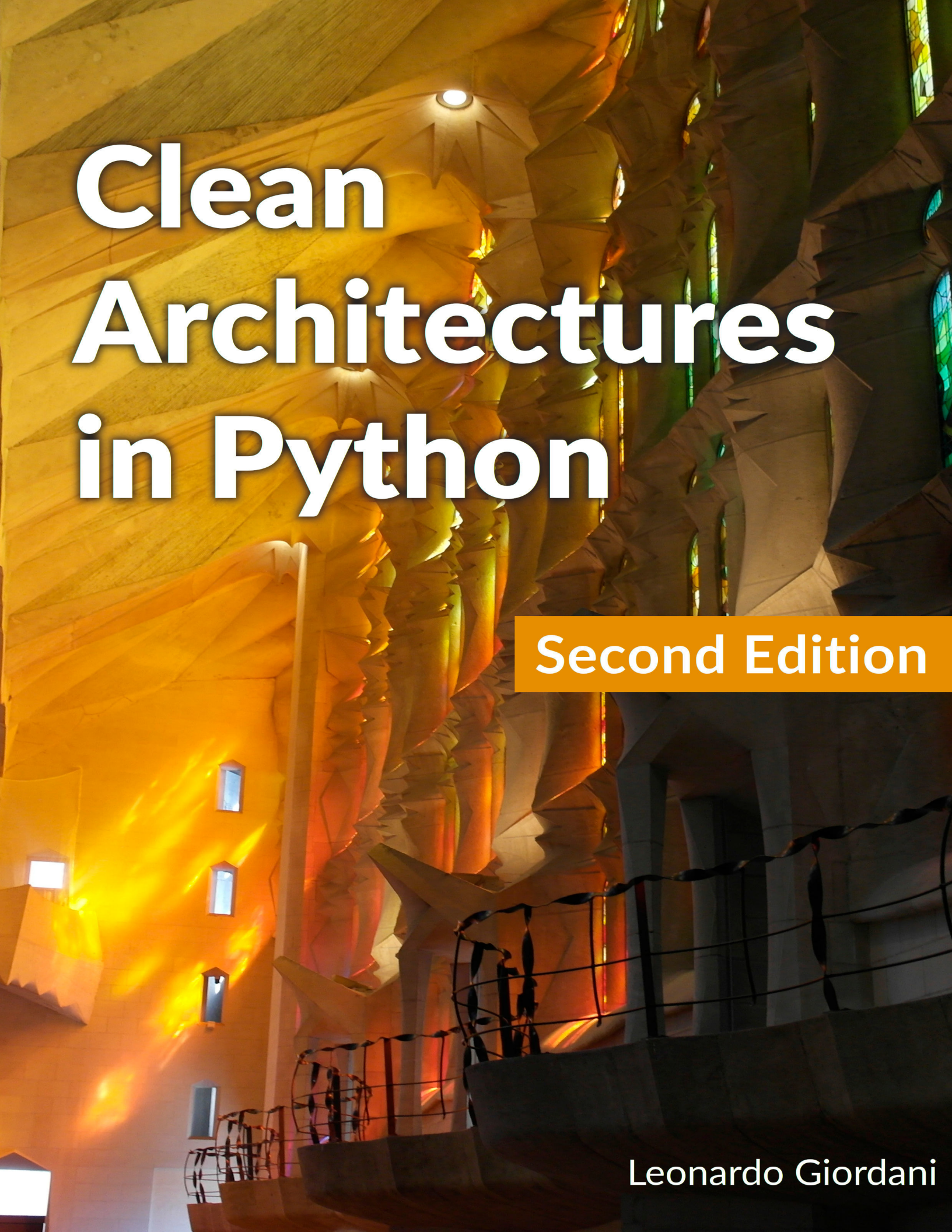




Clean Architectures in Python

Second Edition

Leonardo Giordani

Clean Architectures in Python

A practical approach to better software design

Leonardo Giordani

This book is for sale at <http://leanpub.com/clean-architectures-in-python>

This version was published on 2021-08-20



This is a [Leanpub](#) book. Leanpub empowers authors and publishers with the Lean Publishing process. [Lean Publishing](#) is the act of publishing an in-progress ebook using lightweight tools and many iterations to get reader feedback, pivot until you have the right book and build traction once you do.

© 2018 - 2021 Leonardo Giordani

Tweet This Book!

Please help Leonardo Giordani by spreading the word about this book on [Twitter](#)!

The suggested hashtag for this book is [#pycabook](#).

Find out what other people are saying about the book by clicking on this link to search for this hashtag on Twitter:

[#pycabook](#)

To my father, who taught me to be attentive, curious, and passionate. He succeeded.

To my mother, who taught me to be smart, cautious, and careful. She didn't succeed.

Contents

Introduction	1
What is a software architecture?	2
Why is it called “clean”?	3
Why “architectures”?	4
Why Python?	4
Acknowledgments	4
About the book	6
Prerequisites and structure of the book	6
Typographic conventions	7
Why this book comes for free	8
Submitting issues or patches	8
About the author	8
Changes in the second edition	9
Chapter 01 A day in the life of a clean system	11
The data flow	11
Advantages of a layered architecture	18
Chapter 02 Components of a clean architecture	22
Components of a clean architecture	22
Chapter 03 A basic example	35
Chapter 04 Add a web application	52
Flask setup	52
Test and create an HTTP endpoint	54
WSGI	60
Chapter 05 Error management	62
Request and responses	62
Basic structure	63
Requests and responses in a use case	64
Request validation	66
Responses and failures	70

CONTENTS

Error management in a use case	75
Integrating external systems	77
Chapter 06 Integration with a real external system postgres	89
Decoupling with interfaces	89
A repository based on PostgreSQL	91
Label integration tests	91
Create SQLAlchemy classes	94
Orchestration management	96
Database fixtures	105
Integration tests	109
Chapter 07 Integration with a real external system mongodb	114
Fixtures	114
Docker Compose configuration	117
Application configuration	117
Integration tests	119
The MongoDB repository	122
Chapter 08 Run a production ready system	125
Build a web stack	125
Connect to a production-ready database	132
Changelog	143
Colophon	147

Introduction



Learn about the Force, Luke.
Star Wars, 1977

This book is about a software design methodology. A methodology is a set of guidelines that help you to reach your goal effectively, thus saving time, implementing far-sighted solutions, and avoiding the need to reinvent the wheel time and again.

As other professionals around the world face problems and try to solve them, some of them, having discovered a good way to solve a problem, decide to share their experience, usually in the form of a “best practices” post on a blog, or talk at a conference. We also speak of *patterns*¹, which are formalised best practices, and *anti-patterns*, when it comes to advice about what not to do and why it is better to avoid a certain solution.

Often, when best practices encompass a wide scope, they are designated a *methodology*. The definition of a methodology is to convey a method, more than a specific solution to a problem. The very nature of methodologies means they are not connected to any specific case, in favour of a wider and more generic approach to the subject matter. This also means that applying methodologies without thinking shows that one didn’t grasp the nature of a methodology, which is to help to find a solution and not to provide it.

This is why the main advice I have to give is: be reasonable; try to understand why a methodology leads to a solution and adopt it if it fits your need. I’m saying this at the very beginning of this book because this is how I’d like you to approach this work of mine.

The clean architecture, for example, pushes abstraction to its limits. One of the main concepts is that you should isolate parts of your system as much as possible, so you can replace them without affecting the rest. This requires a lot of abstraction layers, which might affect the performances of the system, and which definitely require a greater initial development effort. You might consider these shortcomings unacceptable, or perhaps be forced to sacrifice cleanness in favour of execution speed, as you cannot afford to waste resources.

In these cases, break the rules.

With methodologies you are always free to keep the parts you consider useful and discard the rest, and if you have understood the reason behind the methodology, you will also be aware of the reasons that support your decisions. My advice is to keep track of such reasons, either in design documents

¹Unless you consider things like `sessionmaker_mock().query.assert_called_with(Room)` something attractive. And this was by far the simplest mock I had to write.

or simply in code comments, as a future reference for you or for any other programmer who might be surprised by a “wrong” solution and be tempted to fix it.

I will try as much as possible to give reasons for the proposed solutions, so you can judge whether those reasons are valid in your case. In general let’s say this book contains possible contributions to your job, it’s not an attempt to dictate THE best way to work.

Spoiler alert: there is no such a thing.

What is a software architecture?

Every production system, be it a software package, a mechanical device, or a simple procedure, is made of components and connections between them. The purpose of the connections is to use the output of some components as inputs of other components, in order to perform a certain action or set of actions.

In a process, the architecture specifies which components are part of an implementation and how they are interconnected.

A simple example is the process of writing a document. The process, in this case, is the conversion of a set of ideas and sentences into a written text, and it can have multiple implementations. A very simple one is when someone writes with a pen on a sheet of paper, but it might become more complex if we add someone who is writing what another person dictates, multiple proof readers who can send back the text with corrections, and a designer who curates the visual rendering of the text. In all these cases the process is the same, and the nature of inputs (ideas, sentences) and outputs (a document or a book) doesn’t change. The different architecture, however, can greatly affect the quality of the output, or the speed with which it is produced.

An architecture can have multiple granularities, which are the “zoom level” we use to look at the components and their connections. The first level is the one that describes the whole process as a black box with inputs and outputs. At this level we are not even concerned with components, we don’t know what’s inside the system and how it works. We only know what it does.

As you zoom in, you start discovering the details of the architecture, that is which components are in the aforementioned black box and how they are connected. These components are in turn black boxes, and you don’t want to know specifically how they work, but you want to know what their inputs and outputs are, where the inputs come from, and how the outputs are used by other components.

This process is virtually unlimited, so there is never one single architecture that describes a complete system, but rather a set of architectures, each one covering the granularity we are interested in.

Let me go over another simple example that has nothing to do with software. Let’s consider a shop as a system and let’s discuss its architecture.

A shop, as a black box, is a place where people enter with money and exit with items (if they found what they were looking for). The input of the system are people and their money, and the outputs

are the same people and items. The shop itself needs to buy what it sells first, so another input is represented by the stock the shop buys from the wholesaler and another output by the money it pays for it. At this level the internal structure of the shop is unknown, we don't even know what it sells. We can however already devise a simple performance analysis, for example comparing the amount of money that goes out (to pay the wholesaler) and the amount of money that comes in (from the customers). If the former is higher than the latter the business is not profitable.

Even in the case of a shop that has positive results we might want to increase its performances, and to do this chances are that we need to understand its internal structure and what we can change to increase its productivity. This may reveal, for example, that the shop has too many workers, who are underemployed waiting for clients because we overestimated the size of the business. Or it might show that the time taken to serve clients is too long and many clients walk away without buying anything. Or maybe there are not enough shelves to display goods and the staff carries stock around all day searching for display space so the shop is in chaos and clients cannot find what they need.

At this level, however, workers are pure entities, and still we don't know much about the shop. To better understand the reasons behind a problem we might need to increase the zoom level and look at the workers for what they are, human beings, and start understanding what their needs are and how to help them to work better.

This example can easily be translated into the software realm. Our shop is a processing unit in the cloud, for example, input and output being the money we pay and the amount of requests the system serves per second, which is probably connected with the income of the business. The internal processes are revealed by a deeper analysis of the resources we allocate (storage, processors, memory), which breaks the abstraction of the “processing unit” and reveals details like the hardware architecture or the operating system. We might go deeper, discussing the framework or the library we used to implement a certain service, the programming language we used, or the specific hardware on which the whole system runs.

Remember that an architecture tries to detail how a process is implemented at a certain granularity, given certain assumptions or requirements. The quality of an architecture can then be judged on the basis of parameters such as its cost, the quality of the outputs, its simplicity or “elegance”, the amount of effort required to change it, and so on.

Why is it called “clean”?

The architecture explained in this book has many names, but the one that is mainly in use nowadays is “clean architecture”. This is the name used by Robert Martin in [his seminal post](#)² where he clearly states this structure is not a novelty, but has been promoted by many software designers over the years. I believe the adjective “clean” describes one of the fundamental aspects of both the software structure and the development approach of this architecture. It is clean, that is, it is easy to understand what happens.

²<https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html>

The clean architecture is the opposite of spaghetti code, where everything is interlaced and there are no single elements that can be easily detached from the rest and replaced without the whole system collapsing. The main point of the clean architecture is to make clear “what is where and why”, and this should be your first concern while you design and implement a software system, whatever architecture or development methodology you want to follow.

The clean architecture is not the perfect architecture and cannot be applied unthinkingly. Like any other solution, it addresses a set of problems and tries to solve them, but there is no panacea that will solve all issues. As already stated, it’s better to understand how the clean architecture solves some problems and decide if the solution suits your need.

Why “architectures”?

While I was writing the first edition of the book it became clear to me that the goal of this book is to begin a journey and not to define the specific steps through which each software designer has to go through. The concepts explained here are rooted in some design principles that are much more important than the resulting physical structure of the system that you will create.

This is why I wanted to stress that what I show in this book can (and hopefully will) be an inspiration for many different architectures that you will create to solve the problems you will have to face.

Or maybe I just wanted to avoid looking like a clone of Robert Martin.

Why Python?

I have been working with Python for 20 years, along with other languages, but I came to love its simplicity and power and so I ended up using it on many projects. When I was first introduced to the clean architecture I was working on a Python application that was meant to glue together the steps of a processing chain for satellite imagery, so my journey with the concepts I will explain started with this language.

I will therefore speak of Python in this book, but the main concepts are valid for any other language, especially object-oriented ones. I will not introduce Python here, so a minimal knowledge of the language syntax is needed to understand the examples and the project I will discuss.

The clean architecture concepts are independent of the language, but the implementation obviously leverages what a specific language allows you to do, so this book is about the clean architecture and an implementation of it that I devised using Python. I really look forward to seeing more books about the clean architecture that explore other implementations in Python and in other languages.

Acknowledgments

- Eleanor de Veras, who proofread the introduction.

- Roberto Ciatti, who introduced me to clean architectures.
- Readers Eric Smith, Faust Gertz, Giovanni Natale, Grant Moore, Hans Chen, Max H. Gerlach, Michael O'Neill, Paul Schwendenman, Ramces Chirino, Rodrigo Monte, Simon Weiss, Thiago C. D'Ávila, robveijk, mathisheeren, 4myhw, Jakob Waibel, 1110sillabo, Maxim Ivanov who fixed bugs, typos and bad grammar submitting issues and pull requests.
- Łukasz Dziejczak, who developed the free “Lato” font (<http://www.latofonts.com>), used for the cover.

The cover photograph is by [pxhere](#)³. A detail of the Sagrada Familia in Barcelona, one of the world's best contemporary artworks, a bright example of architecture in which every single element has a meaning and a purpose. Praise to Antoni Gaudí, brilliant architect and saint, who will always inspire me with his works and his life.

³<https://pxhere.com/en/photo/1453753>

About the book

”

We'll put the band back together, do a few gigs, we get some bread. Bang! Five thousand bucks.

The Blues Brothers, 1980

In 2015 I was introduced to the clean architecture by a colleague of mine, Roberto Ciatti. I started working with him following a strict Test-Driven Development (TDD) approach and learning or better understanding many things I now consider pillars of my programming knowledge.

Unfortunately the project was cancelled, but the clean architecture concepts stuck with me, so I revisited them for a simple open source project I started working on at the time⁴. Meanwhile I read “Object Oriented Software Engineering: A Use-Case Driven Approach” by Ivar Jacobson[^footnote2].

In 2013 I started writing a personal blog, [The Digital Cat](#)⁵, and after having published many Python-related posts I began working on a post to show other programmers the beauty of the clean architecture concepts: “Clean Architectures in Python: a step by step example”, published in 2016, which was well received by the Python community. For a couple of years I considered expanding the post, but I couldn't find the time to do it, and in the meanwhile I realised that many things I had written needed to be corrected, clarified, or simply updated. So I thought that a book could be the best way to present the whole picture effectively, and here we are.

In 2020, after having delayed it for a long time, I decided to review the whole book, updating it and clarifying parts that weren't particularly well written. I also decided to remove the part on TDD. While I believe every programmer should understand TDD, the topic of the book is different, so I updated the material and published it on my blog.

This book is the product of many hours spent thinking, experimenting, studying, and making mistakes. I couldn't have written it without the help of many people, some of whose names I don't know, who provided free documentation, free software, free help. Thanks everybody! I also want to specifically say thanks to many readers who came back to me with suggestions, corrections, or simply with appreciation messages. Thanks all!

Prerequisites and structure of the book

To fully appreciate the book you need to know Python and be familiar with TDD, in particular with unit testing and mocks. Please refer to the series [TDD in Python with pytest](#)⁶ published on my blog

⁴Unless you consider things like `sessionmaker_mock().query.assert_called_with(Room)` something attractive. And this was by far the simplest mock I had to write.

⁵<https://www.thedigitalcatonline.com/>

⁶<https://www.thedigitalcatonline.com/blog/2020/09/10/tdd-in-python-with-pytest-part-1/>

if you need to refresh your knowledge about these topics.

After the two introductory parts that you are reading, chapter 1 goes through a **10,000 feet overview** of a system designed with a clean architecture, while chapter 2 briefly discusses the **components** and the ideas behind this software architecture. Chapter 3 runs through a **concrete example** of clean architecture and chapter 4 expands the example adding a **web application** on top of it. Chapter 5 discusses **error management** and improvements to the Python code developed in the previous chapters. Chapters 6 and 7 show how to plug **different database systems** to the web service created previously, and chapter 8 wraps up the example showing how to run the application with a **production-ready configuration**.

Typographic conventions

This book uses Python, so the majority of the code samples will be in this language, either inline or in a specific code block like this

```
some/path/file_name.py
```

```
1 def example():  
2     print("This is a code block")
```

Note that the path of the file that contains the code is printed just before the source code. Code blocks don't include line numbers, as the part of code that are being discussed are usually repeated in the text. This also makes it possible to copy the code from the PDF directly.

Shell commands are presented with a generic prompt \$

```
1 $ command --option1 value1 --option2 value 2
```

which means that you will copy and execute the string starting from `command`.

I will also use two different asides to link the code repository and to mark important principles.

This box provides a link to the commit or the tag that contains the code that was presented



Source code

<https://github.com/pycabook/rentomatic/tree/master>

This box highlights a concept explained in detail in the current chapter



Concept

This recaps an important concept that is explained in the text.

Why this book comes for free

The first reason I started writing a technical blog was to share with others my discoveries, and to save them the hassle of going through processes I had already cracked. Moreover, I always enjoy the fact that explaining something forces me to better understand that topic, and writing requires even more study to get things clear in my mind, before attempting to introduce other people to the subject.

Much of what I know comes from personal investigations, but without the work of people who shared their knowledge for free I would not have been able to make much progress. The Free Software Movement didn't start with Internet, and I got a taste of it during the 80s and 90s, but the World Wide Web undeniably gave an impressive boost to the speed and quality of this knowledge sharing.

So this book is a way to say thanks to everybody gave their time to write blog posts, free books, software, and to organise conferences, groups, meetups. This is why I teach people at conferences, this is why I write a technical blog, this is the reason behind this book.

That said, if you want to acknowledge my effort with money, feel free. Anyone who publishes a book or travels to conferences incurs expenses, and any help is welcome. However, the best thing you can do is to become part of this process of shared knowledge; experiment, learn and share what you learn. If you'd like to contribute financially you can purchase the book on [Leanpub](#)⁷.

Submitting issues or patches

This book is not a collaborative effort. It is the product of my work, and it expresses my personal view on some topics, and also follows my way of teaching. Both however can be improved, and they might also be wrong, so I am open to suggestions, and I will gladly receive any report about mistakes or any request for clarifications. Feel free to use the GitHub Issues of the [book repository](#)⁸ or of the projects presented in the book. I will answer or fix issues as soon as possible, and if needed I will publish a new version of the book with the correction. Thanks!

About the author

My name is Leonardo Giordani, I was born in 1977, a year that gave to the world Star Wars, bash, Apple II, BSD, Dire Straits, The Silmarillion, among many other things. I'm interested in operating systems and computer languages, photography, fantasy and science fiction, video and board games, guitar playing, and (too) many other things.

⁷<https://leanpub.com/clean-architectures-in-python>

⁸<https://github.com/pycabook/pycabook/issues>

I studied and used several programming languages, among them my favourite are the Motorola 68k Assembly, C, and Python. I love mathematics and cryptography. I'm mainly interested in open source software, and I like both the theoretical and practical aspects of computer science.

For 13 years I have been a C/Python programmer and devops for a satellite imagery company, and I am currently one of the lead developers at [WeGotPOP](https://www.wegotpop.com)⁹, a UK company based in London and New York that creates innovative software for film productions.

In 2013 I started publishing some technical thoughts on my blog, [The Digital Cat](https://www.thedigitalcatonline.com)¹⁰, and in 2018 I published the first edition of the book you are currently reading.

Changes in the second edition

New edition, new typos! I'm pretty sure this is the major change I introduced with this edition.

Jokes aside, this second edition contains many changes, but the core example is the same, and while the code changed a little (I use dataclasses and introduced a management script to orchestrate tests) nothing revolutionary happened from that point of view.

So, if you already read the first edition, you might want to have a look at chapters 6, 7, and 8, where I reworked the way I manage integration tests and the production-ready setup of the project. If you haven't read the first edition I hope you will appreciate the effort I made to introduce the clean architecture with a narrated example in chapter 1, before I start discussing the architecture in more detail and show you some code.

The biggest change that readers of the first edition might notice in the content is that I removed the part on TDD and focused only on the clean architecture. What I wrote on TDD has become a series of 5 posts on my blog, that I reference in the book, but this time I preferred to stay faithful to the title and discuss only the subject matter. This probably means that the book is not suitable for complete beginners any more, but since the resources are out there I don't feel too guilty.

I also experimented with different toolchains. The first edition was created directly with [Leanpub's Markua](https://leanpub.com/markua)¹¹ language, which gave me all I needed to start. While working on the second edition, though, I grew progressively unsatisfied because of the lack of features like admonitions and file names for the source snippets, and a general lack of configuration options. I think Leanpub is doing a great job, but Markua didn't provide all the features that I needed. So I tried [Pandoc](https://pandoc.org/)¹², and I immediately hit the wall of Latex, which is obscure black magic to say the least. I spent a great amount of time hacking templates and Python filters to get more or less what I wanted, but I wasn't happy.

Eventually I discovered [AsciiDoc](https://asciidoc.org/)¹³ and that looked like the perfect solution. I actually published the first version of the second edition with this toolchain, and I was blown away by AsciiDoc in comparison with Markdown. Unfortunately I had a lot of issues trying to customise the standard

⁹<https://www.wegotpop.com>

¹⁰<https://www.thedigitalcatonline.com>

¹¹<https://leanpub.com/markua/read>

¹²<https://pandoc.org/>

¹³<https://asciidoc.org/>

template, and not knowing Ruby worsened my experience. After a while I got to a decent version (which I published), but I kept thinking that I wanted more.

So I decided to try to write my own parser and here we go. This version of the book has been written using Mau, which is available at <https://github.com/Project-Mau>, and Pelican (<https://getpelican.com>), which I already successfully use for my blog. I'm in the process of writing a Mau Visitor that converts the source code into Markua, so that I can use Leanpub's tools to produce a PDF.

I hope you will enjoy the effort I put into this new edition!

Chapter 01 A day in the life of a clean system

”

Must be my lucky day.

Terminator 2, 1991

In this chapter I will introduce the reader to a (very simple) system designed with a clean architecture. The purpose of this introductory chapter is to familiarise with main concepts like separation of concerns and inversion of control, which are paramount in system design. While I describe how data flows in the system, I will purposefully omit details, so that we can focus on the global idea and not worry too much about the implementation. This example will be then explored in all its glorious details in the following chapters, so there will be time to discuss specific choices. For now, try to get the big picture.

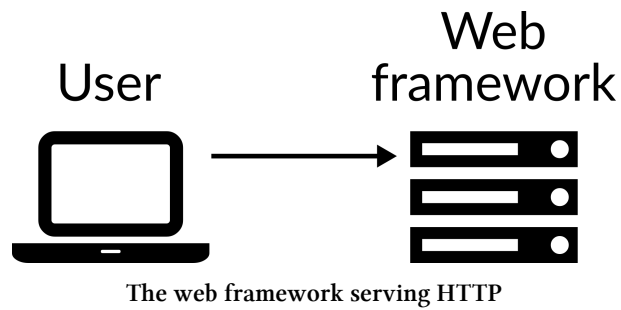
The data flow

In the rest of the book, we will design together part of a simple web application that provides a room renting system. So, let's consider that our “Rent-o-Matic” application¹⁴ is running at <https://www.rentomatic.com>, and that a user wants to see the available rooms. They open the browser and type the address, then clicking on menus and buttons they reach the page with the list of all the rooms that our company rents.

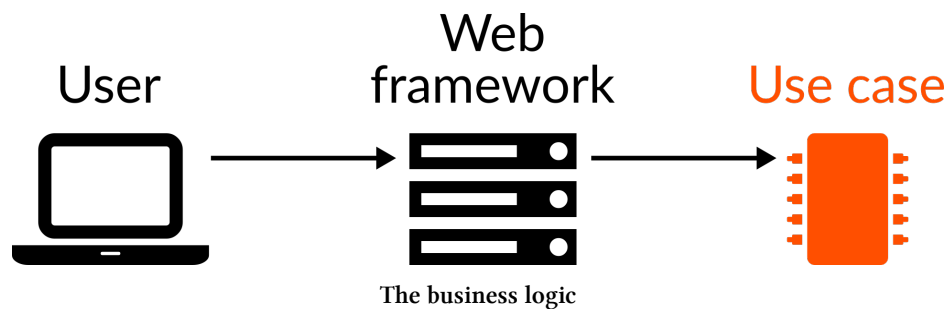
Let's assume that this URL is `/rooms?status=available`. When the user's browser accesses that URL, an HTTP request reaches our system, where there is a component that is waiting for HTTP connections. Let's call this component “web framework”^[^footnote2].

The purpose of the web framework is to understand the HTTP request and to retrieve the data that we need to provide a response. In this simple case there are two important parts of the request, namely the endpoint itself (`/rooms`), and a single query string parameter, `status=available`. Endpoints are like commands for our system, so when a user accesses one of them, they signal to the system that a specific service has been requested, which in this case is the list of all the rooms that are available for rent.

¹⁴Unless you consider things like `sessionmaker_mock().query.assert_called_with(Room)` something attractive. And this was by far the simplest mock I had to write.



The domain in which the web framework operates is that of the HTTP protocol, so when the web framework has decoded the request it should pass the relevant information to another component that will process it. This other component is called *use case*, and it is the crucial and most important component of the whole clean system as it implements the *business logic*.



The business logic is an important concept in system design. You are creating a system because you have some knowledge that you think might be useful to the world, or at the very least marketable. This knowledge is, at the end of the day, a way to process data, a way to extract or present data that maybe others don't have. A search engine can find all the web pages that are related to the terms in a query, a social network shows you the posts of people you follow and sorts them according to a specific algorithm, a travel company finds the best options for your journey between two locations, and so on. All these are good examples of business logic.



Business logic

Business logic is the specific algorithm or process that you want to implement, the way you transform data to provide a service. It is the most important part of the system.

The use case implements a very specific part of the whole business logic. In this case we have a use case to search for rooms with a given value of the parameter `status`. This means that the use case has to extract all the rooms that are managed by our company and filter them to show only the ones that are available.

Why can't the web framework do it? Well, the main purpose of a good system architecture is to *separate concerns*, that is to keep different responsibilities and domains separated. The web framework is there to process the HTTP protocol, and is maintained by programmers that are

concerned with that specific part of the system, and adding the business logic to it mixes two very different fields.



Separation of concerns

Different parts a system should manage different parts of the process. Whenever two separate parts of a system work on the same data or the same part of a process they are *coupled*. While coupling is unavoidable, the higher the coupling between two components the harder is to change one without affecting the other.

As we will see, separating layers allows us to maintain the system with less effort, making single parts of it more testable and easily replaceable.

In the example that we are discussing here, the use case needs to fetch all the rooms that are in an available state, extracting them from a source of data. This is the business logic, and in this case it is very straightforward, as it will probably consist of a simple filtering on the value of an attribute. This might however not be the case. An example of a more advanced business logic might be an ordering based on a recommendation system, which might require the use case to connect with more components than just the data source.

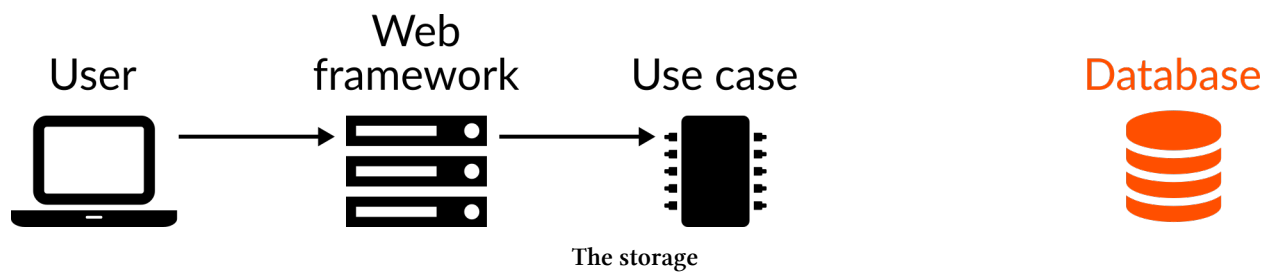
So, the information that the use case wants to process is stored somewhere. Let's call this component *storage system*. Many of you probably already pictured a database in your mind, maybe a relational one, but that is just one of the possible data sources. The abstraction represented by the storage system is: anything that the use case can access and that can provide data is a source. It might be a file, a database (either relational or not), a network endpoint, or a remote sensor.



Abstraction

When designing a system, it is paramount to think in terms of abstractions, or building blocks. A component has a role in the system, regardless of the specific implementation of that component. The higher the level of the abstraction, the less detailed are the components. Clearly, high-level abstractions don't consider practical problems, which is why the abstract design has to be then implemented using specific solutions or technologies.

For simplicity's sake, let's use a relational database like Postgres in this example, as it is likely to be familiar to the majority of readers, but keep in mind the more generic case.



How does the use case connect with the storage system? Clearly, if we hard code into the use case the calls to a specific system (e.g. using SQL) the two components will be *strongly coupled*, which is something we try to avoid in system design. Coupled components are not independent, they are tightly connected, and changes occurring in one of the two force changes in the second one (and vice versa). This also means that testing components is more difficult, as one component cannot live without the other, and when the second component is a complex system like a database this can severely slow down development.

For example, let's assume the use case called directly a specific Python library to access PostgreSQL such as `psycopg`¹⁵. This would couple the use case with that specific source, and a change of database would result in a change of its code. This is far from being ideal, as the use case contains the business logic, which has not changed moving from one database system to the other. Parts of the system that do not contain the business logic should be treated like implementation details.



Implementation detail

A specific solution or technology is called a *detail* when it is not central to the design as a whole. The word doesn't refer to the inherent complexity of the subject, which might be greater than that of more central parts.

A relational database is hundred of times richer and more complex than an HTTP endpoint, and this in turn is more complex than ordering a list of objects, but the core of the application is the use case, not the way we store data or the way we provide access to that. Usually, implementation details are mostly connected with performances or usability, while the core parts implement the pure business logic.

How can we avoid strong coupling? A simple solution is called *inversion of control*, and I will briefly sketch it here, and show a proper implementation in a later section of the book, when we will implement this very example.

Inversion of control happens in two phases. First, the called object (the database in this case) is wrapped with a standard interface. This is a set of functionalities shared by every implementation of the target, and each interface translates the functionalities to calls to the specific language[^footnote3] of the wrapped implementation.



Inversion of control

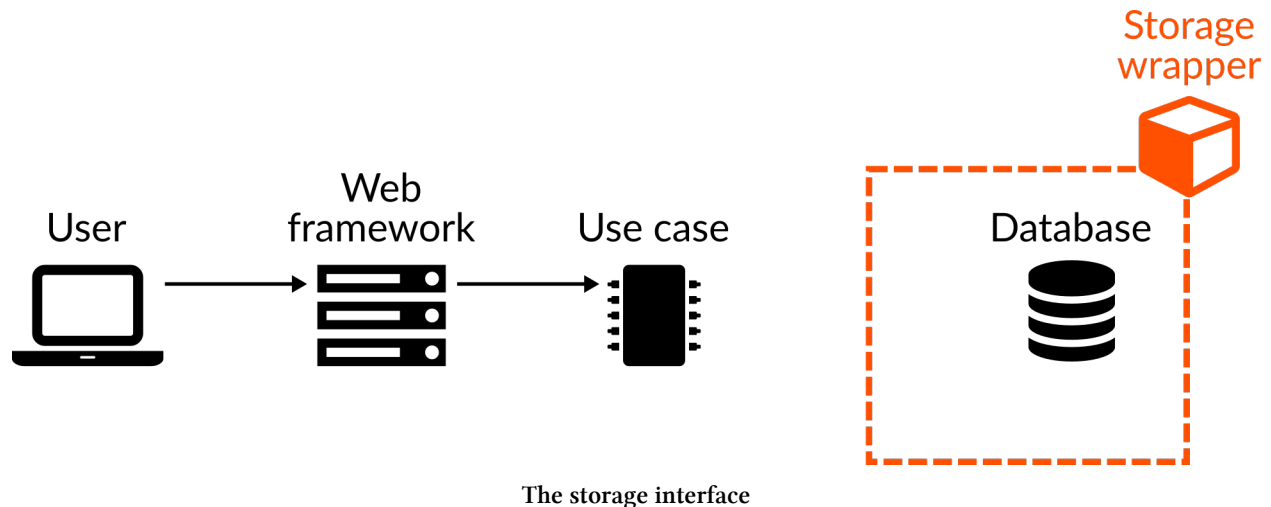
A technique used to avoid strong coupling between components of a system, that involves wrapping them so that they expose a certain interface. A component expecting that interface can then connect to them without knowing the details of the specific implementation, and thus being strongly coupled to the interface instead of the specific implementation.

A real world example of this is that of power plugs: electric appliances are designed to be connected not with specific power plugs, but to any power plug that is build according to the specification

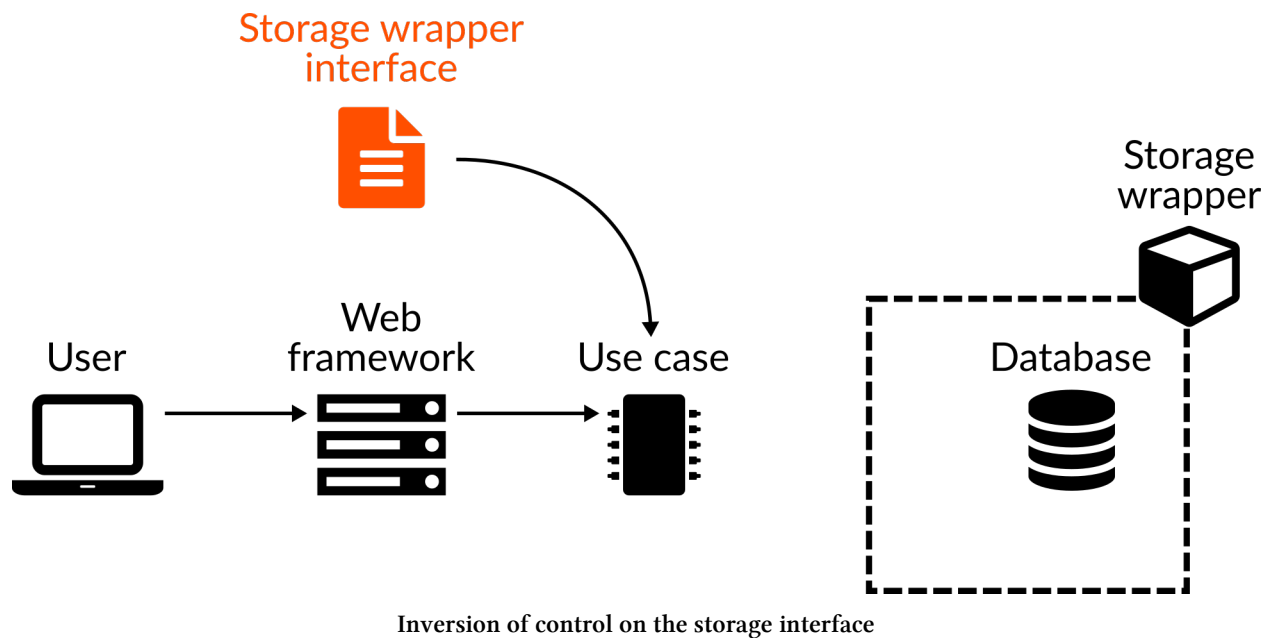
¹⁵<https://www.psycopg.org/>

(size, number of poles, etc). When you buy a TV in the UK, you expect it to come with a UK plug (BS 1363). If it doesn't, you need an *adapter* that allows you to plug electronic devices into sockets of a foreign nation. In this case, we need to connect the use case (TV) to a database (power system) that have not been designed to match a common interface.

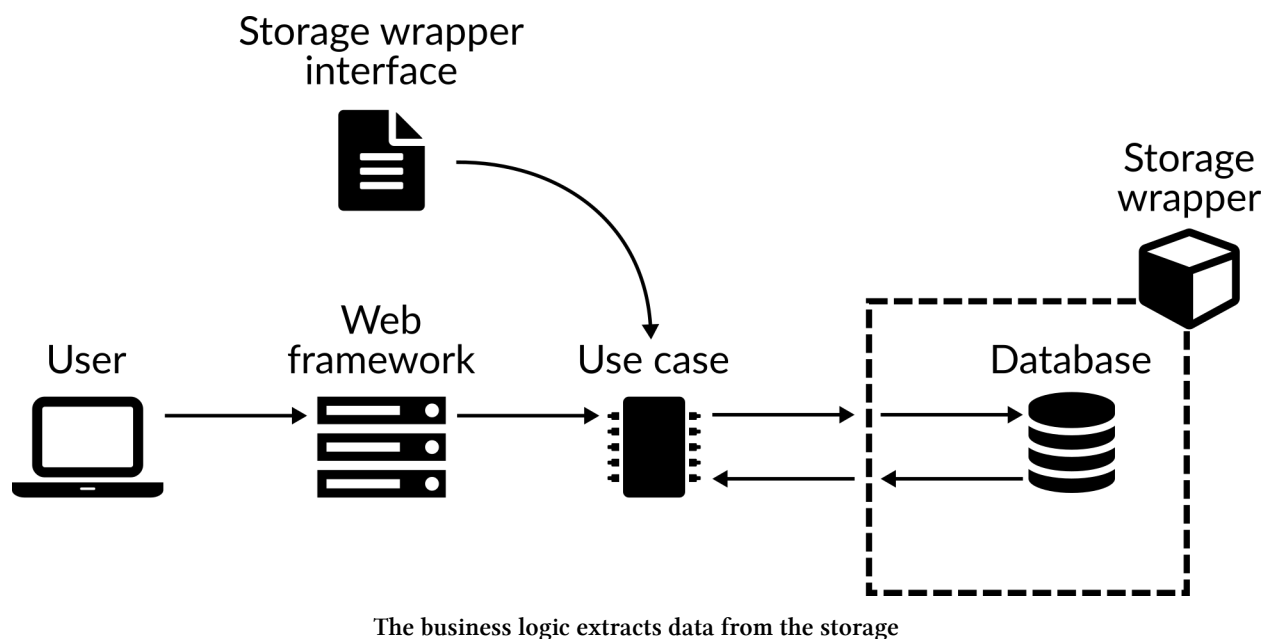
In the example we are discussing, the use case needs to extract all rooms with a given status, so the database wrapper needs to provide a single entry point that we might call `list_rooms_with_status`.



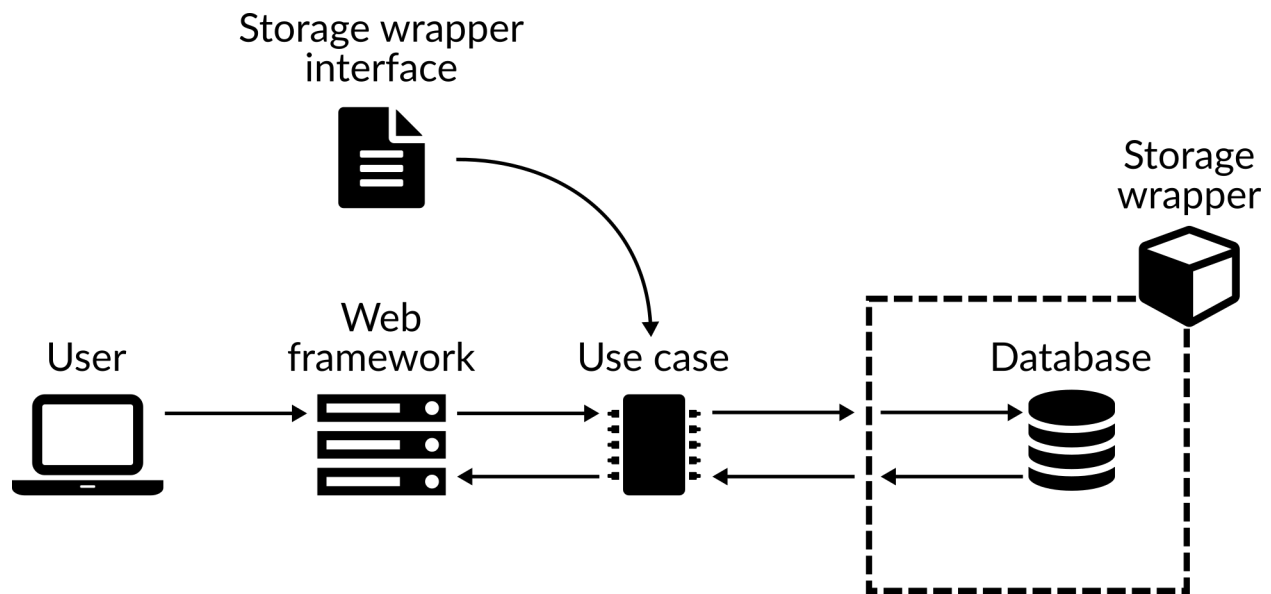
In the second phase of inversion of control the caller (the use case) is modified to avoid hard coding the call to the specific implementation, as this would again couple the two. The use case accepts an incoming object as a parameter of its constructor, and receives a concrete instance of the adapter at creation time. The specific technique used to implement this depends greatly on the programming language we use. Python doesn't have an explicit syntax for interfaces, so we will just assume the object we pass implements the required methods.



Now the use case is connected with the adapter and knows the interface, and it can call the entry point `list_rooms_with_status` passing the status available. The adapter knows the details of the storage system, so it converts the method call and the parameter in a specific call (or set of calls) that extract the requested data, and then converts them in the format expected by the use case. For example, it might return a Python list of dictionaries that represent rooms.

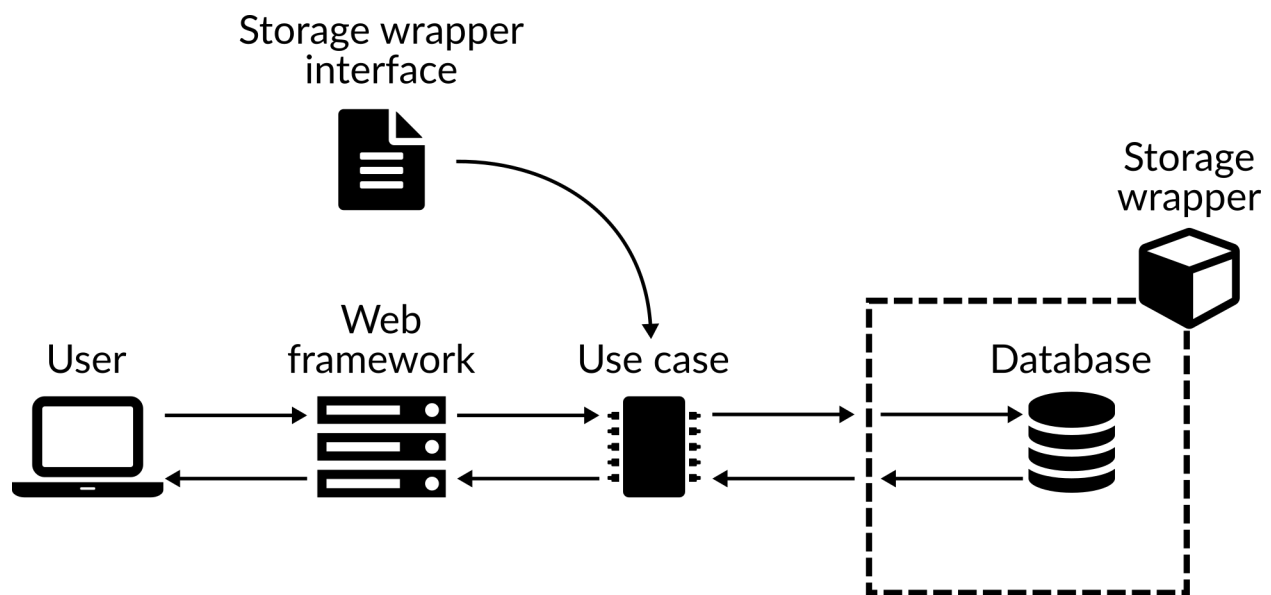


At this point, the use case has to apply the rest of the business logic, if needed, and return the result to the web framework.



The business logic returns processed data to the web framework

The web framework converts the data received from the use case into an HTTP response. In this case, as we are considering an endpoint that is supposed to be reached explicitly by the user of the website, the web framework will return an HTML page in the body of the response, but if this was an internal endpoint, for example called by some asynchronous JavaScript code in the front-end, the body of the response would probably just be a JSON structure.

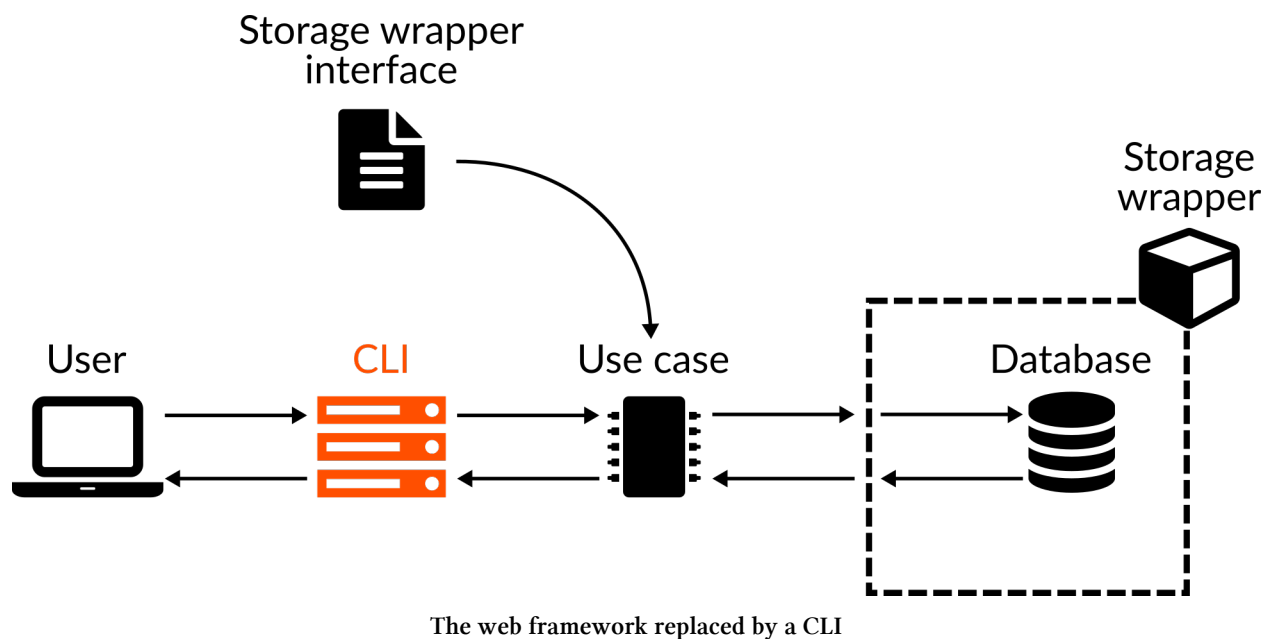


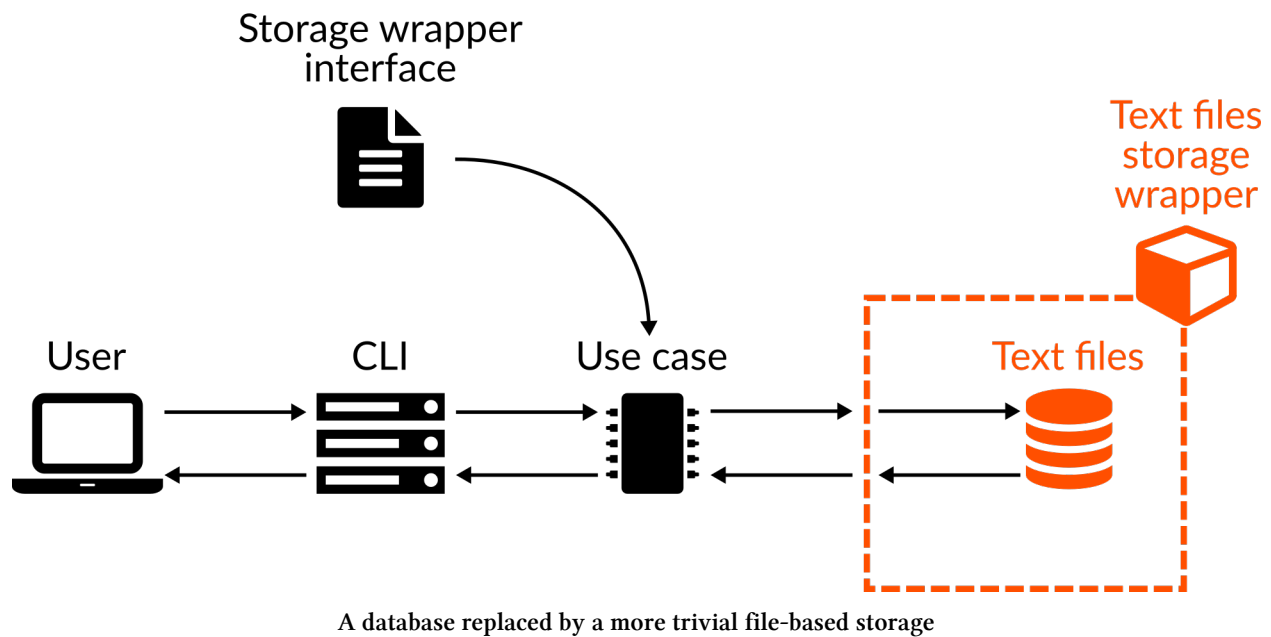
The web framework returns the data in an HTTP response

Advantages of a layered architecture

As you can see, the stages of this process are clearly separated, and there is a great deal of data transformation between them. Using common data formats is one of the way we achieve independence, or loose coupling, between components of a computer system.

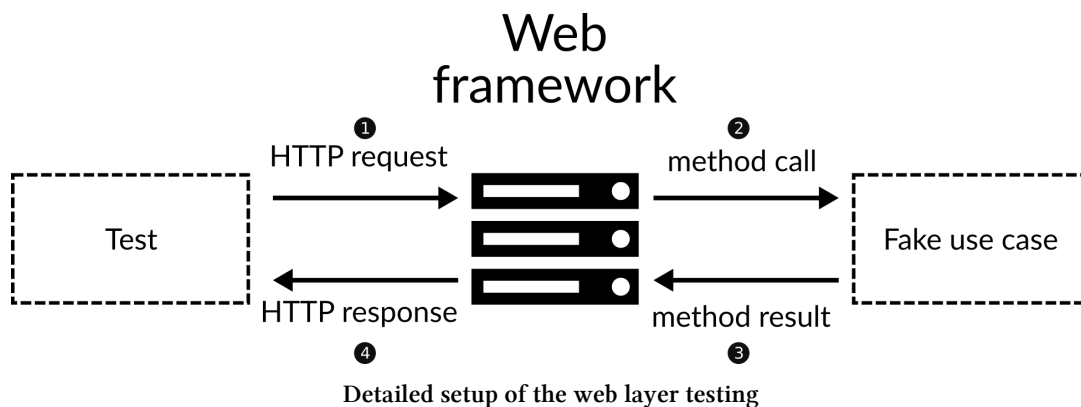
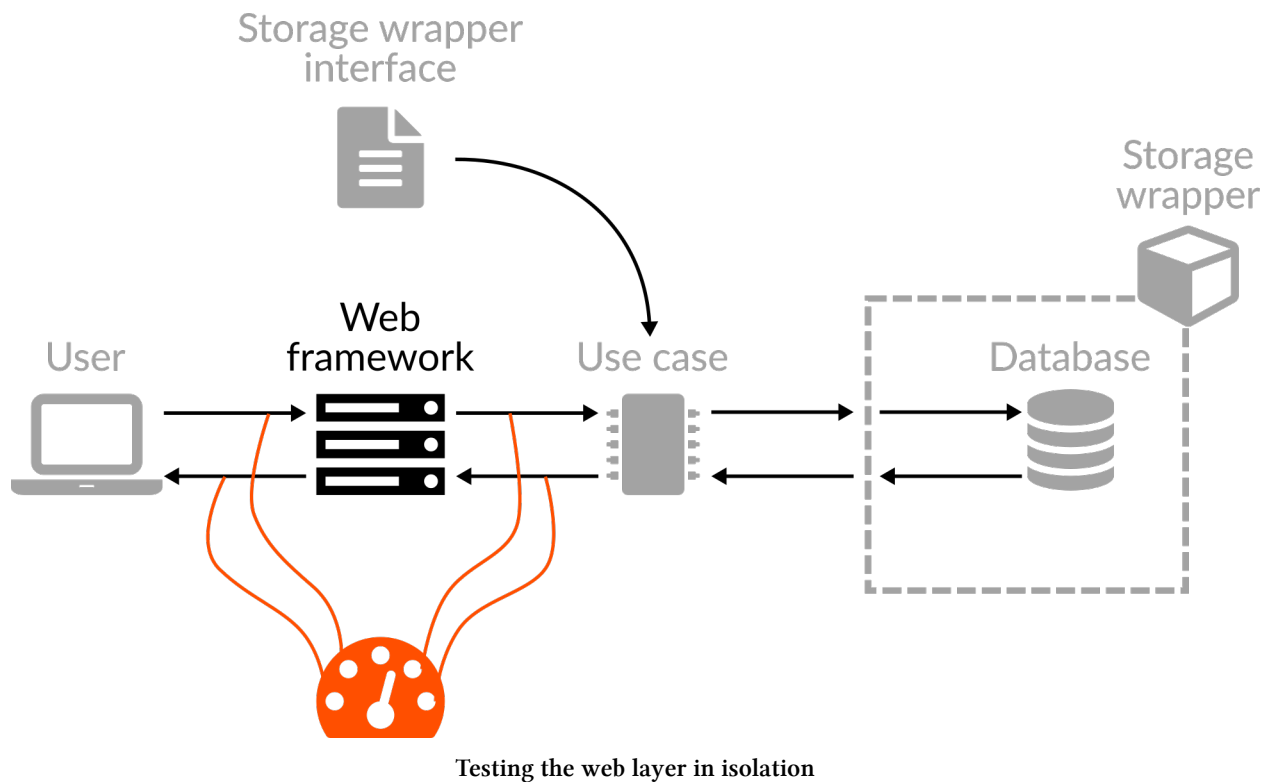
To better understand what loose coupling means for a programmer, let's consider the last picture. In the previous paragraphs I gave an example of a system that uses a web framework for the user interface and a relational database for the data source, but what would change if the front-end part was a command-line interface? And what would change if, instead of a relational database, there was another type of data source, for example a set of text files?





As you can see, both changes would require the replacement of some components. After all, we need different code to manage a command line instead of a web page. But the external shape of the system doesn't change, neither does the way data flows. We created a system in which the user interface (web framework, command-line interface) and the data source (relational database, text files) are details of the implementation, and not core parts of it.

The main immediate advantage of a layered architecture, however, is testability. When you clearly separate components you clearly establish the data each of them has to receive and produce, so you can ideally disconnect a single component and test it in isolation. Let's take the Web framework component that we added and consider it for a moment forgetting the rest of the architecture. We can ideally connect a tester to its inputs and outputs as you can see in the figure



We know that the Web framework receives an HTTP request (1) with a specific target and a specific query string, and that it has to call (2) a method on the use case passing specific parameters. When the use case returns data (3), the Web framework has to convert that into an HTTP response (4). Since this is a test we can have a fake use case, that is an object that just mimics what the use case does without really implementing the business logic. We will then test that the Web framework calls the method (2) with the correct parameters, and that the HTTP response (4) contains the correct data in the proper format, and all this will happen without involving any other part of the system.

So, now that we had a 10,000 feet overview of the system, let's go deeper into its components and

the concepts behind them. In the next chapter I will detail how the design principles called “clean architecture” help to implement and use effectively concepts like separation of concerns, abstraction, implementation, and inversion of control.

Chapter 02 Components of a clean architecture

Components of a clean architecture



Wait a minute. Wait a minute Doc, uh, are you telling me you built a time machine... out of a DeLorean?

Back to the Future, 1985

In this chapter I will analyse the set of software design principles collectively known as “clean architecture”. While this specific name has been introduced by Robert Martin, the concepts it pushes are part of software engineering, and have been successfully used for decades.

Before we dive into a possible implementation of them, which is the core of this book, we need to analyse more in depth the structure of the clean architecture and the components you can find in the system designed following it.

Divide et impera

One of the main goals of a well designed system is to achieve control. From this point of view, a software system is not different from a human working community, like an office or a factory. In such environments there are workers who exchange data or physical objects to create and deliver a final product, be it an object or a service. Workers need information and resources to perform their own job, but most of all they need to have a clear picture of their responsibilities.

While in a human society we value initiative and creativity, however, in a machine such as a software system, components shouldn't be able to do anything that is not clearly stated when the system is designed. Software is not alive, and despite the impressive achievements of artificial intelligence in the latter years, I still believe there is a spark in a human being that cannot be reproduced by code alone.

Whatever our position on AIs, I think we all agree that a system works better if responsibilities are clear. Whether we are dealing with software or human communities, it is always dangerous to be unclear about what a component can or should do, as areas of influence and control naturally overlap. This can lead to all sorts of issues, from simple inefficiencies to complete deadlocks.

A good way to increase order and control in a system is to split it into subsystems, establishing clear and rigid borders between them, to regulate the data exchange. This is an extension of a political

concept (divide et impera) which states that it is simpler to rule a set of interconnected small systems than a single complex one.

In the system we designed in the previous chapter, it is always clear what a component expects to receive when called into play, and it is also impossible (or at least, forbidden) to exchange data in a way that breaks the structure of the system.

You have to remember that a software system is not exactly like a factory or an office. Whenever we discuss machines we have to consider both the way they work (run time) and the way they have been built or will be modified (development time). In principle, computers don't care where data comes from and where it goes. Humans, on the other hand, who have to build and maintain the system, need a clear picture of the data flow to avoid introducing bugs or killing performances.

Data types

An important part in a system is played by data types, that is the way we encapsulate and transmit information. In particular, when we discuss software systems, we need to make sure that types that are shared by different systems are known to all of them. The knowledge of data types and formats is, indeed, a form of coupling. Think about human languages: if you have to talk to an audience, you have to use a language they understand, and this makes you coupled with your audience. This book is written (tentatively) in English, which means that I am coupled with English-speaking readers. If all English speakers in the world suddenly decided to forget the language and replace it with Italian I should write the book from scratch (but with definitely less effort).

When we consider a software system, thus, we need to understand which part defines the types and the data format (the “language”), and ensure that the resulting dependencies don't get in the way of the implementer. In the previous chapter we discovered that there are components in the system that should be considered of primary importance and represent the core of the system (use cases), and others which are less central, often considered implementation details. Again, mind that calling them “details” doesn't mean they are not important or that they are trivial to implement, but that replacing them with different implementations does not affect the core of the system (business logic).

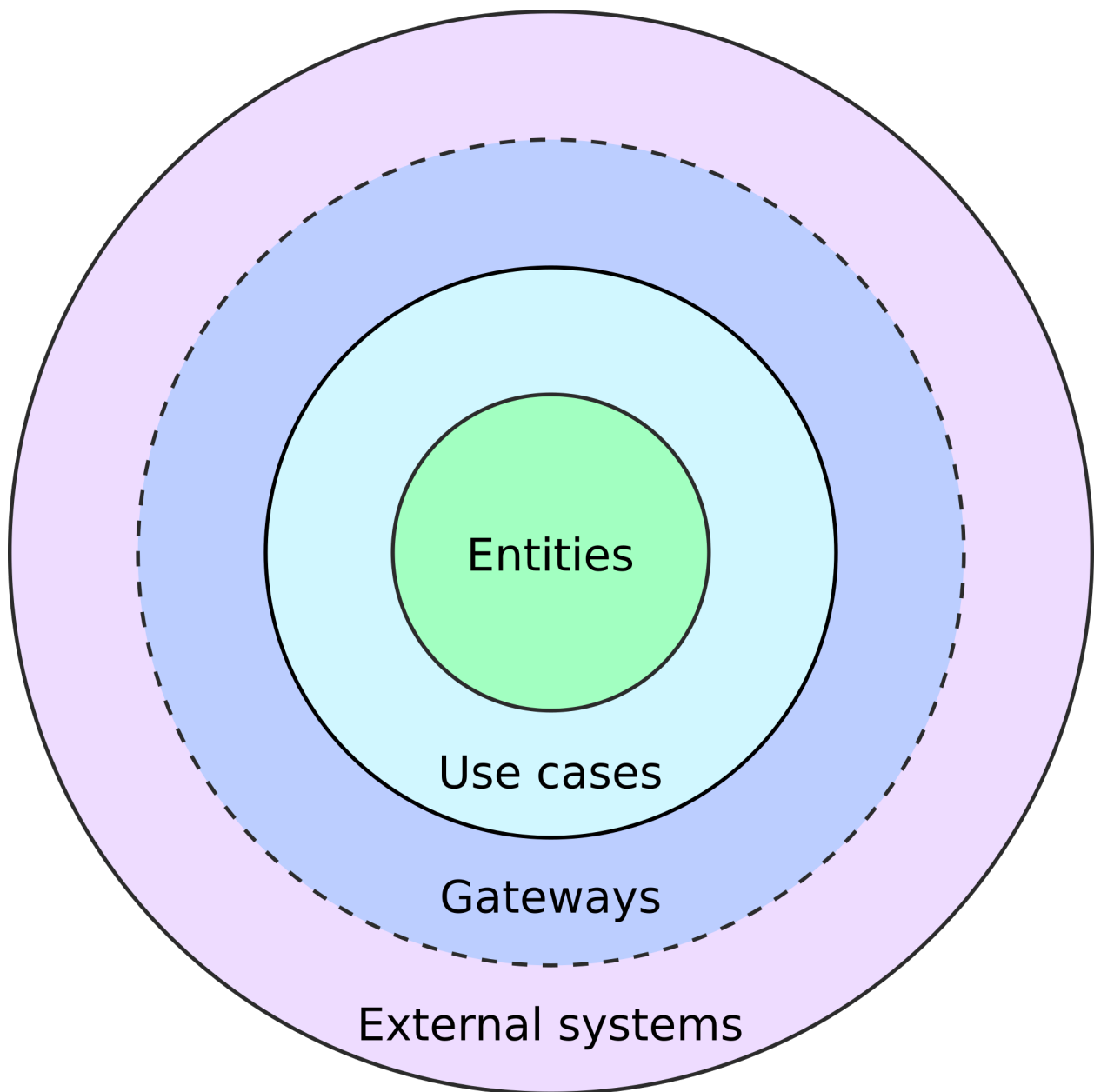
So, there is a hierarchy of components that spawns from the dependencies between them. Some components are defined at the very beginning of the design and do not depend on any other component, while others will come later and depend on them. When data types are involved, the resulting dependencies cannot break this hierarchy, as this would re-introduce a coupling between components that we want to avoid.

Let's go back to the initial example of a shop that buys items from a wholesale, displays them on shelves, and sells them to customers. There is a clear dependency between two components here: the component called “shop” depends on the component called “wholesale”, as the data (“items”) flow from the latter to the former. The size of the shelves in the shop, in turn, depends on the size of the items (types), which is defined by the wholesale, and this follows the dependency we already established.

If the size of the items was defined by the shop, suddenly there would be another dependency opposing the one we already established, making the wholesale depend on the shop. Please note that when it comes to software systems this is not a circular dependency, because the first one is a conceptual dependency while the second one happens at the language level at compile time. At any rate, having two opposite dependencies is definitely confusing, and makes it hard to replace “peripheral” components such as the shop.

The main four layers

The clean architecture tries to capture both the *conceptual hierarchy* of components and the *type hierarchy* through a layered approach. In a clean architecture the components of the system are categorised and belong to a specific layer, with rules relative to the communication between components belonging to the same or to different layers. In particular, a clean architecture is a spherical structure, with inner (lower) layers completely encompassed by outer (higher) ones, and the former being oblivious of the existence of the latter.



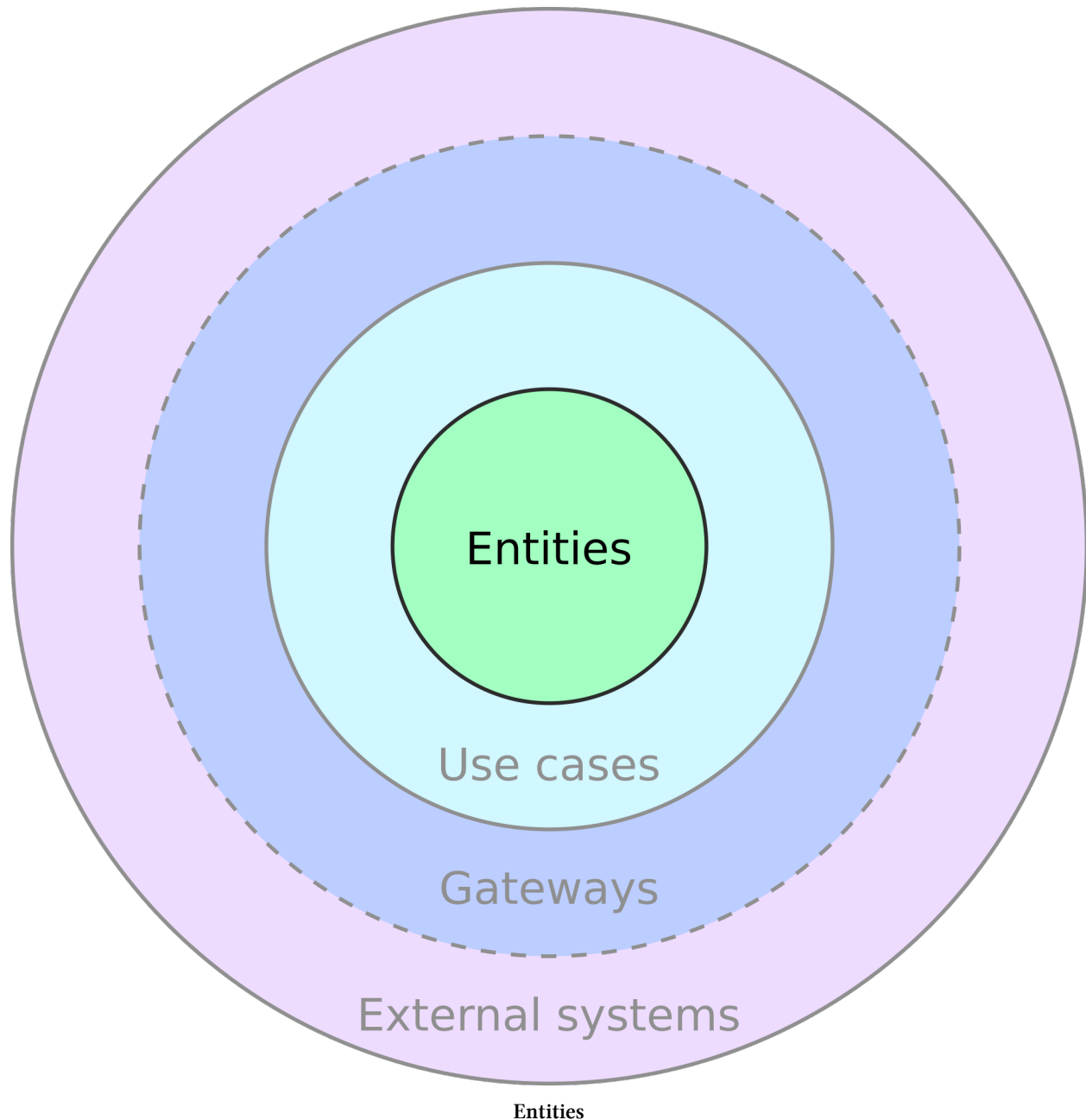
The basic layers of the clean architecture

Remember that in computer science, the words “lower” and “higher” almost always refer to the level of abstraction, and not to the importance of a component for the system. Each part of a system is important, otherwise it would not be there.

Let’s have a look at the main layers depicted in the figure, keeping in mind that a specific implementation may require to create new layers or to split some of these into multiple ones.

Entities

This layer of the clean architecture contains a representation of the domain models, that is everything your system needs to interact with and is sufficiently complex to require a specific representation. For example, strings in Python are complex and very powerful objects. They provide many methods out of the box, so in general, it is useless to create a domain model for them. If your project was a tool to analyse medieval manuscripts, however, you might need to isolate sentences and their features, and at this point it might be reasonable to define a specific entity.



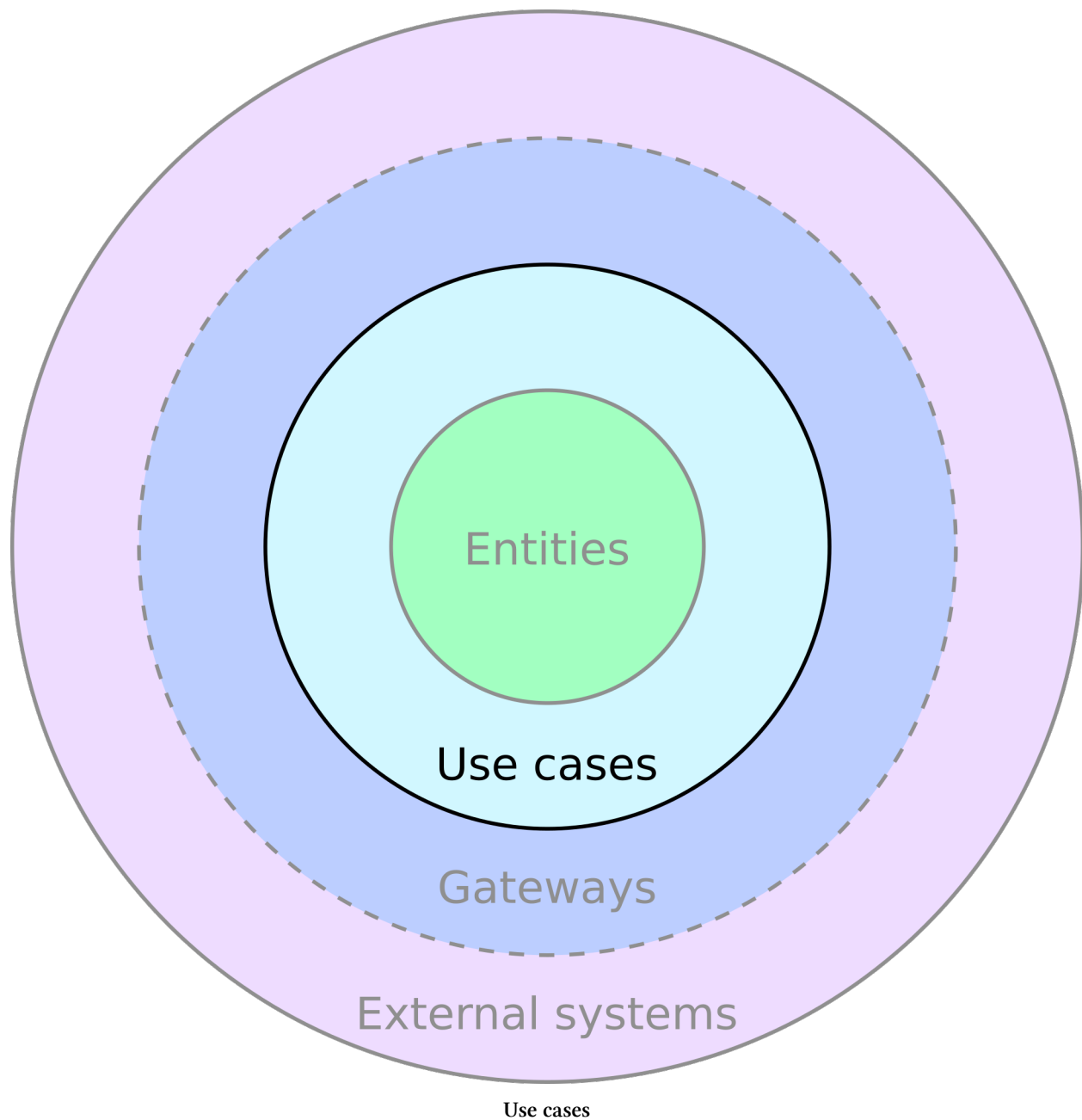
Since we work in Python, this layer will likely contain classes, with methods that simplify the interaction with them. It is very important, however, to understand that the models in this layer are different from the usual models of frameworks like Django. These models are not connected with a storage system, so they cannot be directly saved or queried using their own methods, they don't contain methods to dump themselves to JSON strings, they are not connected with any presentation layer. They are so-called lightweight models.

This is the inmost layer. Entities have mutual knowledge since they live in the same layer, so the architecture allows them to interact directly. This means that one of the Python classes that represent an entity can use another one directly, instantiating it and calling its methods. Entities don't know anything that lives in outer layers, though. They cannot call the database, access methods provided by the presentation framework, or instantiate use cases.

The entities layer provides a solid foundation of types that the outer layers can use to exchange data, and they can be considered the vocabulary of your business.

Use cases

As we said before the most important part of a clean system are use cases, as they implement the business rules, which are the core reason of existence of the system itself. Use cases are the processes that happen in your application, where you use your domain models to work on real data. Examples can be a user logging in, a search with specific filters being performed, or a bank transaction happening when the user wants to buy the content of the cart.

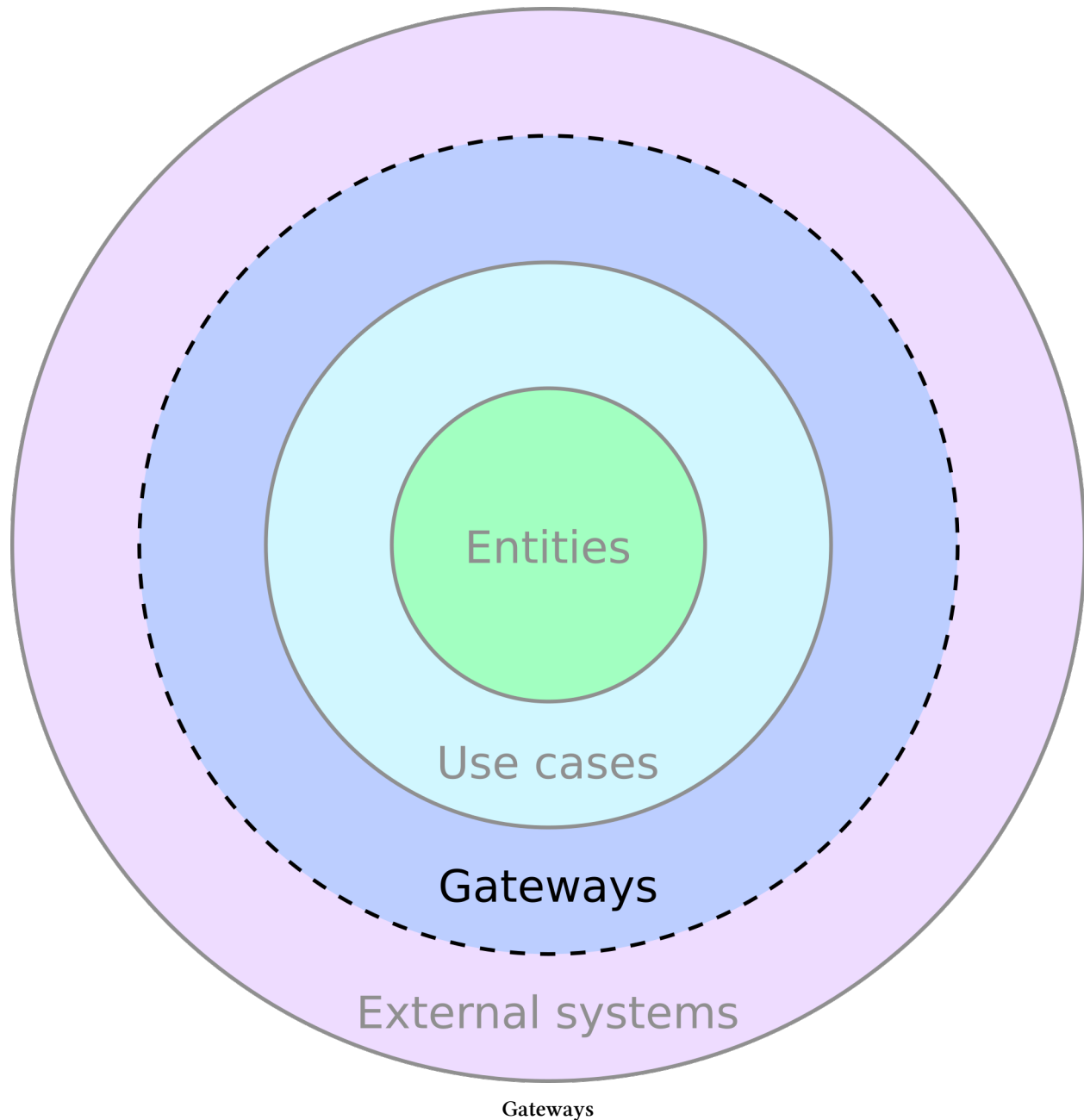


Use cases should be as small as possible. It is very important to isolate small actions into separate use cases, as this makes the whole system easier to test, understand and maintain. Use cases have full access to the entities layer, so they can instantiate and use them directly. They can also call each other, and it is common to create complex use cases composing simple ones.

Gateways

This layer contains components that define interfaces for external systems, that is a common access model to services that do not implement the business rules. The classic example is that of a data

storage, which internal details can be very different across implementations. These implementations share a common interface, otherwise they would not be implementations of the same concept, and the gateway's task is to expose it.

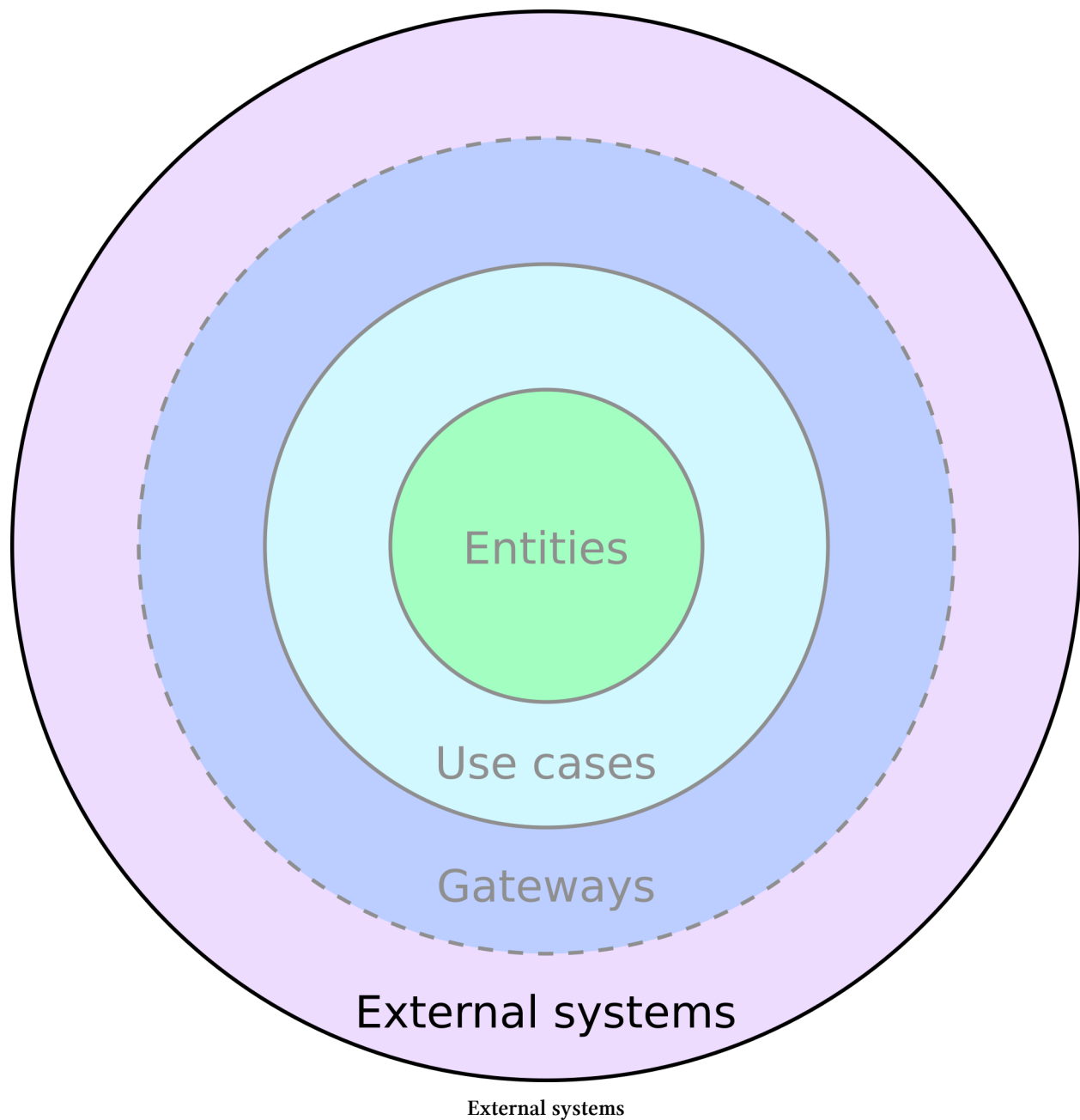


If you recall the simple example I started with, this is where the database interface would live. Gateways have access to entities, so the interface can freely receive and return objects which type has been defined in that layer, as they can freely access use cases. Gateways are used to mask the implementation of external systems, however, so it is rare for a gateway to call a use case, as this can be done by the external system itself. The gateways layer is intimately connected with the external

systems one, which is why the two are separated by a dashed line.

External systems

This part of the architecture is populated by components that implement the interfaces defined in the previous layer. The same interface might be implemented by one or more concrete components, as your system might want to support multiple implementations of that interface at the same time. For example, you might want to expose some use cases both through an HTTP API and a command line interface, or you want to provide support for different types of storage according to some configuration value.



Please remember that the “external” adjective doesn’t always mean that the system is developed by others, or that it is a complex system like a web framework or a database. The word has a topological meaning, which shows that the system we are talking about is peripheral to the core of the architecture, that is it doesn’t implement business logic. So we might want to use a messaging system developed in-house to send notifications to the clients of a certain service, but this is again just a presentation layer, unless our business is specifically centred around creating notification systems.

External systems have full access to gateways, use cases, and entities. While it is easy to understand

the relationship with gateways, which are created to wrap specific systems, it might be less clear what external systems should do with use cases and entities. As for use cases, external systems are usually the parts of the system that trigger them, being the way users run the business logic. A user clicking on a button, visiting a URL, or running a command, are typical examples of interactions with an external system that runs a use case directly. As for entities, an external system can directly process them, for example to return them in a JSON payload, or to map input data into a domain model.

I want to point out a difference between external systems that are used by use cases and external systems that want to call use cases. In the first case the direction of the communication is outwards, and we know that in the clean architecture we can't go outwards without interfaces. Thus, when we access an external system from a use case we always need an interface. When the external system wants to call use cases, instead, the direction of the communication is inwards, and this is allowed directly, as external layers have full access to the internal ones.

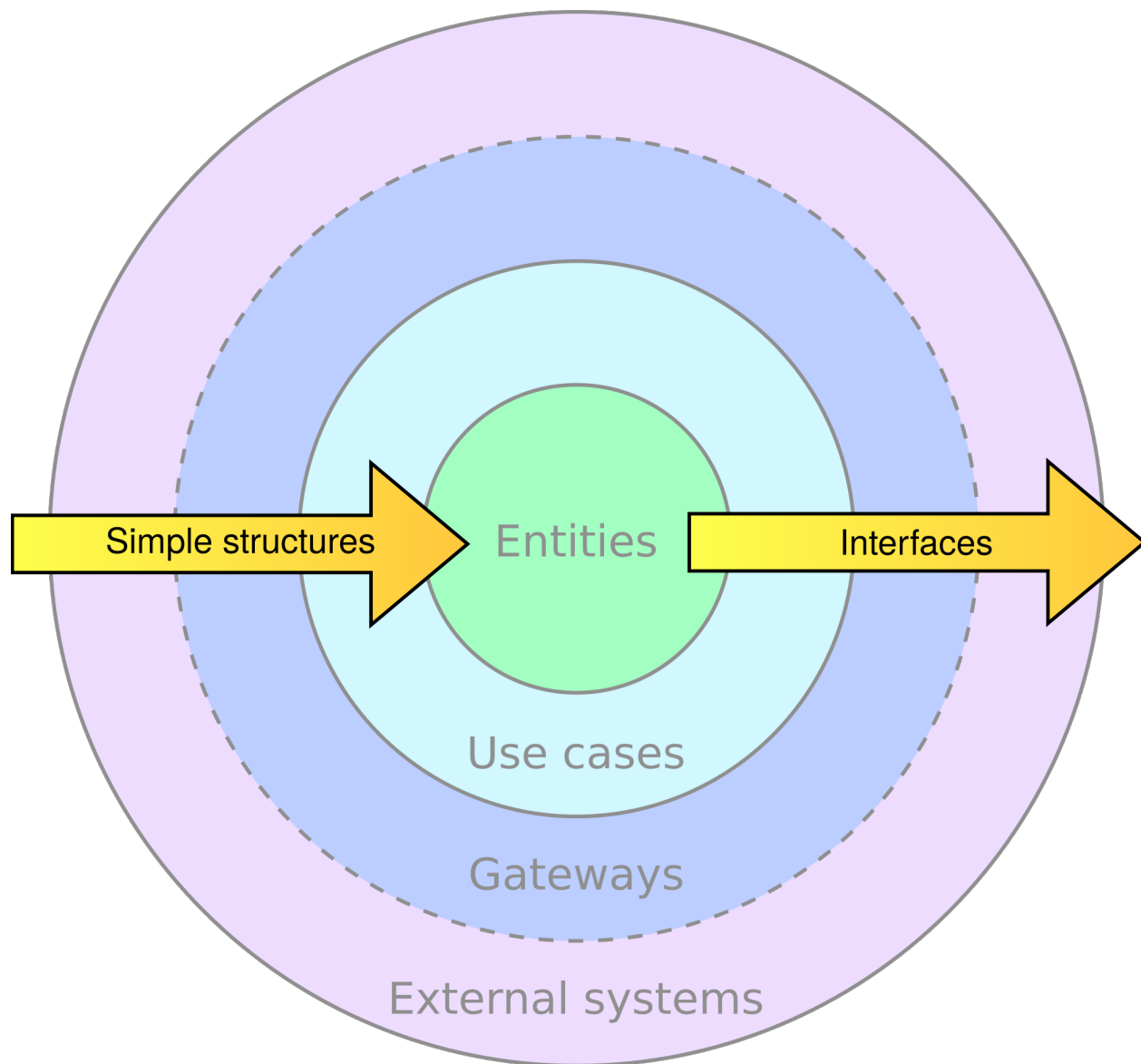
This, practically speaking, translates into two extreme cases, well represented by a database and a web framework. When a use case accesses a storage system there should be a loose coupling between the two, which is why we wrap the storage with an interface and assume that in the use case. When the web framework calls a use case, instead, the code of the endpoint doesn't need any interface to access it.

Communication between layers

The deeper a layer is in this architecture, the more abstract the content is. The inner layers contain representations of business concepts, while the outer layers contain specific details about the real-life implementation. The communication between elements that live in the same layer is unrestricted, but when you want to communicate with elements that have been assigned to other layers you have to follow one simple rule. This rule is the most important thing in a clean architecture, possibly being the core expression of the clean architecture itself.

The Golden Rule: talk inwards with simple structures, talk outwards through interfaces.

Your elements should talk inwards, that is pass data to more abstract elements, using basic structures, that is entities and everything provided by the programming language you are using.



The golden rule of the clean architecture

Your elements should talk outwards using interfaces, that is using only the expected API of a component, without referring to a specific implementation. When an outer layer is created, elements living there will plug themselves into those interfaces and provide a practical implementation.

APIs and shades of grey

The word API is of uttermost importance in a clean architecture. Every layer may be accessed by elements living in inner layers by an API, that is a fixed¹⁶ collection of entry points (methods or objects).

¹⁶Unless you consider things like `sessionmaker_mock().query.assert_called_with(Room)` something attractive. And this was by far the simplest mock I had to write.

The separation between layers and the content of each layer is not always fixed and immutable. A well-designed system shall also cope with practical world issues such as performances, for example, or other specific needs. When designing an architecture it is very important to know “what is where and why”, and this is even more important when you “bend” the rules. Many issues do not have a black-or-white answer, and many decisions are “shades of grey”, that is it is up to you to justify why you put something in a given place.

Keep in mind, however, that you should not break the *structure* of the clean architecture, and be particularly very strict about the data flow. If you break the data flow, you are basically invalidating the whole structure. You should try as hard as possible not to introduce solutions that are based on a break in the data flow, but realistically speaking, if this saves money, do it.

If you do it, there should be a giant warning in your code and your documentation explaining why you did it. If you access an outer layer breaking the interface paradigm usually it is because of some performance issues, as the layered structure can add some overhead to the communications between elements. You should clearly tell other programmers that this happened, because if someone wants to replace the external layer with something different, they should know that there is direct access which is implementation-specific.

For the sake of example, let's say that a use case is accessing the storage layer through an interface, but this turns out to be too slow. You decide then to access directly the API of the specific database you are using, but this breaks the data flow, as now an internal layer (use cases) is accessing an outer one (external interfaces). If someone in the future wants to replace the specific database you are using with a different one, they have to be aware of this, as the new database probably won't provide the same API entry point with the same data.

If you end up breaking the data flow consistently maybe you should consider removing one layer of abstraction, merging the two layers that you are linking.

Chapter 03 A basic example

”

Joshua/WOPR: Wouldn't you prefer a good game of chess?

David: Later. Let's play Global Thermonuclear War.

Wargames, 1983

The goal of the “Rent-o-Matic” project is to create a simple search engine for a room renting company. Objects in the dataset (rooms) are described by some attributes and the search engine shall allow the user to set some filters to narrow the search.

A room is stored in the system through the following values:

- A unique identifier
- A size in square meters
- A renting price in Euro/day
- Latitude and longitude

The description is purposely minimal so that we can focus on the architectural problems and how to solve them. The concepts that I will show are then easily extendable to more complex cases.

As pushed by the clean architecture model, we are interested in separating the different layers of the system. Remember that there are multiple ways to implement the clean architecture concepts, and the code you can come up with strongly depends on what your language of choice allows you to do. The following is an example of clean architecture in Python, and the implementation of the models, use cases and other components that I will show is just one of the possible solutions.

Project setup

Clone the [project repository](https://github.com/pycbook/rentomatic)¹⁷ and move to the branch `second-edition`. The full solution is contained in the branch `second-edition-top`, and the tags I will mention are there. I strongly advise to code along and to resort to my tags only to spot errors.

```
1 $ git clone https://github.com/pycbook/rentomatic
2 $ cd rentomatic
3 $ git checkout --track origin/second-edition
```

Create a virtual environment following your preferred process and install the requirements

¹⁷<https://github.com/pycbook/rentomatic>

```
1 $ pip install -r requirements/dev.txt
```

You should at this point be able to run

```
1 $ pytest -svv
```

and get an output like

```
1 ===== test session starts =====
2 platform linux -- Python XXXX, pytest-XXXX, py-XXXX, pluggy-XXXX --
3 cabook/venv3/bin/python3
4 cachedir: .cache
5 rootdir: cabook/code/calc, inifile: pytest.ini
6 plugins: cov-XXXX
7 collected 0 items
8
9 ===== no tests ran in 0.02s =====
```

Later in the project you might want to see the output of the coverage check, so you can activate it with

```
1 $ pytest -svv --cov=rentomatic --cov-report=term-missing
```

In this chapter, I will not explicitly state when I run the test suite, as I consider it part of the standard workflow. Every time we write a test you should run the suite and check that you get an error (or more), and the code that I give as a solution should make the test suite pass. You are free to try to implement your own code before copying my solution, obviously.

You may notice that I configured the project to use black with an unorthodox line length of 75. I chose that number trying to find a visually pleasant way to present code in the book, avoiding wrapped lines that can make the code difficult to read.



Source code

<https://github.com/pycabook/rentomatic/tree/second-edition>

Domain models

Let us start with a simple definition of the model Room. As said before, the clean architecture models are very lightweight, or at least they are lighter than their counterparts in common web frameworks.

Following the TDD methodology, the first thing that I write are the tests. This test ensures that the model can be initialised with the correct values

tests/domain/test_room.py

```
1 import uuid
2 from rentomatic.domain.room import Room
3
4
5 def test_room_model_init():
6     code = uuid.uuid4()
7     room = Room(
8         code,
9         size=200,
10        price=10,
11        longitude=-0.09998975,
12        latitude=51.75436293,
13    )
14
15    assert room.code == code
16    assert room.size == 200
17    assert room.price == 10
18    assert room.longitude == -0.09998975
19    assert room.latitude == 51.75436293
```

Remember to create an empty file `__init__.py` in every subdirectory of `tests/` that you create, in this case `tests/domain/__init__.py`.

Now let's write the class `Room` in the file `rentomatic/domain/room.py`.

rentomatic/domain/room.py

```
1 import uuid
2 import dataclasses
3
4
5 @dataclasses.dataclass
6 class Room:
7     code: uuid.UUID
8     size: int
9     price: int
10    longitude: float
11    latitude: float
```



Source code

<https://github.com/pycabook/rentomatic/tree/ed2-c03-s01>

The model is very simple and requires little explanation. I'm using dataclasses as they are a compact way to implement simple models like this, but you are free to use standard classes and to implement the method `__init__` explicitly.

Given that we will receive data to initialise this model from other layers, and that this data is likely to be a dictionary, it is useful to create a method that allows us to initialise the model from this type of structure. The code can go into the same file we created before, and is

tests/domain/test_room.py

```
1 def test_room_model_from_dict():
2     code = uuid.uuid4()
3     init_dict = {
4         "code": code,
5         "size": 200,
6         "price": 10,
7         "longitude": -0.09998975,
8         "latitude": 51.75436293,
9     }
10
11     room = Room.from_dict(init_dict)
12
13     assert room.code == code
14     assert room.size == 200
15     assert room.price == 10
16     assert room.longitude == -0.09998975
17     assert room.latitude == 51.75436293
```

A simple implementation of it is then

rentomatic/domain/room.py

```
1 @dataclasses.dataclass
2 class Room:
3     code: uuid.UUID
4     size: int
5     price: int
6     longitude: float
7     latitude: float
8
9     @classmethod
10     def from_dict(cls, d):
11         return cls(**d)
```

**Source code**<https://github.com/pycabook/rentomatic/tree/ed2-c03-s02>

For the same reason mentioned before, it is useful to be able to convert the model into a dictionary, so that we can easily serialise it into JSON or similar language-agnostic formats. The test for the method `to_dict` goes again in `tests/domain/test_room.py`

tests/domain/test_room.py

```
1 def test_room_model_to_dict():
2     init_dict = {
3         "code": uuid.uuid4(),
4         "size": 200,
5         "price": 10,
6         "longitude": -0.09998975,
7         "latitude": 51.75436293,
8     }
9
10    room = Room.from_dict(init_dict)
11
12    assert room.to_dict() == init_dict
```

and the implementation is trivial using dataclasses

rentomatic/domain/room.py

```
1 def to_dict(self):
2     return dataclasses.asdict(self)
```

If you are not using dataclasses you need to explicitly create the dictionary, but that doesn't pose any challenge either. Note that this is not yet a serialisation of the object, as the result is still a Python data structure and not a string.

**Source code**<https://github.com/pycabook/rentomatic/tree/ed2-c03-s03>

It is also very useful to be able to compare instances of a model. The test goes in the same file as the previous test

tests/domain/test_room.py

```
1 def test_room_model_comparison():
2     init_dict = {
3         "code": uuid.uuid4(),
4         "size": 200,
5         "price": 10,
6         "longitude": -0.09998975,
7         "latitude": 51.75436293,
8     }
9
10    room1 = Room.from_dict(init_dict)
11    room2 = Room.from_dict(init_dict)
12
13    assert room1 == room2
```

Again, dataclasses make this very simple, as they provide an implementation of `__eq__` out of the box. If you implement the class without using dataclasses you have to define this method to make it pass the test.



Source code

<https://github.com/pycabook/rentomatic/tree/ed2-c03-s04>

Serializers

Outer layers can use the model `Room`, but if you want to return the model as a result of an API call you need a serializer.

The typical serialization format is JSON, as this is a broadly accepted standard for web-based APIs. The serializer is not part of the model but is an external specialized class that receives the model instance and produces a representation of its structure and values.

This is the test for the JSON serialization of our class `Room`

tests/serializers/test_room.py

```
1 import json
2 import uuid
3
4 from rentomatic.serializers.room import RoomJsonEncoder
5 from rentomatic.domain.room import Room
6
7
8 def test_serialize_domain_room():
9     code = uuid.uuid4()
10
11     room = Room(
12         code=code,
13         size=200,
14         price=10,
15         longitude=-0.09998975,
16         latitude=51.75436293,
17     )
18
19     expected_json = f"""
20         {{
21             "code": "{code}",
22             "size": 200,
23             "price": 10,
24             "longitude": -0.09998975,
25             "latitude": 51.75436293
26         }}
27     """
28
29     json_room = json.dumps(room, cls=RoomJsonEncoder)
30
31     assert json.loads(json_room) == json.loads(expected_json)
```

Here we create the object `Room` and write the expected JSON output (please note that the double curly braces are used to avoid clashes with the f-string formatter). Then we dump the object `Room` to a JSON string and compare the two. To compare the two we load them again into Python dictionaries, to avoid issues with the order of the attributes. Comparing Python dictionaries, indeed, doesn't consider the order of the dictionary fields, while comparing strings obviously does.

Put in the file `rentomatic/serializers/room.py` the code that makes the test pass

rentomatic/serializers/room.py

```
1 import json
2
3
4 class RoomJsonEncoder(json.JSONEncoder):
5     def default(self, o):
6         try:
7             to_serialize = {
8                 "code": str(o.code),
9                 "size": o.size,
10                "price": o.price,
11                "latitude": o.latitude,
12                "longitude": o.longitude,
13            }
14            return to_serialize
15        except AttributeError: # pragma: no cover
16            return super().default(o)
```



Source code

<https://github.com/pycabook/rentomatic/tree/ed2-c03-s05>

Providing a class that inherits from `json.JSONEncoder` let us use the syntax `json_room = json.dumps(room, cls=RoomJsonEncoder)` to serialize the model. Note that we are not using the method `as_dict`, as the UUID code is not directly JSON serialisable. This means that there is a slight degree of code repetition in the two classes, which in my opinion is acceptable, being covered by tests. If you prefer, however, you can call the method `as_dict` and then adjust the code field converting it with `str`.

Use cases

It's time to implement the actual business logic that runs inside our application. Use cases are the places where this happens, and they might or might not be directly linked to the external API of the system.

The simplest use case we can create is one that fetches all the rooms stored in the repository and returns them. In this first part, we will not implement the filters to narrow the search. That code will be introduced in the next chapter when we will discuss error management.

The repository is our storage component, and according to the clean architecture it will be implemented in an outer level (external systems). We will access it as an interface, which in Python means that we will receive an object that we expect will expose a certain API. From the testing point

of view the best way to run code that accesses an interface is to mock the latter. Put this code in the file `tests/use_cases/test_room_list.py`

I will make use of `pytest`'s powerful fixtures, but I will not introduce them. I highly recommend reading the [official documentation](https://docs.pytest.org/en/stable/fixture.html)¹⁸, which is very good and covers many different use cases.

`tests/use_cases/test_room_list.py`

```
1  import pytest
2  import uuid
3  from unittest import mock
4
5  from rentomatic.domain.room import Room
6  from rentomatic.use_cases.room_list import room_list_use_case
7
8
9  @pytest.fixture
10 def domain_rooms():
11     room_1 = Room(
12         code=uuid.uuid4(),
13         size=215,
14         price=39,
15         longitude=-0.09998975,
16         latitude=51.75436293,
17     )
18
19     room_2 = Room(
20         code=uuid.uuid4(),
21         size=405,
22         price=66,
23         longitude=0.18228006,
24         latitude=51.74640997,
25     )
26
27     room_3 = Room(
28         code=uuid.uuid4(),
29         size=56,
30         price=60,
31         longitude=0.27891577,
32         latitude=51.45994069,
33     )
34
35     room_4 = Room(
```

¹⁸<https://docs.pytest.org/en/stable/fixture.html>

```

36         code=uuid.uuid4(),
37         size=93,
38         price=48,
39         longitude=0.33894476,
40         latitude=51.39916678,
41     )
42
43     return [room_1, room_2, room_3, room_4]
44
45
46 def test_room_list_without_parameters(domain_rooms):
47     repo = mock.Mock()
48     repo.list.return_value = domain_rooms
49
50     result = room_list_use_case(repo)
51
52     repo.list.assert_called_with()
53     assert result == domain_rooms

```

The test is straightforward. First, we mock the repository so that it provides a method `list` that returns the list of models we created above the test. Then we initialise the use case with the repository and execute it, collecting the result. The first thing we check is that the repository method was called without any parameter, and the second is the effective correctness of the result.

Calling the method `list` of the repository is an outgoing query action that the use case is supposed to perform, and according to the unit testing rules, we should not test outgoing queries. We should, however, test how our system runs the outgoing query, that is the parameters used to run the query.

Put the implementation of the use case in the file `rentomatic/use_cases/room_list.py`

rentomatic/use_cases/room_list.py

```

1 def room_list_use_case(repo):
2     return repo.list()

```

Such a solution might seem too simple, so let's discuss it. First of all, this use case is just a wrapper around a specific function of the repository, and it doesn't contain any error check, which is something we didn't take into account yet. In the next chapter, we will discuss requests and responses, and the use case will become slightly more complicated.

The next thing you might notice is that I used a simple function. In the first edition of this book I used a class for the use case, and thanks to the nudge of a couple of readers I started to question my choice, so I want to briefly discuss the options you have.

The use case represents the business logic, a process, which means that the simplest implementation you can have in a programming language is a function: some code that receives input arguments and

returns output data. A class is however another option, as in essence it is a collection of variables and functions. So, as in many other cases, the question is if you should use a function or a class, and my answer is that it depends on the degree of complexity of the algorithm that you are implementing.

Your business logic might be complicated, and require the connection with several external systems, though, each one with a specific initialisation, while in this simple case I just pass in the repository. So, in principle, I don't see anything wrong in using classes for use cases, should you need more structure for your algorithms, but be careful not to use them when a simpler solution (functions) can perform the same job, which is the mistake I made in the previous version of this code. Remember that code has to be maintained, so the simpler it is, the better.



Source code

<https://github.com/pycabook/rentomatic/tree/ed2-c03-s06>

The storage system

During the development of the use case, we assumed it would receive an object that contains the data and exposes a `list` function. This object is generally nicknamed “repository”, being the source of information for the use case. It has nothing to do with the Git repository, though, so be careful not to mix the two nomenclatures.

The storage lives in the fourth layer of the clean architecture, the external systems. The elements in this layer are accessed by internal elements through an interface, which in Python just translates to exposing a given set of methods (in this case only `list`). It is worth noting that the level of abstraction provided by a repository in a clean architecture is higher than that provided by an ORM in a framework or by a tool like SQLAlchemy. The repository provides only the endpoints that the application needs, with an interface which is tailored to the specific business problems the application implements.

To clarify the matter in terms of concrete technologies, SQLAlchemy is a wonderful tool to abstract the access to an SQL database, so the internal implementation of the repository could use it to access a PostgreSQL database, for example. But the external API of the layer is not that provided by SQLAlchemy. The API is a reduced set of functions that the use cases call to get the data, and the internal implementation can use a wide range of solutions to achieve the same goal, from raw SQL queries to a complex system of remote calls through a RabbitMQ network.

A very important feature of the repository is that it can return domain models, and this is in line with what framework ORMs usually do. The elements in the third layer have access to all the elements defined in the internal layers, which means that domain models and use cases can be called and used directly from the repository.

For the sake of this simple example, we will not deploy and use a real database system. Given what we said, we are free to implement the repository with the system that better suits our needs, and

in this case I want to keep everything simple. We will thus create a very simple in-memory storage system loaded with some predefined data.

The first thing to do is to write some tests that document the public API of the repository. The file containing the tests is `tests/repository/test_memrepo.py`.

`tests/repository/test_memrepo.py`

```
1  import pytest
2
3  from rentomatic.domain.room import Room
4  from rentomatic.repository.memrepo import MemRepo
5
6
7  @pytest.fixture
8  def room_dicts():
9      return [
10         {
11             "code": "f853578c-fc0f-4e65-81b8-566c5dffa35a",
12             "size": 215,
13             "price": 39,
14             "longitude": -0.09998975,
15             "latitude": 51.75436293,
16         },
17         {
18             "code": "fe2c3195-aeff-487a-a08f-e0bdc0ec6e9a",
19             "size": 405,
20             "price": 66,
21             "longitude": 0.18228006,
22             "latitude": 51.74640997,
23         },
24         {
25             "code": "913694c6-435a-4366-ba0d-da5334a611b2",
26             "size": 56,
27             "price": 60,
28             "longitude": 0.27891577,
29             "latitude": 51.45994069,
30         },
31         {
32             "code": "eed76e77-55c1-41ce-985d-ca49bf6c0585",
33             "size": 93,
34             "price": 48,
35             "longitude": 0.33894476,
36             "latitude": 51.39916678,
37         },
38     ]
```

```

38     ]
39
40
41 def test_repository_list_without_parameters(room_dicts):
42     repo = MemRepo(room_dicts)
43
44     rooms = [Room.from_dict(i) for i in room_dicts]
45
46     assert repo.list() == rooms

```

In this case, we need a single test that checks the behaviour of the method `list`. The implementation that passes the test goes in the file `rentomatic/repository/memrepo.py`

`rentomatic/repository/memrepo.py`

```

1 from rentomatic.domain.room import Room
2
3
4 class MemRepo:
5     def __init__(self, data):
6         self.data = data
7
8     def list(self):
9         return [Room.from_dict(i) for i in self.data]

```



Source code

<https://github.com/pycabook/rentomatic/tree/ed2-c03-s07>

You can easily imagine this class being the wrapper around a real database or any other storage type. While the code might become more complex, its basic structure would remain the same, with a single public method `list`. I will dig into database repositories in a later chapter.

A command-line interface

So far we created the domain models, the serializers, the use cases and the repository, but we are still missing a system that glues everything together. This system has to get the call parameters from the user, initialise a use case with a repository, run the use case that fetches the domain models from the repository, and return them to the user.

Let's see now how the architecture that we just created can interact with an external system like a CLI. The power of a clean architecture is that the external systems are pluggable, which means that

we can defer the decision about the detail of the system we want to use. In this case, we want to give the user an interface to query the system and to get a list of the rooms contained in the storage system, and the simplest choice is a command-line tool.

Later we will create a REST endpoint and we will expose it through a Web server, and it will be clear why the architecture that we created is so powerful.

For the time being, create a file `cli.py` in the same directory that contains `setup.cfg`. This is a simple Python script that doesn't need any specific option to run, as it just queries the storage for all the domain models contained there. The content of the file is the following

`cli.py`

```
1  #!/usr/bin/env python
2
3  from rentomatic.repository.memrepo import MemRepo
4  from rentomatic.use_cases.room_list import room_list_use_case
5
6  repo = MemRepo([])
7  result = room_list_use_case(repo)
8
9  print(result)
```



Source code

<https://github.com/pycabook/rentomatic/tree/ed2-c03-s08>

You can execute this file with `python cli.py` or, if you prefer, run `chmod +x cli.py` (which makes it executable) and then run it with `./cli.py` directly. The expected result is an empty list

```
1  $ ./cli.py
2  []
```

which is correct as the class `MemRepo` in the file `cli.py` has been initialised with an empty list. The simple in-memory storage that we use has no persistence, so every time we create it we have to load some data in it. This has been done to keep the storage layer simple, but keep in mind that if the storage was a proper database this part of the code would connect to it but there would be no need to load data in it.

The most important part of the script is

cli.py

```
1 repo = MemRepo([])
2 result = room_list_use_case(repo)
```

which initialises the repository and runs the use case. This is in general how you end up using your clean architecture and whatever external system you will plug into it. You initialise other systems, run the use case passing the interfaces, and you collect the results.

For the sake of demonstration, let's define some data in the file and load them in the repository

cli.py

```
1 #!/usr/bin/env python
2
3 from rentomatic.repository.memrepo import MemRepo
4 from rentomatic.use_cases.room_list import room_list_use_case
5
6 rooms = [
7     {
8         "code": "f853578c-fc0f-4e65-81b8-566c5dffa35a",
9         "size": 215,
10        "price": 39,
11        "longitude": -0.09998975,
12        "latitude": 51.75436293,
13    },
14    {
15        "code": "fe2c3195-aeff-487a-a08f-e0bdc0ec6e9a",
16        "size": 405,
17        "price": 66,
18        "longitude": 0.18228006,
19        "latitude": 51.74640997,
20    },
21    {
22        "code": "913694c6-435a-4366-ba0d-da5334a611b2",
23        "size": 56,
24        "price": 60,
25        "longitude": 0.27891577,
26        "latitude": 51.45994069,
27    },
28    {
29        "code": "eed76e77-55c1-41ce-985d-ca49bf6c0585",
30        "size": 93,
31        "price": 48,
```

```

32         "longitude": 0.33894476,
33         "latitude": 51.39916678,
34     },
35 ]
36
37 repo = MemRepo(rooms)
38 result = room_list_use_case(repo)
39
40 print([room.to_dict() for room in result])

```



Source code

<https://github.com/pycabook/rentomatic/tree/ed2-c03-s09>

Again, remember that we need to hardcode data due to the trivial nature of our storage, and not to the architecture of the system. Note that I changed the instruction `print` as the repository returns domain models and printing them would result in a list of strings like `<rentomatic.domain.room.Room object at 0x7fb815ec04e0>`, which is not really helpful.

If you run the command line tool now, you will get a richer result than before

```

1  $ ./cli.py
2  [
3      {
4          'code': 'f853578c-fc0f-4e65-81b8-566c5dffa35a',
5          'size': 215,
6          'price': 39,
7          'longitude': -0.09998975,
8          'latitude': 51.75436293
9      },
10     {
11         'code': 'fe2c3195-aeff-487a-a08f-e0bdc0ec6e9a',
12         'size': 405,
13         'price': 66,
14         'longitude': 0.18228006,
15         'latitude': 51.74640997
16     },
17     {
18         'code': '913694c6-435a-4366-ba0d-da5334a611b2',
19         'size': 56,
20         'price': 60,
21         'longitude': 0.27891577,

```

```
22     'latitude': 51.45994069
23 },
24 {
25     'code': 'eed76e77-55c1-41ce-985d-ca49bf6c0585',
26     'size': 93,
27     'price': 48,
28     'longitude': 0.33894476,
29     'latitude': 51.39916678
30 }
31 ]
```

Please note that I formatted the output above to be more readable, but the actual output will be on a single line.

What we saw in this chapter is the core of the clean architecture in action.

We explored the standard layers of entities (the class `Room`), use cases (the function `room_list_use_case`), gateways and external systems (the class `MemRepo`) and we could start to appreciate the advantages of their separation into layers.

Arguably, what we designed is very limited, which is why I will dedicate the rest of the book to showing how to enhance what we have to deal with more complicated cases. We will discuss a **Web interface** in chapter 4, a **richer query language** and **error management** in chapter 5, and the **integration with real external systems** like databases in chapters 6, 7, and 8.

Chapter 04 Add a web application

”

For your information, Hairdo, a major network is interested in me.

Groundhog Day, 1993

In this chapter, I will go through the creation of an HTTP endpoint for the room list use case. An HTTP endpoint is a URL exposed by a Web server that runs a specific logic and returns values in a standard format.

I will follow the REST recommendation, so the endpoint will return a JSON payload. REST is however not part of the clean architecture, which means that you can choose to model your URLs and the format of returned data according to whatever scheme you prefer.

To expose the HTTP endpoint we need a web server written in Python, and in this case, I chose Flask. Flask is a lightweight web server with a modular structure that provides just the parts that the user needs. In particular, we will not use any database/ORM, since we already implemented our own repository layer.

Flask setup

Let us start updating the requirements files. The file `requirements/prod.txt` shall mention Flask, as this package contains a script that runs a local webserver that we can use to expose the endpoint

requirements/prod.txt

1 Flask

The file `requirements/test.txt` will contain the pytest extension to work with Flask (more on this later)

requirements/test.txt

```
1 -r prod.txt
2 pytest
3 tox
4 coverage
5 pytest-cov
6 pytest-flask
```

**Source code**<https://github.com/pycabook/rentomatic/tree/ed2-c04-s01>

Remember to run `pip install -r requirements/dev.txt` again after those changes to install the new packages in your virtual environment.

The setup of a Flask application is not complex, but there are a lot of concepts involved, and since this is not a tutorial on Flask I will run quickly through these steps. I will provide links to the Flask documentation for every concept, though. If you want to dig a bit deeper in this matter you can read my series of posts [Flask Project Setup: TDD, Docker, Postgres and more](#)¹⁹.

The Flask application can be configured using a plain Python object ([documentation](#)²⁰), so I created the file `application/config.py` that contains this code

application/config.py

```
1 import os
2
3 basedir = os.path.abspath(os.path.dirname(__file__))
4
5
6 class Config(object):
7     """Base configuration"""
8
9
10 class ProductionConfig(Config):
11     """Production configuration"""
12
13
14 class DevelopmentConfig(Config):
15     """Development configuration"""
16
17
```

¹⁹<https://www.thedigitalcatonline.com/blog/2020/07/05/flask-project-setup-tdd-docker-postgres-and-more-part-1/>

²⁰http://flask.pocoo.org/docs/latest/api/#flask.Config.from_object

```
18 class TestingConfig(Config):
19     """Testing configuration"""
20
21     TESTING = True
```

Read [this page](#)²¹ to know more about Flask configuration parameters.

Now we need a function that initialises the Flask application ([documentation](#)²²), configures it, and registers the blueprints ([documentation](#)²³). The file `application/app.py` contains the following code, which is an app factory

application/app.py

```
1 from flask import Flask
2
3 from application.rest import room
4
5
6 def create_app(config_name):
7
8     app = Flask(__name__)
9
10    config_module = f"application.config.{config_name.capitalize()}Config"
11
12    app.config.from_object(config_module)
13
14    app.register_blueprint(room.blueprint)
15
16    return app
```



Source code

<https://github.com/pycabook/rentomatic/tree/ed2-c04-s02>

Test and create an HTTP endpoint

Before we create the proper setup of the webserver, we want to create the endpoint that will be exposed. Endpoints are ultimately functions that are run when a user sends a request to a certain URL, so we can still work with TDD, as the final goal is to have code that produces certain results.

²¹<http://flask.pocoo.org/docs/latest/config/>

²²<http://flask.pocoo.org/docs/latest/patterns/appfactories/>

²³<http://flask.pocoo.org/docs/latest/blueprints/>

The problem we have testing an endpoint is that we need the webserver to be up and running when we hit the test URLs. The webserver itself is an external system so we won't test it, but the code that provides the endpoint is part of our application²⁴. It is actually a gateway, that is an interface that allows an HTTP framework to access the use cases.

The extension `pytest-flask` allows us to run Flask, simulate HTTP requests, and test the HTTP responses. This extension hides a lot of automation, so it might be considered a bit “magic” at a first glance. When you install it some fixtures like `client` are available automatically, so you don't need to import them. Moreover, it tries to access another fixture named `app` that you have to define. This is thus the first thing to do.

Fixtures can be defined directly in your test file, but if we want a fixture to be globally available the best place to define it is the file `conftest.py` which is automatically loaded by `pytest`. As you can see there is a great deal of automation, and if you are not aware of it you might be surprised by the results, or frustrated by the errors.

tests/conftest.py

```
1 import pytest
2
3
4 from application.app import create_app
5
6
7 @pytest.fixture
8 def app():
9     app = create_app("testing")
10
11     return app
```

The function `app` runs the app factory to create a Flask app, using the configuration `testing`, which sets the flag `TESTING` to `True`. You can find the description of these flags in the [official documentation](#)²⁵.

At this point, we can write the test for our endpoint.

²⁴Unless you consider things like `sessionmaker_mock().query.assert_called_with(Room)` something attractive. And this was by far the simplest mock I had to write.

²⁵<http://flask.pocoo.org/docs/1.0/config/>

tests/rest/test_room.py

```
1 import json
2 from unittest import mock
3
4 from rentomatic.domain.room import Room
5
6 room_dict = {
7     "code": "3251a5bd-86be-428d-8ae9-6e51a8048c33",
8     "size": 200,
9     "price": 10,
10    "longitude": -0.09998975,
11    "latitude": 51.75436293,
12 }
13
14 rooms = [Room.from_dict(room_dict)]
15
16
17 @mock.patch("application.rest.room.room_list_use_case")
18 def test_get(mock_use_case, client):
19     mock_use_case.return_value = rooms
20
21     http_response = client.get("/rooms")
22
23     assert json.loads(http_response.data.decode("UTF-8")) == [room_dict]
24     mock_use_case.assert_called()
25     assert http_response.status_code == 200
26     assert http_response.mimetype == "application/json"
```

Let's comment it section by section.

tests/rest/test_room.py

```
1 import json
2 from unittest import mock
3
4 from rentomatic.domain.room import Room
5
6 room_dict = {
7     "code": "3251a5bd-86be-428d-8ae9-6e51a8048c33",
8     "size": 200,
9     "price": 10,
10    "longitude": -0.09998975,
11    "latitude": 51.75436293,
```

```
12 }  
13  
14 rooms = [Room.from_dict(room_dict)]
```

The first part contains some imports and sets up a room from a dictionary. This way we can later directly compare the content of the initial dictionary with the result of the API endpoint. Remember that the API returns JSON content, and we can easily convert JSON data into simple Python structures, so starting from a dictionary will come in handy.

tests/rest/test_room.py

```
1 @mock.patch("application.rest.room.room_list_use_case")  
2 def test_get(mock_use_case, client):
```

This is the only test that we have for the time being. During the whole test, we mock the use case, as we are not interested in running it, as it has been already tested elsewhere. We are however interested in checking the arguments passed to the use case, and a mock can provide this information. The test receives the mock from the decorator `patch` and the fixture `client`, which is one of the fixtures provided by `pytest-flask`. The fixture automatically loads `app`, which we defined in `conftest.py`, and is an object that simulates an HTTP client that can access the API endpoints and store the responses of the server.

tests/rest/test_room.py

```
1 mock_use_case.return_value = rooms  
2  
3 http_response = client.get("/rooms")  
4  
5 assert json.loads(http_response.data.decode("UTF-8")) == [room_dict]  
6 mock_use_case.assert_called()  
7 assert http_response.status_code == 200  
8 assert http_response.mimetype == "application/json"
```

The first line initialises the mock use case, instructing it to return the fixed `rooms` variable that we created previously. The central part of the test is the line where we get the API endpoint, which sends an HTTP GET request and collects the server's response.

After this, we check that the data contained in the response is a JSON that contains the data in the structure `room_dict`, that the method `use_case` has been called, that the HTTP response status code is 200, and last that the server sends the correct MIME type back.

It's time to write the endpoint, where we will finally see all the pieces of the architecture working together, as they did in the little CLI program that we wrote previously. Let me show you a template for the minimal Flask endpoint we can create

```

1 blueprint = Blueprint('room', __name__)
2
3
4 @blueprint.route('/rooms', methods=['GET'])
5 def room_list():
6     [LOGIC]
7     return Response([JSON DATA],
8                     mimetype='application/json',
9                     status=[STATUS])

```

As you can see the structure is really simple. Apart from setting the blueprint, which is the way Flask registers endpoints, we create a simple function that runs the endpoint, and we decorate it assigning the endpoint /rooms that serves GET requests. The function will run some logic and eventually return a Response that contains JSON data, the correct MIME type, and an HTTP status that represents the success or failure of the logic.

The above template becomes the following code

application/rest/room.py

```

1 import json
2
3 from flask import Blueprint, Response
4
5 from rentomatic.repository.memrepo import MemRepo
6 from rentomatic.use_cases.room_list import room_list_use_case
7 from rentomatic.serializers.room import RoomJsonEncoder
8
9 blueprint = Blueprint("room", __name__)
10
11 rooms = [
12     {
13         "code": "f853578c-fc0f-4e65-81b8-566c5dffa35a",
14         "size": 215,
15         "price": 39,
16         "longitude": -0.09998975,
17         "latitude": 51.75436293,
18     },
19     {
20         "code": "fe2c3195-aeff-487a-a08f-e0bdc0ec6e9a",
21         "size": 405,
22         "price": 66,
23         "longitude": 0.18228006,
24         "latitude": 51.74640997,

```

```
25     },
26     {
27         "code": "913694c6-435a-4366-ba0d-da5334a611b2",
28         "size": 56,
29         "price": 60,
30         "longitude": 0.27891577,
31         "latitude": 51.45994069,
32     },
33     {
34         "code": "eed76e77-55c1-41ce-985d-ca49bf6c0585",
35         "size": 93,
36         "price": 48,
37         "longitude": 0.33894476,
38         "latitude": 51.39916678,
39     },
40 ]
41
42
43 @blueprint.route("/rooms", methods=["GET"])
44 def room_list():
45     repo = MemRepo(rooms)
46     result = room_list_use_case(repo)
47
48     return Response(
49         json.dumps(result, cls=RoomJsonEncoder),
50         mimetype="application/json",
51         status=200,
52     )
```



Source code

<https://github.com/pycabook/rentomatic/tree/ed2-c04-s03>

Please note that I initialised the memory storage with the same list used for the script `cli.py`. Again, the need of initialising the storage with data (even with an empty list) is due to the limitations of the storage `MemRepo`. The code that runs the use case is

application/rest/room.py

```
1 def room_list():
2     repo = MemRepo(rooms)
3     result = room_list_use_case(repo)
```

which is exactly the same code that we used in the command-line interface. The last part of the code creates a proper HTTP response, serializing the result of the use case using `RoomJsonEncoder`, and setting the HTTP status to 200 (success)

application/rest/room.py

```
1     return Response(
2         json.dumps(result, cls=RoomJsonEncoder),
3         mimetype="application/json",
4         status=200,
5     )
```

This shows you the power of the clean architecture in a nutshell. Writing a CLI interface or a Web service is different only in the presentation layer, not in the logic, which is the same, as it is contained in the use case.

Now that we defined the endpoint, we can finalise the configuration of the webserver, so that we can access the endpoint with a browser. This is not strictly part of the clean architecture, but as I did with the CLI interface I want you to see the final result, to get the whole picture and also to enjoy the effort you put in following the whole discussion up to this point.

WSGI

Python web applications expose a common interface called [Web Server Gateway Interface](https://en.wikipedia.org/wiki/Web_Server_Gateway_Interface)²⁶ or WSGI. So to run the Flask development web server, we have to define a `wsgi.py` file in the main folder of the project, i.e. in the same directory of the file `cli.py`

wsgi.py

```
1 import os
2
3 from application.app import create_app
4
5 app = create_app(os.environ["FLASK_CONFIG"])
```

²⁶https://en.wikipedia.org/wiki/Web_Server_Gateway_Interface

**Source code**

<https://github.com/pycabook/rentomatic/tree/ed2-c04-s04>

When you run the Flask Command Line Interface ([documentation](#)²⁷), it automatically looks for a file named `wsgi.py` and loads it, expecting it to contain a variable named `app` that is an instance of the object `Flask`. As the function `create_app` is a factory we just need to execute it.

At this point, you can execute `FLASK_CONFIG="development" flask run` in the directory that contains this file and you should see a nice message like

```
1  * Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)
```

At this point, you can point your browser to <http://127.0.0.1:5000/rooms> and enjoy the JSON data returned by the first endpoint of your web application.

I hope you can now appreciate the power of the layered architecture that we created. We definitely wrote a lot of code to “just” print out a list of models, but the code we wrote is a skeleton that can easily be extended and modified. It is also fully tested, which is a part of the implementation that many software projects struggle with.

The use case I presented is purposely very simple. It doesn’t require any input and it cannot return error conditions, so the code we wrote completely ignored input validation and error management. These topics are however extremely important, so we need to discuss how a clean architecture can deal with them.

²⁷<http://flask.pocoo.org/docs/1.0/cli/>

Chapter 05 Error management

”

You sent them out there and you didn't even warn them! Why didn't you warn them, Burke?

Aliens, 1986

In every software project, a great part of the code is dedicated to error management, and this code has to be rock solid. Error management is a complex topic, and there is always a corner case that we left out, or a condition that we supposed could never fail, while it does.

In a clean architecture, the main process is the creation of use cases and their execution. This is, therefore, the main source of errors, and the use cases layer is where we have to implement the error management. Errors can obviously come from the domain models layer, but since those models are created by the use cases the errors that are not managed by the models themselves automatically become errors of the use cases.

Request and responses

We can divide the error management code into two different areas. The first one represents and manages **requests**, that is, the input data that reaches our use case. The second one covers the way we return results from the use case through **responses**, the output data. These two concepts shouldn't be confused with HTTP requests and responses, even though there are similarities. We are now considering the way data can be passed to and received from use cases, and how to manage errors. This has nothing to do with the possible use of this architecture to expose an HTTP API.

Request and response objects are an important part of a clean architecture, as they transport call parameters, inputs and results from outside the application into the use cases layer.

More specifically, requests are objects created from incoming API calls, thus they shall deal with things like incorrect values, missing parameters, wrong formats, and so on. Responses, on the other hand, have to contain the actual results of the API calls, but shall also be able to represent error cases and deliver rich information on what happened.

The actual implementation of request and response objects is completely free, the clean architecture says nothing about them. The decision on how to pack and represent data is up to us.

To start working on possible errors and understand how to manage them, I will expand `room_list_use_case` to support filters that can be used to select a subset of the Room objects in storage.

The filters could be, for example, represented by a dictionary that contains attributes of the model Room and the logic to apply to them. Once we accept such a rich structure, we open our use case to

all sorts of errors: attributes that do not exist in the model, thresholds of the wrong type, filters that make the storage layer crash, and so on. All these considerations have to be taken into account by the use case.

Basic structure

We can implement structured requests before we expand the use case to accept filters. We just need a class `RoomListRequest` that can be initialised without parameters, so let us create the file `tests/requests/test_room_list.py` and put there a test for this object.

`tests/requests/test_room_list.py`

```
1 from rentomatic.requests.room_list import RoomListRequest
2
3
4 def test_build_room_list_request_without_parameters():
5     request = RoomListRequest()
6
7     assert bool(request) is True
8
9
10 def test_build_room_list_request_from_empty_dict():
11     request = RoomListRequest.from_dict({})
12
13     assert bool(request) is True
```

While at the moment this request object is basically empty, it will come in handy as soon as we start having parameters for the list use case. The code of the class `RoomListRequest` is the following

`rentomatic/requests/room_list.py`

```
1 class RoomListRequest:
2     @classmethod
3     def from_dict(cls, adict):
4         return cls()
5
6     def __bool__(self):
7         return True
```



Source code

<https://github.com/pycabook/rentomatic/tree/ed2-c05-s01>

The response object is also very simple since for the moment we just need to return a successful result. Unlike the request, the response is not linked to any particular use case, so the test file can be named `tests/test_responses.py`

tests/test_responses.py

```
1 from rentomatic.responses import ResponseSuccess
2
3
4 def test_response_success_is_true():
5     assert bool(ResponseSuccess()) is True
```

and the actual response object is in the file `rentomatic/responses.py`

rentomatic/responses.py

```
1 class ResponseSuccess:
2     def __init__(self, value=None):
3         self.value = value
4
5     def __bool__(self):
6         return True
```



Source code

<https://github.com/pycabook/rentomatic/tree/ed2-c05-s02>

With these two objects, we just laid the foundations for richer management of input and outputs of the use case, especially in the case of error conditions.

Requests and responses in a use case

Let's implement the request and response objects that we developed into the use case. To do this, we need to change the use case so that it accepts a request and return a response. The new version of `tests/use_cases/test_room_list.py` is the following

tests/use_cases/test_room_list.py

```
1  import pytest
2  import uuid
3  from unittest import mock
4
5  from rentomatic.domain.room import Room
6  from rentomatic.use_cases.room_list import room_list_use_case
7  from rentomatic.requests.room_list import RoomListRequest
8
9
10 @pytest.fixture
11 def domain_rooms():
12     room_1 = Room(
13         code=uuid.uuid4(),
14         size=215,
15         price=39,
16         longitude=-0.09998975,
17         latitude=51.75436293,
18     )
19
20     room_2 = Room(
21         code=uuid.uuid4(),
22         size=405,
23         price=66,
24         longitude=0.18228006,
25         latitude=51.74640997,
26     )
27
28     room_3 = Room(
29         code=uuid.uuid4(),
30         size=56,
31         price=60,
32         longitude=0.27891577,
33         latitude=51.45994069,
34     )
35
36     room_4 = Room(
37         code=uuid.uuid4(),
38         size=93,
39         price=48,
40         longitude=0.33894476,
41         latitude=51.39916678,
42     )
```

```

43
44     return [room_1, room_2, room_3, room_4]
45
46
47 def test_room_list_without_parameters(domain_rooms):
48     repo = mock.Mock()
49     repo.list.return_value = domain_rooms
50
51     request = RoomListRequest()
52
53     response = room_list_use_case(repo, request)
54
55     assert bool(response) is True
56     repo.list.assert_called_with()
57     assert response.value == domain_rooms

```

And the changes in the use case are minimal. The new version of the file `rentomatic/use_cases/room_list.py` is the following

`rentomatic/use_cases/room_list.py`

```

1 from rentomatic.responses import ResponseSuccess
2
3
4 def room_list_use_case(repo, request):
5     rooms = repo.list()
6     return ResponseSuccess(rooms)

```



Source code

<https://github.com/pycabook/rentomatic/tree/ed2-c05-s03>

Now we have a standard way to pack input and output values, and the above pattern is valid for every use case we can create. We are still missing some features, however, because so far requests and responses are not used to perform error management.

Request validation

The parameter filters that we want to add to the use case allows the caller to add conditions to narrow the results of the model list operation, using a notation like `<attribute>;__<operator>`. For example, specifying `filters={'price__lt': 100}` should return all the results with a price lower than 100.

Since the model Room has many attributes, the number of possible filters is very high. For simplicity's sake, I will consider the following cases:

- The attribute code supports only `__eq`, which finds the room with the specific code if it exists
- The attribute price supports `__eq`, `__lt`, and `__gt`
- All other attributes cannot be used in filters

The core idea here is that requests are customised for use cases, so they can contain the logic that validates the arguments used to instantiate them. The request is valid or invalid before it reaches the use case, so it is not the responsibility of the latter to check that the input values have proper values or a proper format.

This also means that building a request might result in two different objects, a valid one or an invalid one. For this reason, I decided to split the existing class `RoomListRequest` into `RoomListValidRequest` and `RoomListInvalidRequest`, creating a factory function that returns the proper object.

The first thing to do is to change the existing tests to use the factory.

`tests/requests/test_room_list.py`

```
1 from rentomatic.requests.room_list import build_room_list_request
2
3
4 def test_build_room_list_request_without_parameters():
5     request = build_room_list_request()
6
7     assert request.filters is None
8     assert bool(request) is True
9
10
11 def test_build_room_list_request_with_empty_filters():
12     request = build_room_list_request({})
13
14     assert request.filters == {}
15     assert bool(request) is True
```

Next, I will test that passing the wrong type of object as `filters` or that using incorrect keys results in an invalid request

tests/requests/test_room_list.py

```
1 def test_build_room_list_request_with_invalid_filters_parameter():
2     request = build_room_list_request(filters=5)
3
4     assert request.has_errors()
5     assert request.errors[0]["parameter"] == "filters"
6     assert bool(request) is False
7
8
9 def test_build_room_list_request_with_incorrect_filter_keys():
10    request = build_room_list_request(filters={"a": 1})
11
12    assert request.has_errors()
13    assert request.errors[0]["parameter"] == "filters"
14    assert bool(request) is False
```

Last, I will test the supported and unsupported keys

tests/requests/test_room_list.py

```
1 import pytest
2
3 ...
4
5 @pytest.mark.parametrize(
6     "key", ["code__eq", "price__eq", "price__lt", "price__gt"]
7 )
8 def test_build_room_list_request_accepted_filters(key):
9     filters = {key: 1}
10
11     request = build_room_list_request(filters=filters)
12
13     assert request.filters == filters
14     assert bool(request) is True
15
16
17 @pytest.mark.parametrize("key", ["code__lt", "code__gt"])
18 def test_build_room_list_request_rejected_filters(key):
19     filters = {key: 1}
20
21     request = build_room_list_request(filters=filters)
22
23     assert request.has_errors()
```

```
24     assert request.errors[0]["parameter"] == "filters"
25     assert bool(request) is False
```

Note that I used the decorator `pytest.mark.parametrize` to run the same test on multiple values. Following the TDD approach, adding those tests one by one and writing the code that passes them, I come up with the following code

`rentomatic/requests/room_list.py`

```
1  from collections.abc import Mapping
2
3
4  class RoomListInvalidRequest:
5      def __init__(self):
6          self.errors = []
7
8      def add_error(self, parameter, message):
9          self.errors.append({"parameter": parameter, "message": message})
10
11     def has_errors(self):
12         return len(self.errors) > 0
13
14     def __bool__(self):
15         return False
16
17
18     class RoomListValidRequest:
19         def __init__(self, filters=None):
20             self.filters = filters
21
22         def __bool__(self):
23             return True
24
25
26     def build_room_list_request(filters=None):
27         accepted_filters = ["code__eq", "price__eq", "price__lt", "price__gt"]
28         invalid_req = RoomListInvalidRequest()
29
30         if filters is not None:
31             if not isinstance(filters, Mapping):
32                 invalid_req.add_error("filters", "Is not iterable")
33             return invalid_req
34
```

```

35         for key, value in filters.items():
36             if key not in accepted_filters:
37                 invalid_req.add_error(
38                     "filters", "Key {} cannot be used".format(key)
39                 )
40
41         if invalid_req.has_errors():
42             return invalid_req
43
44     return RoomListValidRequest(filters=filters)

```

The introduction of the factory makes one use case test fails. The new version of that test is

tests/use_cases/test_room_list.py

```

1  ...
2
3  from rentomatic.requests.room_list import build_room_list_request
4
5  ...
6
7  def test_room_list_without_parameters(domain_rooms):
8      repo = mock.Mock()
9      repo.list.return_value = domain_rooms
10
11     request = build_room_list_request()
12
13     response = room_list_use_case(repo, request)
14
15     assert bool(response) is True
16     repo.list.assert_called_with()
17     assert response.value == domain_rooms

```



Source code

<https://github.com/pycabook/rentomatic/tree/ed2-c05-s04>

Responses and failures

There is a wide range of errors that can happen while the use case code is executed. Validation errors, as we just discussed in the previous section, but also business logic errors or errors that come from

the repository layer or other external systems that the use case interfaces with. Whatever the error, the use case shall always return an object with a known structure (the response), so we need a new object that provides good support for different types of failures.

As happened for the requests there is no unique way to provide such an object, and the following code is just one of the possible solutions. First of all, after some necessary imports, I test that responses have a boolean value

tests/test_responses.py

```
1  from rentomatic.responses import (
2      ResponseSuccess,
3      ResponseFailure,
4      ResponseTypes,
5      build_response_from_invalid_request,
6  )
7  from rentomatic.requests.room_list import RoomListInvalidRequest
8
9  SUCCESS_VALUE = {"key": ["value1", "value2"]}
10 GENERIC_RESPONSE_TYPE = "Response"
11 GENERIC_RESPONSE_MESSAGE = "This is a response"
12
13
14 def test_response_success_is_true():
15     response = ResponseSuccess(SUCCESS_VALUE)
16
17     assert bool(response) is True
18
19
20 def test_response_failure_is_false():
21     response = ResponseFailure(
22         GENERIC_RESPONSE_TYPE, GENERIC_RESPONSE_MESSAGE
23     )
24
25     assert bool(response) is False
```

Then I test the structure of responses, checking type and value. ResponseFailure objects should also have an attribute message

tests/test_responses.py

```
1 def test_response_success_has_type_and_value():
2     response = ResponseSuccess(SUCCESS_VALUE)
3
4     assert response.type == ResponseTypes.SUCCESS
5     assert response.value == SUCCESS_VALUE
6
7
8 def test_response_failure_has_type_and_message():
9     response = ResponseFailure(
10         GENERIC_RESPONSE_TYPE, GENERIC_RESPONSE_MESSAGE
11     )
12
13     assert response.type == GENERIC_RESPONSE_TYPE
14     assert response.message == GENERIC_RESPONSE_MESSAGE
15     assert response.value == {
16         "type": GENERIC_RESPONSE_TYPE,
17         "message": GENERIC_RESPONSE_MESSAGE,
18     }
```

The remaining tests are all about ResponseFailure. First, a test to check that it can be initialised with an exception

tests/test_responses.py

```
1 def test_response_failure_initialisation_with_exception():
2     response = ResponseFailure(
3         GENERIC_RESPONSE_TYPE, Exception("Just an error message")
4     )
5
6     assert bool(response) is False
7     assert response.type == GENERIC_RESPONSE_TYPE
8     assert response.message == "Exception: Just an error message"
```

Since we want to be able to build a response directly from an invalid request, getting all the errors contained in the latter, we need to test that case

tests/test_responses.py

```

1 def test_response_failure_from_empty_invalid_request():
2     response = build_response_from_invalid_request(
3         RoomListInvalidRequest()
4     )
5
6     assert bool(response) is False
7     assert response.type == ResponseTypes.PARAMETERS_ERROR
8
9
10 def test_response_failure_from_invalid_request_with_errors():
11     request = RoomListInvalidRequest()
12     request.add_error("path", "Is mandatory")
13     request.add_error("path", "can't be blank")
14
15     response = build_response_from_invalid_request(request)
16
17     assert bool(response) is False
18     assert response.type == ResponseTypes.PARAMETERS_ERROR
19     assert response.message == "path: Is mandatory\npath: can't be blank"

```

Let's write the classes that make the tests pass

rentomatic/responses.py

```

1 class ResponseTypes:
2     PARAMETERS_ERROR = "ParametersError"
3     RESOURCE_ERROR = "ResourceError"
4     SYSTEM_ERROR = "SystemError"
5     SUCCESS = "Success"
6
7
8 class ResponseFailure:
9     def __init__(self, type_, message):
10         self.type = type_
11         self.message = self._format_message(message)
12
13     def _format_message(self, msg):
14         if isinstance(msg, Exception):
15             return "{}: {}".format(
16                 msg.__class__.__name__, "{}".format(msg)
17             )
18         return msg

```

```

19
20     @property
21     def value(self):
22         return {"type": self.type, "message": self.message}
23
24     def __bool__(self):
25         return False
26
27
28 class ResponseSuccess:
29     def __init__(self, value=None):
30         self.type = ResponseTypes.SUCCESS
31         self.value = value
32
33     def __bool__(self):
34         return True
35
36
37 def build_response_from_invalid_request(invalid_request):
38     message = "\n".join(
39         [
40             "{}: {}".format(err["parameter"], err["message"])
41             for err in invalid_request.errors
42         ]
43     )
44     return ResponseFailure(ResponseTypes.PARAMETERS_ERROR, message)

```

Through the method `_format_message()` we enable the class to accept both string messages and Python exceptions, which is very handy when dealing with external libraries that can raise exceptions we do not know or do not want to manage.

The error types contained in the class `ResponseTypes` are very similar to HTTP errors, and this will be useful later when we will return responses from the web framework. `PARAMETERS_ERROR` signals that something was wrong in the input parameters passed by the request. `RESOURCE_ERROR` signals that the process ended correctly, but the requested resource is not available, for example when reading a specific value from a data storage. Last, `SYSTEM_ERROR` signals that something went wrong with the process itself, and will be used mostly to signal an exception in the Python code.



Source code

<https://github.com/pycabook/rentomatic/tree/ed2-c05-s05>

Error management in a use case

Our implementation of requests and responses is finally complete, so we can now implement the last version of our use case. The function `room_list_use_case` is still missing a proper validation of the incoming request, and is not returning a suitable response in case something went wrong.

The test `test_room_list_without_parameters` must match the new API, so I added `filters=None` to `assert_called_with`

`tests/use_cases/test_room_list.py`

```

1  def test_room_list_without_parameters(domain_rooms):
2      repo = mock.Mock()
3      repo.list.return_value = domain_rooms
4
5      request = build_room_list_request()
6
7      response = room_list_use_case(repo, request)
8
9      assert bool(response) is True
10     repo.list.assert_called_with(filters=None)
11     assert response.value == domain_rooms

```

There are three new tests that we can add to check the behaviour of the use case when `filters` is not `None`. The first one checks that the value of the key `filters` in the dictionary used to create the request is actually used when calling the repository. These last two tests check the behaviour of the use case when the repository raises an exception or when the request is badly formatted.

`tests/use_cases/test_room_list.py`

```

1  import pytest
2  import uuid
3  from unittest import mock
4
5  from rentomatic.domain.room import Room
6  from rentomatic.use_cases.room_list import room_list_use_case
7  from rentomatic.requests.room_list import build_room_list_request
8  from rentomatic.responses import ResponseTypes
9
10 ...
11
12 def test_room_list_with_filters(domain_rooms):
13     repo = mock.Mock()
14     repo.list.return_value = domain_rooms

```

```
15
16     qry_filters = {"code__eq": 5}
17     request = build_room_list_request(filters=qry_filters)
18
19     response = room_list_use_case(repo, request)
20
21     assert bool(response) is True
22     repo.list.assert_called_with(filters=qry_filters)
23     assert response.value == domain_rooms
24
25
26 def test_room_list_handles_generic_error():
27     repo = mock.Mock()
28     repo.list.side_effect = Exception("Just an error message")
29
30     request = build_room_list_request(filters={})
31
32     response = room_list_use_case(repo, request)
33
34     assert bool(response) is False
35     assert response.value == {
36         "type": ResponseTypes.SYSTEM_ERROR,
37         "message": "Exception: Just an error message",
38     }
39
40
41 def test_room_list_handles_bad_request():
42     repo = mock.Mock()
43
44     request = build_room_list_request(filters=5)
45
46     response = room_list_use_case(repo, request)
47
48     assert bool(response) is False
49     assert response.value == {
50         "type": ResponseTypes.PARAMETERS_ERROR,
51         "message": "filters: Is not iterable",
52     }
```

Now change the use case to contain the new use case implementation that makes all the tests pass

rentomatic/use_cases/room_list.py

```
1 from rentomatic.responses import (  
2     ResponseSuccess,  
3     ResponseFailure,  
4     ResponseTypes,  
5     build_response_from_invalid_request,  
6 )  
7  
8  
9 def room_list_use_case(repo, request):  
10     if not request:  
11         return build_response_from_invalid_request(request)  
12     try:  
13         rooms = repo.list(filters=request.filters)  
14         return ResponseSuccess(rooms)  
15     except Exception as exc:  
16         return ResponseFailure(ResponseTypes.SYSTEM_ERROR, exc)
```

As you can see, the first thing that the use case does is to check if the request is valid. Otherwise, it returns a `ResponseFailure` built with the same request object. Then the actual business logic is implemented, calling the repository and returning a successful response. If something goes wrong in this phase the exception is caught and returned as an aptly formatted `ResponseFailure`.



Source code

<https://github.com/pycabook/rentomatic/tree/ed2-c05-s06>

Integrating external systems

I want to point out a big problem represented by mocks.

As we are testing objects using mocks for external systems, like the repository, no tests fail at the moment, but trying to run the Flask development server will certainly return an error. As a matter of fact, neither the repository nor the HTTP server are in sync with the new API, but this cannot be shown by unit tests if they are properly written. This is the reason why we need integration tests, since external systems that rely on a certain version of the API are running only at that point, and this can raise issues that were masked by mocks.

For this simple project, my integration test is represented by the Flask development server, which at this point crashes. If you run `FLASK_CONFIG="development" flask run` and open <http://127.0.0.1:5000/rooms> with your browser you will get an Internal Server Error, and on the command line this exception

```
1 TypeError: room_list_use_case() missing 1 required positional argument: 'request'
```

The same error is returned by the CLI interface. After the introduction of requests and responses we didn't change the REST endpoint, which is one of the connections between the external world and the use case. Given that the API of the use case changed, we need to change the code of the endpoints that call the use case.

The HTTP server

As we can see from the exception above the use case is called with the wrong parameters in the REST endpoint. The new version of the test is

tests/rest/test_room.py

```
1 import json
2 from unittest import mock
3
4 import pytest
5
6 from rentomatic.domain.room import Room
7 from rentomatic.responses import (
8     ResponseFailure,
9     ResponseSuccess,
10    ResponseTypes,
11 )
12
13 room_dict = {
14     "code": "3251a5bd-86be-428d-8ae9-6e51a8048c33",
15     "size": 200,
16     "price": 10,
17     "longitude": -0.09998975,
18     "latitude": 51.75436293,
19 }
20
21 rooms = [Room.from_dict(room_dict)]
22
23
24 @mock.patch("application.rest.room.room_list_use_case")
25 def test_get(mock_use_case, client):
26     mock_use_case.return_value = ResponseSuccess(rooms)
27
28     http_response = client.get("/rooms")
29
```

```
30     assert json.loads(http_response.data.decode("UTF-8")) == [room_dict]
31
32     mock_use_case.assert_called()
33     args, kwargs = mock_use_case.call_args
34     assert args[1].filters == {}
35
36     assert http_response.status_code == 200
37     assert http_response.mimetype == "application/json"
38
39
40 @mock.patch("application.rest.room.room_list_use_case")
41 def test_get_with_filters(mock_use_case, client):
42     mock_use_case.return_value = ResponseSuccess(rooms)
43
44     http_response = client.get(
45         "/rooms?filter_price__gt=2&filter_price__lt=6"
46     )
47
48     assert json.loads(http_response.data.decode("UTF-8")) == [room_dict]
49
50     mock_use_case.assert_called()
51     args, kwargs = mock_use_case.call_args
52     assert args[1].filters == {"price__gt": "2", "price__lt": "6"}
53
54     assert http_response.status_code == 200
55     assert http_response.mimetype == "application/json"
56
57
58 @pytest.mark.parametrize(
59     "response_type, expected_status_code",
60     [
61         (ResponseTypes.PARAMETERS_ERROR, 400),
62         (ResponseTypes.RESOURCE_ERROR, 404),
63         (ResponseTypes.SYSTEM_ERROR, 500),
64     ],
65 )
66 @mock.patch("application.rest.room.room_list_use_case")
67 def test_get_response_failures(
68     mock_use_case,
69     client,
70     response_type,
71     expected_status_code,
72 ):
```



```
73     mock_use_case.return_value = ResponseFailure(  
74         response_type,  
75         message="Just an error message",  
76     )  
77  
78     http_response = client.get("/rooms?dummy_request_string")  
79  
80     mock_use_case.assert_called()  
81  
82     assert http_response.status_code == expected_status_code
```

The function `test_get` was already present but has been changed to reflect the use of requests and responses. The first change is that the use case in the mock has to return a proper response

```
1 mock_use_case.return_value = ResponseSuccess(rooms)
```

and the second is the assertion on the call of the use case. It should be called with a properly formatted request, but since we can't compare requests, we need a way to look into the call arguments. This can be done with

```
1 mock_use_case.assert_called()  
2 args, kwargs = mock_use_case.call_args  
3 assert args[1].filters == {}
```

as the use case should receive a request with empty filters as an argument.

The function `test_get_with_filters` performs the same operation but passing a query string to the URL `/rooms`, which requires a different assertion

```
1 assert args[1].filters == {'price__gt': '2', 'price__lt': '6'}
```

Both the tests pass with a new version of the endpoint `room_list`

application/rest/room.py

```
1  import json
2
3  from flask import Blueprint, request, Response
4
5  from rentomatic.repository.memrepo import MemRepo
6  from rentomatic.use_cases.room_list import room_list_use_case
7  from rentomatic.serializers.room import RoomJsonEncoder
8  from rentomatic.requests.room_list import build_room_list_request
9  from rentomatic.responses import ResponseTypes
10
11  blueprint = Blueprint("room", __name__)
12
13  STATUS_CODES = {
14      ResponseTypes.SUCCESS: 200,
15      ResponseTypes.RESOURCE_ERROR: 404,
16      ResponseTypes.PARAMETERS_ERROR: 400,
17      ResponseTypes.SYSTEM_ERROR: 500,
18  }
19
20  rooms = [
21      {
22          "code": "f853578c-fc0f-4e65-81b8-566c5dffa35a",
23          "size": 215,
24          "price": 39,
25          "longitude": -0.09998975,
26          "latitude": 51.75436293,
27      },
28      {
29          "code": "fe2c3195-aeff-487a-a08f-e0bdc0ec6e9a",
30          "size": 405,
31          "price": 66,
32          "longitude": 0.18228006,
33          "latitude": 51.74640997,
34      },
35      {
36          "code": "913694c6-435a-4366-ba0d-da5334a611b2",
37          "size": 56,
38          "price": 60,
39          "longitude": 0.27891577,
40          "latitude": 51.45994069,
41      },
42      {
```

```

43         "code": "eed76e77-55c1-41ce-985d-ca49bf6c0585",
44         "size": 93,
45         "price": 48,
46         "longitude": 0.33894476,
47         "latitude": 51.39916678,
48     },
49 ]
50
51
52 @blueprint.route("/rooms", methods=["GET"])
53 def room_list():
54     qrystr_params = {
55         "filters": {},
56     }
57
58     for arg, values in request.args.items():
59         if arg.startswith("filter_"):
60             qrystr_params["filters"][arg.replace("filter_", "")] = values
61
62     request_object = build_room_list_request(
63         filters=qrystr_params["filters"]
64     )
65
66     repo = MemRepo(rooms)
67     response = room_list_use_case(repo, request_object)
68
69     return Response(
70         json.dumps(response.value, cls=RoomJsonEncoder),
71         mimetype="application/json",
72         status=STATUS_CODES[response.type],
73     )

```

Please note that I'm using a variable named `request_object` here to avoid clashing with the fixture request provided by `pytest-flask`. While `request` contains the HTTP request sent to the web framework by the browser, `request_object` is the request we send to the use case.



Source code

<https://github.com/pycabook/rentomatic/tree/ed2-c05-s07>

The repository

If we run the Flask development webserver now and try to access the endpoint `/rooms`, we will get a nice response that says

```
1 {"type": "SystemError", "message": "TypeError: list() got an unexpected keyword argu\
2 ment 'filters'"}

```

and if you look at the HTTP response²⁸ you can see an HTTP 500 error, which is exactly the mapping of our `SystemError` use case error, which in turn signals a Python exception, which is in the message part of the error.

This error comes from the repository, which has not been migrated to the new API. We need then to change the method `list` of the class `MemRepo` to accept the parameter `filters` and to act accordingly. Pay attention to this point. The filters might have been considered part of the business logic and implemented in the use case itself, but we decided to leverage what the storage system can do, so we moved filtering in that external system. This is a reasonable choice as databases can usually perform filtering and ordering very well. Even though the in-memory storage we are currently using is not a database, we are preparing to use a real external storage.

The new version of repository tests is

`tests/repository/test_memrepo.py`

```
1 import pytest
2
3 from rentomatic.domain.room import Room
4 from rentomatic.repository.memrepo import MemRepo
5
6
7 @pytest.fixture
8 def room_dicts():
9     return [
10         {
11             "code": "f853578c-fc0f-4e65-81b8-566c5dffa35a",
12             "size": 215,
13             "price": 39,
14             "longitude": -0.09998975,
15             "latitude": 51.75436293,
16         },
17         {
18             "code": "fe2c3195-aeff-487a-a08f-e0bdc0ec6e9a",
19             "size": 405,

```

²⁸Unless you consider things like `sessionmaker_mock().query.assert_called_with(Room)` something attractive. And this was by far the simplest mock I had to write.

```

20         "price": 66,
21         "longitude": 0.18228006,
22         "latitude": 51.74640997,
23     },
24     {
25         "code": "913694c6-435a-4366-ba0d-da5334a611b2",
26         "size": 56,
27         "price": 60,
28         "longitude": 0.27891577,
29         "latitude": 51.45994069,
30     },
31     {
32         "code": "eed76e77-55c1-41ce-985d-ca49bf6c0585",
33         "size": 93,
34         "price": 48,
35         "longitude": 0.33894476,
36         "latitude": 51.39916678,
37     },
38 ]
39
40
41 def test_repository_list_without_parameters(room_dicts):
42     repo = MemRepo(room_dicts)
43
44     rooms = [Room.from_dict(i) for i in room_dicts]
45
46     assert repo.list() == rooms
47
48
49 def test_repository_list_with_code_equal_filter(room_dicts):
50     repo = MemRepo(room_dicts)
51
52     rooms = repo.list(
53         filters={"code__eq": "fe2c3195-aeff-487a-a08f-e0bdc0ec6e9a"}
54     )
55
56     assert len(rooms) == 1
57     assert rooms[0].code == "fe2c3195-aeff-487a-a08f-e0bdc0ec6e9a"
58
59
60 @pytest.mark.parametrize("price", [60, "60"])
61 def test_repository_list_with_price_equal_filter(room_dicts, price):
62     repo = MemRepo(room_dicts)

```

```

63
64     rooms = repo.list(filters={"price__eq": price})
65
66     assert len(rooms) == 1
67     assert rooms[0].code == "913694c6-435a-4366-ba0d-da5334a611b2"
68
69
70 @pytest.mark.parametrize("price", [60, "60"])
71 def test_repository_list_with_price_less_than_filter(room_dicts, price):
72     repo = MemRepo(room_dicts)
73
74     rooms = repo.list(filters={"price__lt": price})
75
76     assert len(rooms) == 2
77     assert set([r.code for r in rooms]) == {
78         "f853578c-fc0f-4e65-81b8-566c5dffa35a",
79         "eed76e77-55c1-41ce-985d-ca49bf6c0585",
80     }
81
82
83 @pytest.mark.parametrize("price", [48, "48"])
84 def test_repository_list_with_price_greater_than_filter(room_dicts, price):
85     repo = MemRepo(room_dicts)
86
87     rooms = repo.list(filters={"price__gt": price})
88
89     assert len(rooms) == 2
90     assert set([r.code for r in rooms]) == {
91         "913694c6-435a-4366-ba0d-da5334a611b2",
92         "fe2c3195-aeff-487a-a08f-e0bdc0ec6e9a",
93     }
94
95
96 def test_repository_list_with_price_between_filter(room_dicts):
97     repo = MemRepo(room_dicts)
98
99     rooms = repo.list(filters={"price__lt": 66, "price__gt": 48})
100
101     assert len(rooms) == 1
102     assert rooms[0].code == "913694c6-435a-4366-ba0d-da5334a611b2"

```

As you can see, I added many tests. One test for each of the four accepted filters (code__eq, price__

_eq, price__lt, price__gt, see `rentomatic/requests/room_list.py`), and one final test that tries two different filters at the same time.

Again, keep in mind that this is the API exposed by the storage, not the one exposed by the use case. The fact that the two match is a design decision, but your mileage may vary.

The new version of the repository is

`rentomatic/repository/memrepo.py`

```

1  from rentomatic.domain.room import Room
2
3
4  class MemRepo:
5      def __init__(self, data):
6          self.data = data
7
8      def list(self, filters=None):
9
10         result = [Room.from_dict(i) for i in self.data]
11
12         if filters is None:
13             return result
14
15         if "code__eq" in filters:
16             result = [r for r in result if r.code == filters["code__eq"]]
17
18         if "price__eq" in filters:
19             result = [
20                 r for r in result if r.price == int(filters["price__eq"])
21             ]
22
23         if "price__lt" in filters:
24             result = [
25                 r for r in result if r.price < int(filters["price__lt"])
26             ]
27
28         if "price__gt" in filters:
29             result = [
30                 r for r in result if r.price > int(filters["price__gt"])
31             ]
32
33     return result

```

At this point, you can start the Flask development webserver with `FLASK_CONFIG="development"`

`flask run`, and get the list of all your rooms at <http://localhost:5000/rooms>. You can also use filters in the URL, like http://localhost:5000/rooms?filter*codeeq=f853578c-fc0f-4e65-81b8-566c5dffa35a which returns the room with the given code or http://localhost:5000/rooms?filter*pricelt=50 which returns all the rooms with a price less than 50.



Source code

<https://github.com/pycabook/rentomatic/tree/ed2-c05-s08>

The CLI

At this point fixing the CLI is extremely simple, as we just need to imitate what we did for the HTTP server, only without considering the filters as they were not part of the command line tool.

`cli.py`

```
1  #!/usr/bin/env python
2
3  from rentomatic.repository.memrepo import MemRepo
4  from rentomatic.use_cases.room_list import room_list_use_case
5  from rentomatic.requests.room_list import build_room_list_request
6
7  rooms = [
8      {
9          "code": "f853578c-fc0f-4e65-81b8-566c5dffa35a",
10         "size": 215,
11         "price": 39,
12         "longitude": -0.09998975,
13         "latitude": 51.75436293,
14     },
15     {
16         "code": "fe2c3195-aeff-487a-a08f-e0bdc0ec6e9a",
17         "size": 405,
18         "price": 66,
19         "longitude": 0.18228006,
20         "latitude": 51.74640997,
21     },
22     {
23         "code": "913694c6-435a-4366-ba0d-da5334a611b2",
24         "size": 56,
25         "price": 60,
26         "longitude": 0.27891577,
27         "latitude": 51.45994069,
```



```
28     },
29     {
30         "code": "eed76e77-55c1-41ce-985d-ca49bf6c0585",
31         "size": 93,
32         "price": 48,
33         "longitude": 0.33894476,
34         "latitude": 51.39916678,
35     },
36 ]
37
38 request = build_room_list_request()
39 repo = MemRepo(rooms)
40 response = room_list_use_case(repo, request)
41
42 print([room.to_dict() for room in response.value])
```



Source code

<https://github.com/pycabook/rentomatic/tree/ed2-c05-s09>

We now have a very robust system to manage input validation and error conditions, and it is generic enough to be used with any possible use case. Obviously, we are free to add new types of errors to increase the granularity with which we manage failures, but the present version already covers everything that can happen inside a use case.

In the next chapter, we will have a look at repositories based on real database engines, showing how to test external systems with integration tests, using PostgreSQL as a database. In a later chapter I will show how the clean architecture allows us to switch very easily between different external systems, moving the system to MongoDB.

Chapter 06 Integration with a real external system postgres

”

Ooooh, I'm very sorry Hans. I didn't get that memo.
Maybe you should've put it on the bulletin board.

Die Hard, 1988

The basic in-memory repository I implemented for the project is enough to show the concept of the repository layer abstraction. It is not enough to run a production system, though, so we need to implement a connection with a real storage like a database. Whenever we use an external system and we want to test the interface we can use mocks, but at a certain point we need to ensure that the two systems actually work together, and this is when we need to start creating integration tests.

In this chapter I will show how to set up and run integration tests between our application and a real database. At the end of the chapter I will have a repository that allows the application to interface with PostgreSQL, and a battery of tests that run using a real database instance running in Docker.

This chapter will show you one of the biggest advantages of a clean architecture, namely the simplicity with which you can replace existing components with others, possibly based on a completely different technology.

Decoupling with interfaces

The clean architecture we devised in the previous chapters defines a use case that receives a repository instance as an argument and uses its `list` method to retrieve the contained entries. This allows the use case to form a very loose coupling with the repository, being connected only through the API exposed by the object and not to the real implementation. In other words, the use cases are polymorphic with respect to the method `list`.

This is very important and it is the core of the clean architecture design. Being connected through an API, the use case and the repository can be replaced by different implementations at any time, given that the new implementation provides the requested interface.

It is worth noting, for example, that the initialisation of the object is not part of the API that the use cases are using since the repository is initialised in the main script and not in each use case. The method `__init__`, thus, doesn't need to be the same among the repository implementations, which gives us a great deal of flexibility, as different storage systems may need different initialisation values.

The simple repository we implemented in one of the previous chapters is

rentomatic/repository/memrepo.py

```

1  from rentomatic.domain.room import Room
2
3
4  class MemRepo:
5      def __init__(self, data):
6          self.data = data
7
8      def list(self, filters=None):
9
10         result = [Room.from_dict(i) for i in self.data]
11
12         if filters is None:
13             return result
14
15         if "code__eq" in filters:
16             result = [r for r in result if r.code == filters["code__eq"]]
17
18         if "price__eq" in filters:
19             result = [
20                 r for r in result if r.price == int(filters["price__eq"])
21             ]
22
23         if "price__lt" in filters:
24             result = [
25                 r for r in result if r.price < int(filters["price__lt"])
26             ]
27
28         if "price__gt" in filters:
29             result = [
30                 r for r in result if r.price > int(filters["price__gt"])
31             ]
32
33     return result

```

whose interface is made of two parts: the initialisation and the method `list`. The method `__init__` accepts values because this specific object doesn't act as long-term storage, so we are forced to pass some data every time we instantiate the class.

A repository based on a proper database will not need to be filled with data when initialised, its main job being that of storing data between sessions, but will nevertheless need to be initialised at least with the database address and access credentials.

Furthermore, we have to deal with a proper external system, so we have to devise a strategy to test it, as this might require a running database engine in the background. Remember that we are creating a specific implementation of a repository, so everything will be tailored to the actual database system that we will choose.

A repository based on PostgreSQL

Let's start with a repository based on a popular SQL database, [PostgreSQL](https://www.postgresql.org)²⁹. It can be accessed from Python in many ways, but the best one is probably through the [SQLAlchemy](https://www.sqlalchemy.org)³⁰ interface. SQLAlchemy is an ORM, a package that maps objects (as in object-oriented) to a relational database. ORMs can normally be found in web frameworks like Django or in standalone packages like the one we are considering.

The important thing about ORMs is that they are very good examples of something you shouldn't try to mock. Properly mocking the SQLAlchemy structures that are used when querying the DB results in very complex code that is difficult to write and almost impossible to maintain, as every single change in the queries results in a series of mocks that have to be written again³¹.

We need therefore to set up an integration test. The idea is to create the DB, set up the connection with SQLAlchemy, test the condition we need to check, and destroy the database. Since the action of creating and destroying the DB can be expensive in terms of time, we might want to do it just at the beginning and at the end of the whole test suite, but even with this change, the tests will be slow. This is why we will also need to use labels to avoid running them every time we run the suite. Let's face this complex task one step at a time.

Label integration tests

The first thing we need to do is to label integration tests, exclude them by default and create a way to run them. Since pytest supports labels, called *marks*, we can use this feature to add a global mark to a whole module. Create the file `tests/repository/postgres/test_postgresrepo.py` and put in it this code

²⁹<https://www.postgresql.org>

³⁰<https://www.sqlalchemy.org>

³¹Unless you consider things like `sessionmaker_mock().query.assert_called_with(Room)` something attractive. And this was by far the simplest mock I had to write.

tests/repository/postgres/test_postgresrepo.py

```

1  import pytest
2
3  pytestmark = pytest.mark.integration
4
5
6  def test_dummy():
7      pass

```

The module attribute `pytestmark` labels every test in the module with the tag `integration`. To verify that this works I added a `test_dummy` test function which always passes.

The marker should be registered in `pytest.ini`

pytest.ini

```

1  [pytest]
2  minversion = 2.0
3  norecursedirs = .git .tox requirements*
4  python_files = test*.py
5  markers =
6      integration: integration tests

```

You can now run `pytest -svv -m integration` to ask pytest to run only the tests marked with that label. The option `-m` supports a rich syntax that you can learn by reading the [documentation](https://docs.pytest.org/en/latest/example/markers.html)³².

```

1  $ pytest -svv -m integration
2  ===== test session starts =====
3  platform linux -- Python XXXX, pytest-XXXX, py-XXXX, pluggy-XXXX --
4  cabook/venv3/bin/python3
5  cachedir: .cache
6  rootdir: cabook/code/calc, inifile: pytest.ini
7  plugins: cov-XXXX
8  collected 36 items / 35 deselected / 1 selected
9
10 tests/repository/postgres/test_postgresrepo.py::test_dummy PASSED
11
12 ===== 1 passed, 35 deselected in 0.20s =====

```

While this is enough to run integration tests selectively, it is not enough to skip them by default. To do this, we can alter the pytest setup to label all those tests as skipped, but this will give us no means to run them. The standard way to implement this is to define a new command-line option and to process each marked test according to the value of this option.

To do it open the file `tests/conftest.py` that we already created and add the following code

³²<https://docs.pytest.org/en/latest/example/markers.html>

tests/conftest.py

```

1 def pytest_addoption(parser):
2     parser.addoption(
3         "--integration", action="store_true", help="run integration tests"
4     )
5
6
7 def pytest_runtest_setup(item):
8     if "integration" in item.keywords and not item.config.getvalue(
9         "integration"
10    ):
11         pytest.skip("need --integration option to run")

```

The first function is a hook into the pytest CLI parser that adds the option `--integration`. When this option is specified on the command line the pytest setup will contain the key `integration` with value `True`.

The second function is a hook into the pytest setup of every single test. The variable `item` contains the test itself (actually a `_pytest.python.Function` object), which in turn contains two useful pieces of information. The first is the attribute `item.keywords`, that contains the test marks, alongside many other interesting things like the name of the test, the file, the module, and also information about the patches that happen inside the test. The second is the attribute `item.config` that contains the parsed pytest command line.

So, if the test is marked with `integration` (`'integration' in item.keywords`) and the option `--integration` is not present (`not item.config.getvalue("integration")`) the test is skipped.

This is the output with `--integration`

```

1 $ pytest -svv --integration
2 ===== test session starts =====
3 platform linux -- Python XXXX, pytest-XXXX, py-XXXX, pluggy-XXXX --
4 cabook/venv3/bin/python3
5 cachedir: .cache
6 rootdir: cabook/code/calc, inifile: pytest.ini
7 plugins: cov-XXXX
8 collected 36 items
9
10 ...
11 tests/repository/postgres/test_postgresrepo.py::test_dummy PASSED
12 ...
13
14 ===== 36 passed in 0.26s =====

```

and this is the output without the custom option

```

1  $ pytest -svv
2  ===== test session starts =====
3  platform linux -- Python XXXX, pytest-XXXX, py-XXXX, pluggy-XXXX --
4  cabook/venv3/bin/python3
5  cachedir: .cache
6  rootdir: cabook/code/calc, inifile: pytest.ini
7  plugins: cov-XXXX
8  collected 36 items
9
10 ...
11 tests/repository/postgres/test_postgresrepo.py::test_dummy SKIPPED
12 ...
13
14 ===== 35 passed, 1 skipped in 0.27s =====

```



Source code

<https://github.com/pycabook/rentomatic/tree/ed2-c06-s01>

Create SQLAlchemy classes

Creating and populating the test database with initial data will be part of the test suite, but we need to define somewhere the tables that will be contained in the database. This is where SQLAlchemy's ORM comes into play, as we will define those tables in terms of Python objects.

Add the packages SQLAlchemy and psycopg2 to the requirements file `prod.txt`

requirements/prod.txt

```

1  Flask
2  SQLAlchemy
3  psycopg2

```

and update the installed packages with

```

1  $ pip install -r requirements/dev.txt

```

Create the file `rentomatic/repository/postgres_objects.py` with the following content

rentomatic/repository/postgres_objects.py

```

1  from sqlalchemy import Column, Integer, String, Float
2  from sqlalchemy.ext.declarative import declarative_base
3
4  Base = declarative_base()
5
6
7  class Room(Base):
8      __tablename__ = 'room'
9
10     id = Column(Integer, primary_key=True)
11
12     code = Column(String(36), nullable=False)
13     size = Column(Integer)
14     price = Column(Integer)
15     longitude = Column(Float)
16     latitude = Column(Float)

```

Let's comment it section by section

```

1  from sqlalchemy import Column, Integer, String, Float
2  from sqlalchemy.ext.declarative import declarative_base
3
4  Base = declarative_base()

```

We need to import many things from the SQLAlchemy package to set up the database and to create the table. Remember that SQLAlchemy has a declarative approach, so we need to instantiate the object Base and then use it as a starting point to declare the tables/objects.

```

1  class Room(Base):
2      __tablename__ = 'room'
3
4      id = Column(Integer, primary_key=True)
5
6      code = Column(String(36), nullable=False)
7      size = Column(Integer)
8      price = Column(Integer)
9      longitude = Column(Float)
10     latitude = Column(Float)

```

This is the class that represents the room in the database. It is important to understand that this is not the class we are using in the business logic, but the class that defines the table in the SQL database

that we will use to map the Room entity. The structure of this class is thus dictated by the needs of the storage layer, and not by the use cases. You might want for instance to store `longitude` and `latitude` in a JSON field, to allow for easier extendibility, without changing the definition of the domain model. In the simple case of the Rent-o-matic project, the two classes almost overlap, but this is not the case generally speaking.

Obviously, this means that you have to keep the storage and the domain levels in sync and that you need to manage migrations on your own. You can use tools like Alembic, but the migrations will not come directly from domain model changes.



Source code

<https://github.com/pycabook/rentomatic/tree/ed2-c06-s02>

Orchestration management

When we run the integration tests the Postgres database engine must be already running in the background, and it must be already configured, for example, with a pristine database ready to be used. Moreover, when all the tests have been executed the database should be removed and the database engine stopped.

This is a perfect job for Docker, which can run complex systems in isolation with minimal configuration. We have a choice here: we might want to orchestrate the creation and destruction of the database with an external script or try to implement everything in the test suite. The first solution is what many frameworks use, and what I explored in my series of posts [Flask Project Setup: TDD, Docker, Postgres and more](#)³³, so in this chapter I will show an implementation of that solution.

As I explained in the posts I mentioned the plan is to create a management script that spins up and tears down the required containers, runs the tests in between. The management script can be used also to run the application itself, or to create development setups, but in this case I will simplify it to manage only the tests. I highly recommend that you read those posts if you want to get the big picture behind the setup I will use.

The first thing we have to do if we plan to use Docker Compose is to add the requirement to `requirements/test.txt`

³³<https://www.thedigitalcatonline.com/blog/2020/07/05/flask-project-setup-tdd-docker-postgres-and-more-part-1/>

requirements/test.txt

```
1 -r prod.txt
2 tox
3 coverage
4 pytest
5 pytest-cov
6 pytest-flask
7 docker-compose
```

and install it running `pip install -r requirements/dev.txt`. The management script is the following

manage.py

```
1  #!/usr/bin/env python
2
3  import os
4  import json
5  import subprocess
6  import time
7
8  import click
9  import psycopg2
10 from psycopg2.extensions import ISOLATION_LEVEL_AUTOCOMMIT
11
12
13 # Ensure an environment variable exists and has a value
14 def setenv(variable, default):
15     os.environ[variable] = os.getenv(variable, default)
16
17
18 APPLICATION_CONFIG_PATH = "config"
19 DOCKER_PATH = "docker"
20
21
22 def app_config_file(config):
23     return os.path.join(APPLICATION_CONFIG_PATH, f"{config}.json")
24
25
26 def docker_compose_file(config):
27     return os.path.join(DOCKER_PATH, f"{config}.yaml")
28
29
```

```
30 def read_json_configuration(config):
31     # Read configuration from the relative JSON file
32     with open(app_config_file(config)) as f:
33         config_data = json.load(f)
34
35     # Convert the config into a usable Python dictionary
36     config_data = dict((i["name"], i["value"]) for i in config_data)
37
38     return config_data
39
40
41 def configure_app(config):
42     configuration = read_json_configuration(config)
43
44     for key, value in configuration.items():
45         setenv(key, value)
46
47
48 @click.group()
49 def cli():
50     pass
51
52
53 def docker_compose_cmdline(commands_string=None):
54     config = os.getenv("APPLICATION_CONFIG")
55     configure_app(config)
56
57     compose_file = docker_compose_file(config)
58
59     if not os.path.isfile(compose_file):
60         raise ValueError(f"The file {compose_file} does not exist")
61
62     command_line = [
63         "docker-compose",
64         "-p",
65         config,
66         "-f",
67         compose_file,
68     ]
69
70     if commands_string:
71         command_line.extend(commands_string.split(" "))
72
```

```
73     return command_line
74
75
76 def run_sql(statements):
77     conn = psycopg2.connect(
78         dbname=os.getenv("POSTGRES_DB"),
79         user=os.getenv("POSTGRES_USER"),
80         password=os.getenv("POSTGRES_PASSWORD"),
81         host=os.getenv("POSTGRES_HOSTNAME"),
82         port=os.getenv("POSTGRES_PORT"),
83     )
84
85     conn.set_isolation_level(ISOLATION_LEVEL_AUTOCOMMIT)
86     cursor = conn.cursor()
87     for statement in statements:
88         cursor.execute(statement)
89
90     cursor.close()
91     conn.close()
92
93
94 def wait_for_logs(cmdline, message):
95     logs = subprocess.check_output(cmdline)
96     while message not in logs.decode("utf-8"):
97         time.sleep(1)
98     logs = subprocess.check_output(cmdline)
99
100
101 @cli.command()
102 @click.argument("args", nargs=-1)
103 def test(args):
104     os.environ["APPLICATION_CONFIG"] = "testing"
105     configure_app(os.getenv("APPLICATION_CONFIG"))
106
107     cmdline = docker_compose_cmdline("up -d")
108     subprocess.call(cmdline)
109
110     cmdline = docker_compose_cmdline("logs postgres")
111     wait_for_logs(cmdline, "ready to accept connections")
112
113     run_sql([f"CREATE DATABASE {os.getenv('APPLICATION_DB')}"])
114
115     cmdline = [
```

```

116         "pytest",
117         "-svv",
118         "--cov=application",
119         "--cov-report=term-missing",
120     ]
121     cmdline.extend(args)
122     subprocess.call(cmdline)
123
124     cmdline = docker_compose_cmdline("down")
125     subprocess.call(cmdline)
126
127
128 if __name__ == "__main__":
129     cli()

```

Let's see what it does block by block.

```

1  #! /usr/bin/env python
2
3  import os
4  import json
5  import subprocess
6  import time
7
8  import click
9  import psycopg2
10 from psycopg2.extensions import ISOLATION_LEVEL_AUTOCOMMIT
11
12
13 # Ensure an environment variable exists and has a value
14 def setenv(variable, default):
15     os.environ[variable] = os.getenv(variable, default)
16
17
18 APPLICATION_CONFIG_PATH = "config"
19 DOCKER_PATH = "docker"

```

Some Docker containers (like the PostgreSQL one that we will use shortly) depend on environment variables to perform the initial setup, so we need to define a function to set environment variables if they are not already initialised. We also define a couple of paths for configuration files.

```

1  def app_config_file(config):
2      return os.path.join(APPLICATION_CONFIG_PATH, f"{config}.json")
3
4
5  def docker_compose_file(config):
6      return os.path.join(DOCKER_PATH, f"{config}.yaml")
7
8
9  def read_json_configuration(config):
10     # Read configuration from the relative JSON file
11     with open(app_config_file(config)) as f:
12         config_data = json.load(f)
13
14     # Convert the config into a usable Python dictionary
15     config_data = dict((i["name"], i["value"]) for i in config_data)
16
17     return config_data
18
19
20 def configure_app(config):
21     configuration = read_json_configuration(config)
22
23     for key, value in configuration.items():
24         setenv(key, value)

```

As in principle I expect to have a different configuration at least for development, testing, and production, I introduced `app_config_file` and `docker_compose_file` that return the specific file for the environment we are working in. The function `read_json_configuration` has been isolated from `configure_app` as it will be imported by the tests to initialise the database repository.

```

1  @click.group()
2  def cli():
3      pass
4
5
6  def docker_compose_cmdline(commands_string=None):
7      config = os.getenv("APPLICATION_CONFIG")
8      configure_app(config)
9
10     compose_file = docker_compose_file(config)
11
12     if not os.path.isfile(compose_file):
13         raise ValueError(f"The file {compose_file} does not exist")

```

```

14
15     command_line = [
16         "docker-compose",
17         "-p",
18         config,
19         "-f",
20         compose_file,
21     ]
22
23     if commands_string:
24         command_line.extend(commands_string.split(" "))
25
26     return command_line

```

This is a simple function that creates the Docker Compose command line that avoids repeating long lists of options whenever we need to orchestrate the containers.

```

1  def run_sql(statements):
2      conn = psycopg2.connect(
3          dbname=os.getenv("POSTGRES_DB"),
4          user=os.getenv("POSTGRES_USER"),
5          password=os.getenv("POSTGRES_PASSWORD"),
6          host=os.getenv("POSTGRES_HOSTNAME"),
7          port=os.getenv("POSTGRES_PORT"),
8      )
9
10     conn.set_isolation_level(ISOLATION_LEVEL_AUTOCOMMIT)
11     cursor = conn.cursor()
12     for statement in statements:
13         cursor.execute(statement)
14
15     cursor.close()
16     conn.close()
17
18
19  def wait_for_logs(cmdline, message):
20     logs = subprocess.check_output(cmdline)
21     while message not in logs.decode("utf-8"):
22         time.sleep(1)
23     logs = subprocess.check_output(cmdline)

```

The function `run_sql` allows us to run SQL commands on a running Postgres database, and will come in handy when we will create the empty test database. The second function, `wait_for_logs`

is a simple way to monitor the Postgres container and to be sure it's ready to be used. Whenever you spin up containers programmatically you need to be aware that they have a certain startup time before they are ready, and act accordingly.

```

1  @cli.command()
2  @click.argument("args", nargs=-1)
3  def test(args):
4      os.environ["APPLICATION_CONFIG"] = "testing"
5      configure_app(os.getenv("APPLICATION_CONFIG"))
6
7      cmdline = docker_compose_cmdline("up -d")
8      subprocess.call(cmdline)
9
10     cmdline = docker_compose_cmdline("logs postgres")
11     wait_for_logs(cmdline, "ready to accept connections")
12
13     run_sql([f"CREATE DATABASE {os.getenv('APPLICATION_DB')}"])
14
15     cmdline = [
16         "pytest",
17         "-svv",
18         "--cov=application",
19         "--cov-report=term-missing",
20     ]
21     cmdline.extend(args)
22     subprocess.call(cmdline)
23
24     cmdline = docker_compose_cmdline("down")
25     subprocess.call(cmdline)
26
27
28  if __name__ == "__main__":
29      cli()

```

This function is the last that we define, and the only command provided by our management script. First of all the application is configured with the name `testing`, which means that we will use the configuration file `config/testing.json` and the Docker Compose file `docker/testing.yml`. All these names and paths are just conventions that comes from the arbitrary setup of this management script, so you are clearly free to structure your project in a different way.

The function then spins up the containers according to the Docker Compose file, running `docker-compose up -d`. It waits for the log message that communicates the database is ready to accept connections and runs the SQL command that creates the testing database.

After this it runs Pytest with a default set of options, adding all the options that we will provide on the command line, and eventually tears down the Docker Compose containers.

To complete the setup we need to define a configuration file for Docker Compose

docker/testing.yml

```
1 version: '3.8'
2
3 services:
4   postgres:
5     image: postgres
6     environment:
7       POSTGRES_DB: ${POSTGRES_DB}
8       POSTGRES_USER: ${POSTGRES_USER}
9       POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
10    ports:
11      - "${POSTGRES_PORT}:5432"
```

And finally a JSON configuration file

config/testing.json

```
1 [
2   {
3     "name": "FLASK_ENV",
4     "value": "production"
5   },
6   {
7     "name": "FLASK_CONFIG",
8     "value": "testing"
9   },
10  {
11    "name": "POSTGRES_DB",
12    "value": "postgres"
13  },
14  {
15    "name": "POSTGRES_USER",
16    "value": "postgres"
17  },
18  {
19    "name": "POSTGRES_HOSTNAME",
20    "value": "localhost"
21  },
22  {
```

```
23     "name": "POSTGRES_PORT",
24     "value": "5433"
25 },
26 {
27     "name": "POSTGRES_PASSWORD",
28     "value": "postgres"
29 },
30 {
31     "name": "APPLICATION_DB",
32     "value": "test"
33 }
34 ]
```

A couple of notes about this configuration. First of all it defines both `FLASK_ENV` and `FLASK_CONFIG`. The first is, as you might remember, an internal Flask variable that can only be `development` or `production`, and is connected with the internal debugger. The second is the variable that we use to configure our Flask application with the objects in `application/config.py`. For testing purposes we set `FLASK_ENV` to `production` as we don't need the internal debugger, and `FLASK_CONFIG` to `testing`, which will result in the application being configured with the class `TestingConfig`. This class sets the internal Flask parameter `TESTING` to `True`.

The rest of the JSON configuration initialises variables whose names start with the prefix `POSTGRES_`. These are variables required by the Postgres Docker container. When the container is run, it automatically creates a database with the name specified by `POSTGRES_DB`. It also creates a user with a password, using the values specified in `POSTGRES_USER` and `POSTGRES_PASSWORD`.

Last, I introduced the variable `APPLICATION_DB` because I want to create a specific database which is not the one the default one. The default port `POSTGRES_PORT` has been changed from the standard value 5432 to 5433 to avoid clashing with any database already running on the machine (either natively or containerised). As you can see in the Docker Compose configuration file this changes only the external mapping of the container and not the actual port the database engine is using inside the container.

With all these files in place we are ready to start designing our tests.



Source code

<https://github.com/pycabook/rentomatic/tree/ed2-c06-s03>

Database fixtures

As we defined the configuration of the database in a JSON file we need a fixture that loads that same configuration, so that we can connect to the database during the tests. As we already have the

function `read_json_configuration` in the management script we just need to wrap that. This is a fixture that is not specific to the Postgres repository, so I will introduce it in `tests/conftest.py`

tests/conftest.py

```

1  from manage import read_json_configuration
2
3  ...
4
5  @pytest.fixture(scope="session")
6  def app_configuration():
7      return read_json_configuration("testing")

```

As you can see I hardcoded the name of the configuration file for simplicity's sake. Another solution might be to create an environment variable with the application configuration in the management script and to read it from here.

The rest of the fixtures contains code that is specific to Postgres, so it is better to keep the code separated in a more specific file `conftest.py`

tests/repository/postgres/conftest.py

```

1  import sqlalchemy
2  import pytest
3
4  from rentomatic.repository.postgres_objects import Base, Room
5
6
7  @pytest.fixture(scope="session")
8  def pg_session_empty(app_configuration):
9      conn_str = "postgresql+psycopg2://{user}:{password}@{host}:{port}/{db}".format(
10         app_configuration["POSTGRES_USER"],
11         app_configuration["POSTGRES_PASSWORD"],
12         app_configuration["POSTGRES_HOSTNAME"],
13         app_configuration["POSTGRES_PORT"],
14         app_configuration["APPLICATION_DB"],
15     )
16     engine = sqlalchemy.create_engine(conn_str)
17     connection = engine.connect()
18
19     Base.metadata.create_all(engine)
20     Base.metadata.bind = engine
21
22     DBSession = sqlalchemy.orm.sessionmaker(bind=engine)
23     session = DBSession()

```

```
24
25     yield session
26
27     session.close()
28     connection.close
29
30
31 @pytest.fixture(scope="session")
32 def pg_test_data():
33     return [
34         {
35             "code": "f853578c-fc0f-4e65-81b8-566c5dffa35a",
36             "size": 215,
37             "price": 39,
38             "longitude": -0.09998975,
39             "latitude": 51.75436293,
40         },
41         {
42             "code": "fe2c3195-aeff-487a-a08f-e0bdc0ec6e9a",
43             "size": 405,
44             "price": 66,
45             "longitude": 0.18228006,
46             "latitude": 51.74640997,
47         },
48         {
49             "code": "913694c6-435a-4366-ba0d-da5334a611b2",
50             "size": 56,
51             "price": 60,
52             "longitude": 0.27891577,
53             "latitude": 51.45994069,
54         },
55         {
56             "code": "eed76e77-55c1-41ce-985d-ca49bf6c0585",
57             "size": 93,
58             "price": 48,
59             "longitude": 0.33894476,
60             "latitude": 51.39916678,
61         },
62     ]
63
64
65 @pytest.fixture(scope="function")
66 def pg_session(pg_session_empty, pg_test_data):
```

```

67     for r in pg_test_data:
68         new_room = Room(
69             code=r["code"],
70             size=r["size"],
71             price=r["price"],
72             longitude=r["longitude"],
73             latitude=r["latitude"],
74         )
75         pg_session_empty.add(new_room)
76         pg_session_empty.commit()
77
78     yield pg_session_empty
79
80     pg_session_empty.query(Room).delete()

```

The first fixture `pg_session_empty` creates a session to the empty initial database, while `pg_test_data` defines the values that we will load into the database. As we are not mutating this set of values we don't need to create a fixture, but this is the easier way to make it available both to the other fixtures and to the tests. The last fixture `pg_session` fills the database with Postgres objects created with the test data. Pay attention that these are not entities, but the Postgres objects we created to map them.

Note that this last fixture has a function scope, thus it is run for every test. Therefore, we delete all rooms after the yield returns, leaving the database exactly as it was before the test. Generally speaking you should always clean up after tests. The endpoint we are testing does not write to the database so in this specific case there is no real need to clean up, but I prefer to implement a complete solution from step zero.

We can test this whole setup changing the function `test_dummy` so that it fetches all the rows of the table `Room` and verifying that the query returns 4 values.

The new version of `tests/repository/postgres/test_postgresrepo.py` is

```

1  import pytest
2  from rentomatic.repository.postgres_objects import Room
3
4  pytestmark = pytest.mark.integration
5
6
7  def test_dummy(pg_session):
8      assert len(pg_session.query(Room).all()) == 4

```

At this point you can run the test suite with integration tests. You should notice a clear delay when pytest executes the function `test_dummy` as Docker will take some time to spin up the database container and prepare the data

```

1  $ ./manage.py test -- --integration
2  ===== test session starts =====
3  platform linux -- Python XXXX, pytest-XXXX, py-XXXX, pluggy-XXXX --
4  cabook/venv3/bin/python3
5  cachedir: .cache
6  rootdir: cabook/code/calc, inifile: pytest.ini
7  plugins: cov-XXXX
8  collected 36 items
9
10 ...
11 tests/repository/postgres/test_postgresrepo.py::test_dummy PASSED
12 ...
13
14 ===== 36 passed in 0.26s =====

```

Note that to pass the option `--integration` we need to use `--` otherwise Click would consider the option as belonging to the script `./manage.py` instead of passing it as a pytest argument.



Source code

<https://github.com/pycabook/rentomatic/tree/ed2-c06-s04>

Integration tests

At this point we can create the real tests in the file `test_postgresrepo.py`, replacing the function `test_dummy`. All test receive the fixtures `app_configuration`, `pg_session`, and `pg_test_data`. The first fixture allows us to initialise the class `PostgresRepo` using the proper parameters. The second creates the database using the test data that is then contained in the third fixture.

The tests for this repository are basically a copy of the ones created for `MemRepo`, which is not surprising. Usually, you want to test the very same conditions, whatever the storage system. Towards the end of the chapter we will see, however, that while these files are initially the same, they can evolve differently as we find bugs or corner cases that come from the specific implementation (in-memory storage, PostgreSQL, and so on).

tests/repository/postgres/test_postgresrepo.py

```
1 import pytest
2 from rentomatic.repository import postgresrepo
3
4 pytestmark = pytest.mark.integration
5
6
7 def test_repository_list_without_parameters(
8     app_configuration, pg_session, pg_test_data
9 ):
10     repo = postgresrepo.PostgresRepo(app_configuration)
11
12     repo_rooms = repo.list()
13
14     assert set([r.code for r in repo_rooms]) == set(
15         [r["code"] for r in pg_test_data]
16     )
17
18
19 def test_repository_list_with_code_equal_filter(
20     app_configuration, pg_session, pg_test_data
21 ):
22     repo = postgresrepo.PostgresRepo(app_configuration)
23
24     repo_rooms = repo.list(
25         filters={"code__eq": "fe2c3195-aeff-487a-a08f-e0bdc0ec6e9a"}
26     )
27
28     assert len(repo_rooms) == 1
29     assert repo_rooms[0].code == "fe2c3195-aeff-487a-a08f-e0bdc0ec6e9a"
30
31
32 def test_repository_list_with_price_equal_filter(
33     app_configuration, pg_session, pg_test_data
34 ):
35     repo = postgresrepo.PostgresRepo(app_configuration)
36
37     repo_rooms = repo.list(filters={"price__eq": 60})
38
39     assert len(repo_rooms) == 1
40     assert repo_rooms[0].code == "913694c6-435a-4366-ba0d-da5334a611b2"
41
42
```

```
43 def test_repository_list_with_price_less_than_filter(  
44     app_configuration, pg_session, pg_test_data  
45 ):  
46     repo = postgresrepo.PostgresRepo(app_configuration)  
47  
48     repo_rooms = repo.list(filters={"price__lt": 60})  
49  
50     assert len(repo_rooms) == 2  
51     assert set([r.code for r in repo_rooms]) == {  
52         "f853578c-fc0f-4e65-81b8-566c5dffa35a",  
53         "eed76e77-55c1-41ce-985d-ca49bf6c0585",  
54     }  
55  
56  
57 def test_repository_list_with_price_greater_than_filter(  
58     app_configuration, pg_session, pg_test_data  
59 ):  
60     repo = postgresrepo.PostgresRepo(app_configuration)  
61  
62     repo_rooms = repo.list(filters={"price__gt": 48})  
63  
64     assert len(repo_rooms) == 2  
65     assert set([r.code for r in repo_rooms]) == {  
66         "913694c6-435a-4366-ba0d-da5334a611b2",  
67         "fe2c3195-aeff-487a-a08f-e0bdc0ec6e9a",  
68     }  
69  
70  
71 def test_repository_list_with_price_between_filter(  
72     app_configuration, pg_session, pg_test_data  
73 ):  
74     repo = postgresrepo.PostgresRepo(app_configuration)  
75  
76     repo_rooms = repo.list(filters={"price__lt": 66, "price__gt": 48})  
77  
78     assert len(repo_rooms) == 1  
79     assert repo_rooms[0].code == "913694c6-435a-4366-ba0d-da5334a611b2"
```

Remember that I introduced these tests one at a time and that I'm not showing you the full TDD workflow only for brevity's sake. The code of the class `PostgresRepo` has been developed following a strict TDD approach, and I recommend you to do the same. The resulting code goes in `rentomatic/repository/postgresrepo.py`, the same directory where we created the file

postgres_objects.py.

rentomatic/repository/postgresrepo.py

```

1  from sqlalchemy import create_engine
2  from sqlalchemy.orm import sessionmaker
3
4  from rentomatic.domain import room
5  from rentomatic.repository.postgres_objects import Base, Room
6
7
8  class PostgresRepo:
9      def __init__(self, configuration):
10         connection_string = "postgresql+psycopg2://{user}:{password}@{host}/{db}".format(
11             configuration["POSTGRES_USER"],
12             configuration["POSTGRES_PASSWORD"],
13             configuration["POSTGRES_HOSTNAME"],
14             configuration["POSTGRES_PORT"],
15             configuration["APPLICATION_DB"],
16         )
17
18         self.engine = create_engine(connection_string)
19         Base.metadata.create_all(self.engine)
20         Base.metadata.bind = self.engine
21
22     def _create_room_objects(self, results):
23         return [
24             room.Room(
25                 code=q.code,
26                 size=q.size,
27                 price=q.price,
28                 latitude=q.latitude,
29                 longitude=q.longitude,
30             )
31             for q in results
32         ]
33
34     def list(self, filters=None):
35         DBSession = sessionmaker(bind=self.engine)
36         session = DBSession()
37
38         query = session.query(Room)
39
40         if filters is None:

```

```
41         return self._create_room_objects(query.all())
42
43     if "code__eq" in filters:
44         query = query.filter(Room.code == filters["code__eq"])
45
46     if "price__eq" in filters:
47         query = query.filter(Room.price == filters["price__eq"])
48
49     if "price__lt" in filters:
50         query = query.filter(Room.price < filters["price__lt"])
51
52     if "price__gt" in filters:
53         query = query.filter(Room.price > filters["price__gt"])
54
55     return self._create_room_objects(query.all())
```



Source code

<https://github.com/pycabook/rentomatic/tree/ed2-c06-s05>

You might notice that PostgresRepo is very similar to MemRepo. This is the case because the case we are dealing with here, the list of Room objects, is pretty simple, so I don't expect great differences between an in-memory database and a production-ready relational one. As the use cases get more complex you will need to start leveraging the features provided by the engine that you are using, and methods such as `list` might evolve to become very different.

Note that the method `list` returns domain models, which is allowed as the repository is implemented in one of the outer layers of the architecture.

As you can see, while setting up a proper integration testing environment is not trivial, the changes that our architecture required to work with a real repository are very limited. I think this is a good demonstration of the flexibility of a layered approach such as the one at the core of the clean architecture.

Since this chapter mixed the setup of the integration testing with the introduction of a new repository, I will dedicate the next chapter purely to introduce a repository based on MongoDB, using the same structure that I created in this chapter. Supporting multiple databases (in this case even relational and non-relational) is not an uncommon pattern, as it allows you to use the approach that best suits each use case.

Chapter 07 Integration with a real external system mongodb

”

There's, uh, another example.

Jurassic Park, 1993

The previous chapter showed how to integrate a real external system with the core of the clean architecture. Unfortunately I also had to introduce a lot of code to manage the integration tests and to globally move forward to a proper setup. In this chapter I will leverage the work we just did to show only the part strictly connected with the external system. Swapping the database from PostgreSQL to MongoDB is the perfect way to show how flexible the clean architecture is, and how easy it is to introduce different approaches like a non-relational database instead of a relational one.

Fixtures

Thanks to the flexibility of clean architecture, providing support for multiple storage systems is a breeze. In this section, I will implement the class `MongoRepo` that provides an interface towards MongoDB, a well-known NoSQL database. We will follow the same testing strategy we used for PostgreSQL, with a Docker container that runs the database and docker-compose that orchestrates the whole system.

You will appreciate the benefits of the complex testing structure that I created in the previous chapter. That structure allows me to reuse some of the fixtures now that I want to implement tests for a new storage system.

Let's start defining the file `tests/repository/mongodb/conftest.py`, which will contains pytest fixtures for MongoDB, mirroring the file we created for PostgreSQL

tests/repository/mongodb/conftest.py

```
1 import pymongo
2 import pytest
3
4
5 @pytest.fixture(scope="session")
6 def mg_database_empty(app_configuration):
7     client = pymongo.MongoClient(
8         host=app_configuration["MONGODB_HOSTNAME"],
9         port=int(app_configuration["MONGODB_PORT"]),
10        username=app_configuration["MONGODB_USER"],
11        password=app_configuration["MONGODB_PASSWORD"],
12        authSource="admin",
13    )
14    db = client[app_configuration["APPLICATION_DB"]]
15
16    yield db
17
18    client.drop_database(app_configuration["APPLICATION_DB"])
19    client.close()
20
21
22 @pytest.fixture(scope="function")
23 def mg_test_data():
24     return [
25         {
26             "code": "f853578c-fc0f-4e65-81b8-566c5dffa35a",
27             "size": 215,
28             "price": 39,
29             "longitude": -0.09998975,
30             "latitude": 51.75436293,
31         },
32         {
33             "code": "fe2c3195-aeff-487a-a08f-e0bdc0ec6e9a",
34             "size": 405,
35             "price": 66,
36             "longitude": 0.18228006,
37             "latitude": 51.74640997,
38         },
39         {
40             "code": "913694c6-435a-4366-ba0d-da5334a611b2",
41             "size": 56,
42             "price": 60,
```

```

43         "longitude": 0.27891577,
44         "latitude": 51.45994069,
45     },
46     {
47         "code": "eed76e77-55c1-41ce-985d-ca49bf6c0585",
48         "size": 93,
49         "price": 48,
50         "longitude": 0.33894476,
51         "latitude": 51.39916678,
52     },
53 ]
54
55
56 @pytest.fixture(scope="function")
57 def mg_database(mg_database_empty, mg_test_data):
58     collection = mg_database_empty.rooms
59
60     collection.insert_many(mg_test_data)
61
62     yield mg_database_empty
63
64     collection.delete_many({})

```

As you can see these functions are very similar to the ones that we defined for Postgres. The function `mg_database_empty` is tasked to create the MongoDB client and the empty database, and to dispose them after the `yield`. The fixture `mg_test_data` provides the same data provided by `pg_test_data` and `mg_database` fills the empty database with it. While the SQLAlchemy package works through a session, PyMongo library creates a client and uses it directly, but the overall structure is the same.

Since we are importing the PyMongo library we need to change the production requirements

requirements/prod.txt

```

1 Flask
2 SQLAlchemy
3 psycopg2
4 pymongo

```

and run `pip install -r requirements/dev.txt`.



Source code

<https://github.com/pycabook/rentomatic/tree/ed2-c07-s01>

Docker Compose configuration

We need to add an ephemeral MongoDB container to the testing Docker Compose configuration. The MongoDB image needs only the variables `MONGO_INITDB_ROOT_USERNAME` and `MONGO_INITDB_ROOT_PASSWORD` as it doesn't create any initial database. As we did for the PostgreSQL container we assign a specific port that will be different from the standard one, to allow tests to be executed while other containers are running.

`docker/testing.yml`

```
1 version: '3.8'
2
3 services:
4   postgres:
5     image: postgres
6     environment:
7       POSTGRES_DB: ${POSTGRES_DB}
8       POSTGRES_USER: ${POSTGRES_USER}
9       POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
10    ports:
11      - "${POSTGRES_PORT}:5432"
12   mongo:
13     image: mongo
14     environment:
15       MONGO_INITDB_ROOT_USERNAME: ${MONGODB_USER}
16       MONGO_INITDB_ROOT_PASSWORD: ${MONGODB_PASSWORD}
17     ports:
18      - "${MONGODB_PORT}:27017"
```



Source code

<https://github.com/pycabook/rentomatic/tree/ed2-c07-s02>

Application configuration

Docker Compose, the testing framework, and the application itself are configured through a single JSON file, that we need to update with the actual values we want to use for MongoDB

config/testing.json

```
1  [  
2    {  
3      "name": "FLASK_ENV",  
4      "value": "production"  
5    },  
6    {  
7      "name": "FLASK_CONFIG",  
8      "value": "testing"  
9    },  
10   {  
11     "name": "POSTGRES_DB",  
12     "value": "postgres"  
13   },  
14   {  
15     "name": "POSTGRES_USER",  
16     "value": "postgres"  
17   },  
18   {  
19     "name": "POSTGRES_HOSTNAME",  
20     "value": "localhost"  
21   },  
22   {  
23     "name": "POSTGRES_PORT",  
24     "value": "5433"  
25   },  
26   {  
27     "name": "POSTGRES_PASSWORD",  
28     "value": "postgres"  
29   },  
30   {  
31     "name": "MONGODB_USER",  
32     "value": "root"  
33   },  
34   {  
35     "name": "MONGODB_HOSTNAME",  
36     "value": "localhost"  
37   },  
38   {  
39     "name": "MONGODB_PORT",  
40     "value": "27018"  
41   },  
42   {
```

```
43     "name": "MONGODB_PASSWORD",
44     "value": "mongodb"
45 },
46 {
47     "name": "APPLICATION_DB",
48     "value": "test"
49 }
50 ]
```

Since the standard port from MongoDB is 27017 I chose 27018 for the tests. Remember that this is just an example, however. In a real scenario we might have multiple environments and also multiple setups for our testing, and in that case we might want to assign a random port to the container and use Python to extract the value and pass it to the application.

Please also note that I chose to use the same variable `APPLICATION_DB` for the name of the PostgreSQL and MongoDB databases. Again, this is a simple example, and your mileage may vary in more complex scenarios.



Source code

<https://github.com/pycabook/rentomatic/tree/ed2-c07-s03>

Integration tests

The integration tests are a mirror of the ones we wrote for Postgres, as we are covering the same use case. If you use multiple databases in the same system you probably want to serve different use cases, so in a real case this might probably be a more complicated step. It is completely reasonable, however, that you might want to simply provide support for multiple databases that your client can choose to plug into the system, and in that case you will do exactly what I did here, copying and adjusting the same test battery.

`tests/repository/mongodb/test_mongorepo.py`

```
1 import pytest
2 from rentomatic.repository import mongorepo
3
4 pytestmark = pytest.mark.integration
5
6
7 def test_repository_list_without_parameters(
8     app_configuration, mg_database, mg_test_data
9 ):
```



```
10     repo = mongorepo.MongoRepo(app_configuration)
11
12     repo_rooms = repo.list()
13
14     assert set([r.code for r in repo_rooms]) == set(
15         [r["code"] for r in mg_test_data]
16     )
17
18
19 def test_repository_list_with_code_equal_filter(
20     app_configuration, mg_database, mg_test_data
21 ):
22     repo = mongorepo.MongoRepo(app_configuration)
23
24     repo_rooms = repo.list(
25         filters={"code__eq": "fe2c3195-aeff-487a-a08f-e0bdc0ec6e9a"}
26     )
27
28     assert len(repo_rooms) == 1
29     assert repo_rooms[0].code == "fe2c3195-aeff-487a-a08f-e0bdc0ec6e9a"
30
31
32 def test_repository_list_with_price_equal_filter(
33     app_configuration, mg_database, mg_test_data
34 ):
35     repo = mongorepo.MongoRepo(app_configuration)
36
37     repo_rooms = repo.list(filters={"price__eq": 60})
38
39     assert len(repo_rooms) == 1
40     assert repo_rooms[0].code == "913694c6-435a-4366-ba0d-da5334a611b2"
41
42
43 def test_repository_list_with_price_less_than_filter(
44     app_configuration, mg_database, mg_test_data
45 ):
46     repo = mongorepo.MongoRepo(app_configuration)
47
48     repo_rooms = repo.list(filters={"price__lt": 60})
49
50     assert len(repo_rooms) == 2
51     assert set([r.code for r in repo_rooms]) == {
52         "f853578c-fc0f-4e65-81b8-566c5dffa35a",
```

```
53         "eed76e77-55c1-41ce-985d-ca49bf6c0585",
54     }
55
56
57 def test_repository_list_with_price_greater_than_filter(
58     app_configuration, mg_database, mg_test_data
59 ):
60     repo = mongorepo.MongoRepo(app_configuration)
61
62     repo_rooms = repo.list(filters={"price__gt": 48})
63
64     assert len(repo_rooms) == 2
65     assert set([r.code for r in repo_rooms]) == {
66         "913694c6-435a-4366-ba0d-da5334a611b2",
67         "fe2c3195-aeff-487a-a08f-e0bdc0ec6e9a",
68     }
69
70
71 def test_repository_list_with_price_between_filter(
72     app_configuration, mg_database, mg_test_data
73 ):
74     repo = mongorepo.MongoRepo(app_configuration)
75
76     repo_rooms = repo.list(filters={"price__lt": 66, "price__gt": 48})
77
78     assert len(repo_rooms) == 1
79     assert repo_rooms[0].code == "913694c6-435a-4366-ba0d-da5334a611b2"
80
81
82 def test_repository_list_with_price_as_string(
83     app_configuration, mg_database, mg_test_data
84 ):
85     repo = mongorepo.MongoRepo(app_configuration)
86
87     repo_rooms = repo.list(filters={"price__lt": "60"})
88
89     assert len(repo_rooms) == 2
90     assert set([r.code for r in repo_rooms]) == {
91         "f853578c-fc0f-4e65-81b8-566c5dffa35a",
92         "eed76e77-55c1-41ce-985d-ca49bf6c0585",
93     }
```

I added a test called `test_repository_list_with_price_as_string` that checks what happens when the price in the filter is expressed as a string. Experimenting with the MongoDB shell I found that in this case the query wasn't working, so I included the test to be sure the implementation didn't forget to manage this condition.



Source code

<https://github.com/pycabook/rentomatic/tree/ed2-c07-s04>

The MongoDB repository

The `MongoRepo` class is obviously not the same as the Postgres interface, as the PyMongo library is different from SQLAlchemy, and the structure of a NoSQL database differs from the one of a relational one. The file `rentomatic/repository/mongorepo.py` is

`rentomatic/repository/mongorepo.py`

```
1  import pymongo
2
3  from rentomatic.domain import room
4
5
6  class MongoRepo:
7      def __init__(self, configuration):
8          client = pymongo.MongoClient(
9              host=configuration["MONGODB_HOSTNAME"],
10             port=int(configuration["MONGODB_PORT"]),
11             username=configuration["MONGODB_USER"],
12             password=configuration["MONGODB_PASSWORD"],
13             authSource="admin",
14         )
15
16         self.db = client[configuration["APPLICATION_DB"]]
17
18     def _create_room_objects(self, results):
19         return [
20             room.Room(
21                 code=q["code"],
22                 size=q["size"],
23                 price=q["price"],
24                 latitude=q["latitude"],
25                 longitude=q["longitude"],
```

```

26         )
27         for q in results
28     ]
29
30     def list(self, filters=None):
31         collection = self.db.rooms
32
33         if filters is None:
34             result = collection.find()
35         else:
36             mongo_filter = {}
37             for key, value in filters.items():
38                 key, operator = key.split("__")
39
40                 filter_value = mongo_filter.get(key, {})
41
42                 if key == "price":
43                     value = int(value)
44
45                 filter_value["${}".format(operator)] = value
46                 mongo_filter[key] = filter_value
47
48             result = collection.find(mongo_filter)
49
50     return self._create_room_objects(result)

```

which makes use of the similarity between the filters of the Rent-o-matic project and the ones of the MongoDB system^{footnote}: [The similitude between the two systems is not accidental, as I was studying MongoDB at the time I wrote the first article about clean architectures, so I was obviously influenced by it].



Source code

<https://github.com/pycabook/rentomatic/tree/ed2-c07-s05>

I think this very brief chapter clearly showed the merits of a layered approach and of a proper testing setup. So far we implemented and tested an interface towards two very different databases like PostgreSQL and MongoDB, but both interfaces are usable by the same use case, which ultimately means the same API endpoint.

While we properly tested the integration with these external systems, we still don't have a way to run the whole system in what we call a production-ready environment, that is in a way that can be exposed to external users. In the next chapter I will show you how we can leverage the same setup we used for the tests to run Flask, PostgreSQL, and the use case we created in a way that can be used in production.

Chapter 08 Run a production ready system



Vilos Coahaagen said troops would be used to ensure full production.

Total Recall, 1990

Now that we developed a repository that connects with PostgreSQL we can discuss how to properly set up the application to run a production-ready system. This part is not strictly related to the clean architecture, but I think it's worth completing the example, showing how the system that we designed can end up being the core of a real web application.

Clearly, the definition “production-ready” refers to many different configuration that ultimately depend on the load and the business requirements of the system. As the goal is to show a complete example and not to cover real production requirements I will show a solution that uses real external systems like PostgreSQL and Nginx, without being too concerned about performances.

Build a web stack

Now that we successfully containerised the tests we might try to devise a production-ready setup of the whole application, running both a web server and a database in Docker containers. Once again, I will follow the approach that I show in the series of posts I mentioned in one of the previous sections.

To run a production-ready infrastructure we need to put a WSGI server in front of the web framework and a Web server in front of it. We will also need to run a database container that we will initialise only once.

The steps towards a production-ready configuration are not complicated and the final setup won't be ultimately too different from what we already did for the tests. We need to

- Create a JSON configuration with environment variables suitable for production
- Create a suitable configuration for Docker Compose and configure the containers
- Add commands to manage `.py` that allow us to control the processes

Let's create the file `config/production.json`, which is very similar to the one we created for the tests

config/production.json

```
1  [  
2    {  
3      "name": "FLASK_ENV",  
4      "value": "production"  
5    },  
6    {  
7      "name": "FLASK_CONFIG",  
8      "value": "production"  
9    },  
10   {  
11     "name": "POSTGRES_DB",  
12     "value": "postgres"  
13   },  
14   {  
15     "name": "POSTGRES_USER",  
16     "value": "postgres"  
17   },  
18   {  
19     "name": "POSTGRES_HOSTNAME",  
20     "value": "localhost"  
21   },  
22   {  
23     "name": "POSTGRES_PORT",  
24     "value": "5432"  
25   },  
26   {  
27     "name": "POSTGRES_PASSWORD",  
28     "value": "postgres"  
29   },  
30   {  
31     "name": "APPLICATION_DB",  
32     "value": "application"  
33   }  
34 ]
```

Please note that now both `FLASK_ENV` and `FLASK_CONFIG` are set to `production`. Please remember that the first is an internal Flask variable with two possible fixed values (`development` and `production`), while the second one is an arbitrary name that has the final effect of loading a specific configuration object (`ProductionConfig` in this case). I also changed `POSTGRES_PORT` back to the default `5432` and `APPLICATION_DB` to `application` (an arbitrary name).

Let's define which containers we want to run in our production environment, and how we want to connect them. We need a production-ready database and I will use Postgres, as I already did during the tests. Then we need to wrap Flask with a production HTTP server, and for this job I will use gunicorn. Last, we need a Web Server to act as load balancer.

The file `docker/production.yml` will contain the Docker Compose configuration, according to the convention we defined in `manage.py`

docker/production.yml

```
1  version: '3.8'
2
3  services:
4    db:
5      image: postgres
6      environment:
7        POSTGRES_DB: ${POSTGRES_DB}
8        POSTGRES_USER: ${POSTGRES_USER}
9        POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
10     ports:
11       - "${POSTGRES_PORT}:5432"
12     volumes:
13       - pgdata:/var/lib/postgresql/data
14   web:
15     build:
16       context: ${PWD}
17       dockerfile: docker/web/Dockerfile.production
18     environment:
19       FLASK_ENV: ${FLASK_ENV}
20       FLASK_CONFIG: ${FLASK_CONFIG}
21       APPLICATION_DB: ${APPLICATION_DB}
22       POSTGRES_USER: ${POSTGRES_USER}
23       POSTGRES_HOSTNAME: "db"
24       POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
25       POSTGRES_PORT: ${POSTGRES_PORT}
26     command: gunicorn -w 4 -b 0.0.0.0 wsgi:app
27     volumes:
28       - ${PWD}:/opt/code
29   nginx:
30     image: nginx
31     volumes:
32       - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
33     ports:
34       - 8080:8080
35
```



```
36 volumes:  
37 pgdata:
```

As you can see the Postgres configuration is not different from the one we used in the file `testing.yml`, but I added the option `volumes` (both in `db` and at the end of the file) that allows me to create a stable volume. If you don't do it, the database will be destroyed once you shut down the container.

The container `web` runs the Flask application through `gunicorn`. The environment variables come once again from the JSON configuration, and we need to define them because the application needs to know how to connect with the database and how to run the web framework. The command `gunicorn -w 4 -b 0.0.0.0 wsgi:app` loads the WSGI application we created in `wsgi.py` and runs it in 4 concurrent processes. This container is created using `docker/web/Dockerfile.production` which I still have to define.

The last container is `nginx`, which we will use as it is directly from the Docker Hub. The container runs Nginx with the configuration stored in `/etc/nginx/nginx.conf`, which is the file we overwrite with the local one `./nginx/nginx.conf`. Please note that I configured it to use port 8080 instead of the standard port 80 for HTTP to avoid clashing with other software that you might be running on your computer.

The Dockerfile for the web application is the following

docker/web/Dockerfile.production

```
1 FROM python:3  
2  
3 ENV PYTHONUNBUFFERED 1  
4  
5 RUN mkdir /opt/code  
6 RUN mkdir /opt/requirements  
7 WORKDIR /opt/code  
8  
9 ADD requirements /opt/requirements  
10 RUN pip install -r /opt/requirements/prod.txt
```

This is a very simple container that uses the standard `python:3` image, where I added the production requirements contained in `requirements/prod.txt`. To make the Docker container work we need to add `gunicorn` to this last file

requirements/prod.txt

```
1 Flask
2 SQLAlchemy
3 psycopg2
4 pymongo
5 gunicorn
```

The configuration for Nginx is

docker/nginx/nginx.conf

```
1 worker_processes 1;
2
3 events { worker_connections 1024; }
4
5 http {
6
7     sendfile on;
8
9     upstream app {
10         server web:8000;
11     }
12
13     server {
14         listen 8080;
15
16         location / {
17             proxy_pass      http://app;
18             proxy_redirect   off;
19             proxy_set_header Host $host;
20             proxy_set_header X-Real-IP $remote_addr;
21             proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
22             proxy_set_header X-Forwarded-Host $server_name;
23         }
24     }
25 }
```

As for the rest of the project, this configuration is very basic and lacks some important parts that are mandatory in a real production environment, such as HTTPS. In its essence, though, it is however not too different from the configuration of a production-ready Nginx container.

As we will use Docker Compose, the script `manage.py` needs a simple change, which is a command that wraps `docker-compose` itself. We need the script to just initialise environment variables

according to the content of the JSON configuration file and then run Docker Compose. As we already have the function `docker_compose_cmdline` the job is pretty simple

manage.py

```

1  # Ensure an environment variable exists and has a value
2  import os
3  import json
4  import signal
5  import subprocess
6  import time
7
8  ...
9
10 def setenv(variable, default):
11     os.environ[variable] = os.getenv(variable, default)
12
13
14 setenv("APPLICATION_CONFIG", "production")
15
16 APPLICATION_CONFIG_PATH = "config"
17 DOCKER_PATH = "docker"
18
19 ...
20
21 @cli.command(context_settings={"ignore_unknown_options": True})
22 @click.argument("subcommand", nargs=-1, type=click.Path())
23 def compose(subcommand):
24     configure_app(os.getenv("APPLICATION_CONFIG"))
25     cmdline = docker_compose_cmdline() + list(subcommand)
26
27     try:
28         p = subprocess.Popen(cmdline)
29         p.wait()
30     except KeyboardInterrupt:
31         p.send_signal(signal.SIGINT)
32         p.wait()

```

As you can see I forced the variable `APPLICATION_CONFIG` to be `production` if not specified. Usually, my default configuration is the development one, but in this simple case I haven't defined one, so this will do for now.

The new command is `compose`, that leverages Click's argument decorator to collect subcommands and attach them to the Docker Compose command line. I also use the `signal` library, which I added to the imports, to control keyboard interruptions.

**Source code**

<https://github.com/pycabook/rentomatic/tree/ed2-c08-s01>

When all these changes are in place we can test the application Dockerfile building the container.

```
1 $ ./manage.py compose build web
```

This command runs the Click command `compose` that first reads environment variables from the file `config/production.json`, and then runs `docker-compose` passing it the subcommand `build web`.

Your output should be the following (with different image IDs)

```
1 Building web
2 Step 1/7 : FROM python:3
3   ---> 768307cdb962
4 Step 2/7 : ENV PYTHONUNBUFFERED 1
5   ---> Using cache
6   ---> 0f2bb60286d3
7 Step 3/7 : RUN mkdir /opt/code
8   ---> Using cache
9   ---> e1278ef74291
10 Step 4/7 : RUN mkdir /opt/requirements
11   ---> Using cache
12   ---> 6d23f8abf0eb
13 Step 5/7 : WORKDIR /opt/code
14   ---> Using cache
15   ---> 8a3b6ae6d21c
16 Step 6/7 : ADD requirements /opt/requirements
17   ---> Using cache
18   ---> 75133f765531
19 Step 7/7 : RUN pip install -r /opt/requirements/prod.txt
20   ---> Using cache
21   ---> db644df9ba04
22
23 Successfully built db644df9ba04
24 Successfully tagged production_web:latest
```

If this is successful you can run Docker Compose

```

1 $ ./manage.py compose up -d
2 Creating production_web_1    ... done
3 Creating production_db_1    ... done
4 Creating production_nginx_1 ... done

```

and the output of `docker ps` should show three containers running

```

1 $ docker ps
2 IMAGE          PORTS          NAMES
3 nginx          80/tcp, 0.0.0.0:8080->8080/tcp  production_nginx_1
4 postgres       0.0.0.0:5432->5432/tcp         production_db_1
5 production_web

```

Note that I removed several columns to make the output readable.

At this point we can open <http://localhost:8080/rooms> with our browser and see the result of the HTTP request received by Nginx, passed to gunicorn, and processed by Flask using the use case `room_list_use_case`.

The application is not actually using the database yet, as the Flask endpoint `room_list` in `application/rest/room.py` initialises the class `MemRepo` and loads it with some static values, which are the ones we see in our browser.

Connect to a production-ready database

Before we start changing the code of the application remember to tear down the system running

```

1 $ ./manage.py compose down
2 Stopping production_web_1    ... done
3 Stopping production_nginx_1 ... done
4 Stopping production_db_1    ... done
5 Removing production_web_1    ... done
6 Removing production_nginx_1 ... done
7 Removing production_db_1    ... done
8 Removing network production_default

```

Thanks to the common interface between repositories, moving from the memory-based `MemRepo` to `PostgresRepo` is very simple. Clearly, as the external database will not contain any data initially, the response of the use case will be empty.

First of all, let's move the application to the Postgres repository. The new version of the endpoint is

application/rest/room.py

```
1  import os
2  import json
3
4  from flask import Blueprint, request, Response
5
6  from rentomatic.repository.postgresrepo import PostgresRepo
7  from rentomatic.use_cases.room_list import room_list_use_case
8  from rentomatic.serializers.room import RoomJsonEncoder
9  from rentomatic.requests.room_list import build_room_list_request
10 from rentomatic.responses import ResponseTypes
11
12 blueprint = Blueprint("room", __name__)
13
14 STATUS_CODES = {
15     ResponseTypes.SUCCESS: 200,
16     ResponseTypes.RESOURCE_ERROR: 404,
17     ResponseTypes.PARAMETERS_ERROR: 400,
18     ResponseTypes.SYSTEM_ERROR: 500,
19 }
20
21 postgres_configuration = {
22     "POSTGRES_USER": os.environ["POSTGRES_USER"],
23     "POSTGRES_PASSWORD": os.environ["POSTGRES_PASSWORD"],
24     "POSTGRES_HOSTNAME": os.environ["POSTGRES_HOSTNAME"],
25     "POSTGRES_PORT": os.environ["POSTGRES_PORT"],
26     "APPLICATION_DB": os.environ["APPLICATION_DB"],
27 }
28
29
30 @blueprint.route("/rooms", methods=["GET"])
31 def room_list():
32     qrystr_params = {
33         "filters": {},
34     }
35
36     for arg, values in request.args.items():
37         if arg.startswith("filter_"):
38             qrystr_params["filters"][arg.replace("filter_", "")] = values
39
40     request_object = build_room_list_request(
41         filters=qrystr_params["filters"]
42     )
```

```

43
44     repo = PostgresRepo(postgres_configuration)
45     response = room_list_use_case(repo, request_object)
46
47     return Response(
48         json.dumps(response.value, cls=RoomJsonEncoder),
49         mimetype="application/json",
50         status=STATUS_CODES[response.type],
51     )

```

As you can see the main change is that `repo = MemRepo(rooms)` becomes `repo = PostgresRepo(postgres_configuration)`. Such a simple change is made possible by the clean architecture and its strict layered approach. The only other notable change is that we replaced the initial data for the memory-based repository with a dictionary containing connection data, which comes from the environment variables set by the management script.

This is enough to make the application connect to the Postgres database that we are running in a container, but as I mentioned we also need to initialise the database. The bare minimum that we need is an empty database with the correct name. Remember that in this particular setup we use for the application a different database (`APPLICATION_DB`) from the one that the Postgres container creates automatically at startup (`POSTGRES_DB`). I added a specific command to the management script to perform this task

manage.py

```

1  @cli.command()
2  def init_postgres():
3      configure_app(os.getenv("APPLICATION_CONFIG"))
4
5      try:
6          run_sql([f"CREATE DATABASE {os.getenv('APPLICATION_DB')}"])
7      except psycopg2.errors.DuplicateDatabase:
8          print(
9              (
10                 f"The database {os.getenv('APPLICATION_DB')} already",
11                 "exists and will not be recreated",
12             )
13          )

```

Now spin up your containers

```

1 $ ./manage.py compose up -d
2 Creating network "production_default" with the default driver
3 Creating volume "production_pgdata" with default driver
4 Creating production_web_1 ... done
5 Creating production_nginx_1 ... done
6 Creating production_db_1 ... done

```

and run the new command that we created

```

1 $ ./manage.py init-postgres

```

Mind the change between the name of the function `init_postgres` and the name of the command `init-postgres`. You only need to run this command once, but repeated executions will not affect the database.

We can check what this command did connecting to the database. We can do it executing `psql` in the database container

```

1 $ ./manage.py compose exec db psql -U postgres
2 psql (13.4 (Debian 13.4-1.pgdg100+1))
3 Type "help" for help.
4
5 postgres=#

```

Please note that we need to specify the user `-U postgres`. That is the user that we created through the variable `POSTGRES_USER` in `config/production.json`. Once logged in, we can use the command `\l` to see the available databases

```

1 postgres=# \l
2
3                               List of databases
4  Name      | Owner   | Encoding | Collate  | Ctype    | Access privileges
5  -----+-----+-----+-----+-----+-----
6  application | postgres | UTF8     | en_US.utf8 | en_US.utf8 |
7  postgres   | postgres | UTF8     | en_US.utf8 | en_US.utf8 |
8  template0  | postgres | UTF8     | en_US.utf8 | en_US.utf8 | =c/postgres +
9              |          |          |          |          | postgres=CTc/postgres
10 template1  | postgres | UTF8     | en_US.utf8 | en_US.utf8 | =c/postgres +
11              |          |          |          |          | postgres=CTc/postgres
12
13 postgres=#

```


Please note that the two databases `template0` and `template1` are system databases created by Postgres (see [the documentation](https://www.postgresql.org/docs/current/manage-ag-templatedbs.html)³⁴), `postgres` is the default database created by the Docker container (the name is `postgres` by default, but in this case it comes from the environment variable `POSTGRES_DB` in `config/production.json`) and `application` is the database created by `./manage.py init-postgres` (from `APPLICATION_DB`).

We can connect to a database with the command `\c`

```
1 postgres=# \c application
2 You are now connected to database "application" as user "postgres".
3 application=#
```

Please note that the prompt changes with the name of the current database. Finally, we can list the available tables with `\dt`

```
1 application=# \dt
2 Did not find any relations.
```

As you can see there are no tables yet. This is no surprise as we didn't do anything to make Postgres aware of the models that we created. Please remember that everything we are doing here is done in an external system and it is not directly connected with entities.

As you remember, we mapped entities to storage objects, and since we are using Postgres we leveraged SQLAlchemy classes, so now we need to create the database tables that correspond to them.

Migrations

We need a way to create the tables that correspond to the objects that we defined in `rentomatic/repository/postgres/objects.py`. The best strategy, when we use an ORM like SQLAlchemy, is to create and run migrations, and for this we can use [Alembic](https://alembic.sqlalchemy.org/)³⁵.

If you are still connected with `psql` please exit with `\q`, then edit `requirements/prod.txt` and add `alembic`

³⁴<https://www.postgresql.org/docs/current/manage-ag-templatedbs.html>

³⁵<https://alembic.sqlalchemy.org/>

```
1 Flask
2 SQLAlchemy
3 psycopg2
4 pymongo
5 gunicorn
6 alembic
```

As usual, remember to run `pip install -r requirements/dev.txt` to update the virtual environment.

Alembic is capable of connecting to the database and run Python scripts (called “migrations”) to alter the tables according to the SQLAlchemy models. To do this, however, we need to give Alembic access to the database providing username, password, hostname, and the database name. We also need to give Alembic access to the Python classes that represent the models.

First of all let’s initialise Alembic. In the project’s main directory (where `manage.py` is stored) run

```
1 $ alembic init migrations
```

which creates a directory called `migrations` that contains Alembic’s configuration files, together with the migrations that will be created in `migrations/versions`. it will also create the file `alembic.ini` which contains the configuration values. The name `migrations` is completely arbitrary, so feel free to use a different one if you prefer.

The specific file we need to adjust to make Alembic aware of our models and our database is `migrations/env.py`. Add the highlighted lines

`migrations/env.py`

```
1 import os
2
3 from logging.config import fileConfig
4
5 from sqlalchemy import engine_from_config
6 from sqlalchemy import pool
7
8 from alembic import context
9
10 # this is the Alembic Config object, which provides
11 # access to the values within the .ini file in use.
12 config = context.config
13
14 section = config.config_ini_section
15 config.set_section_option(
16     section, "POSTGRES_USER", os.environ.get("POSTGRES_USER")
```

```
17 )
18 config.set_section_option(
19     section, "POSTGRES_PASSWORD", os.environ.get("POSTGRES_PASSWORD")
20 )
21 config.set_section_option(
22     section, "POSTGRES_HOSTNAME", os.environ.get("POSTGRES_HOSTNAME")
23 )
24 config.set_section_option(
25     section, "APPLICATION_DB", os.environ.get("APPLICATION_DB")
26 )
27
28 # Interpret the config file for Python logging.
29 # This line sets up loggers basically.
30 fileConfig(config.config_file_name)
31
32 # add your model's MetaData object here
33 # for 'autogenerate' support
34 # from myapp import mymodel
35 # target_metadata = mymodel.Base.metadata
36 # target_metadata = None
37 from rentomatic.repository.postgres_objects import Base
38
39 target_metadata = Base.metadata
40
41 # other values from the config, defined by the needs of env.py,
42 # can be acquired:
43 # my_important_option = config.get_main_option("my_important_option")
44 # ... etc.
```

Through `config.set_section_option` we are adding relevant configuration values to the main Alembic INI file section (`config.config_ini_section`), extracting them from the environment variables. We are also importing the file that contains the SQLAlchemy objects. You can find documentation on this procedure at <https://alembic.sqlalchemy.org/en/latest/api/config.html>.

Once this is done we need to change the INI file to use the new variables

alembic.ini

```

1  # the output encoding used when revision files
2  # are written from script.py.mako
3  # output_encoding = utf-8
4
5  sqlalchemy.url = postgresql://%(POSTGRES_USER)s:%(POSTGRES_PASSWORD)s@%(POSTGRES_HOST\
6  TNAME)s/%(APPLICATION_DB)s
7
8  [post_write_hooks]
9  # post_write_hooks defines scripts or Python functions that are run
10 # on newly generated revision scripts.  See the documentation for further
11 # detail and examples

```

The syntax `%(VARNAME)s` is the basic variable interpolation used by `ConfigParser` (see [the documentation](#)³⁶).

At this point we can run Alembic to migrate our database. In many cases, you can rely on Alembic's autogeneration functionality to generate the migrations, and this is what we can do to create the initial models. The Alembic command is `revision` with the `--autogenerate` flag, but we need to pass the environment variables on the command line. This is clearly a job for `migrate.py` but let's first run it to see what happens to the database. Later we will create a better setup to avoid passing variables manually

```

1  $ POSTGRES_USER=postgres\
2    POSTGRES_PASSWORD=postgres\
3    POSTGRES_HOSTNAME=localhost\
4    APPLICATION_DB=application\
5    alembic revision --autogenerate -m "Initial"

```

This will generate the file `migrations/versions/4d4c19952a36_initial.py`. Pay attention that the initial hash will be different for you. If you want you can open that file and see how Alembic generates the table and creates the columns.

So far we created the migration but we still need to apply it to the database. Make sure you are running the Docker containers (run `./manage.py compose up -d` otherwise) as Alembic is going to connect to the database, and run

³⁶<https://docs.python.org/3.8/library/configparser.html#configparser.BasicInterpolation>

```

1 $ POSTGRES_USER=postgres\
2   POSTGRES_PASSWORD=postgres\
3   POSTGRES_HOSTNAME=localhost\
4   APPLICATION_DB=application\
5   alembic upgrade head

```

At this point we can connect to the database and check the existing tables

```

1 $ ./manage.py compose exec db psql -U postgres -d application
2 psql (13.4 (Debian 13.4-1.pgdg100+1))
3 Type "help" for help.
4
5 application=# \dt
6               List of relations
7  Schema |      Name      | Type  | Owner
8  -----+-----+-----+-----
9  public | alembic_version | table | postgres
10 public | room           | table | postgres
11 (2 rows)
12
13 application=#

```

Please note that I used the option `-d` of `psql` to connect directly to the database `application`. As you can see, now we have two tables. The first, `alembic_version` is a simple one that Alembic uses to keep track of the state of the db, while `room` is the one that will contain our Room entities.

We can double-check the Alembic version

```

1 application=# select * from alembic_version;
2   version_num
3  -----
4   4d4c19952a36
5  (1 row)

```

As I mentioned before, the hash given to the migration will be different in your case, but that value that you see in this table should be consistent with the name of the migration script.

We can also see the structure of the table `room`

```

1 application=# \d room
2                                     Table "public.room"
3   Column      |      Type      | Collation | Nullable |      Default      |
4   -----+-----+-----+-----+-----+
5   id           | integer        |           | not null | nextval('room_id_seq'::r\
6   egclass)
7   code         | character varying(36) |           | not null |
8   size         | integer        |           |          |
9   price        | integer        |           |          |
10  longitude    | double precision |           |          |
11  latitude     | double precision |           |          |
12  Indexes:
13    "room_pkey" PRIMARY KEY, btree (id)

```

Clearly, there are still no rows contained in that table

```

1 application=# select * from room;
2  id | code | size | price | longitude | latitude
3  ---+-----+-----+-----+-----+-----+
4  (0 rows)

```

And indeed, if you open <http://localhost:8080/rooms> with your browser you will see a successful response, but no data.

To see some data we need to write something into the database. This is normally done through a form in the web application and a specific endpoint, but for the sake of simplicity in this case we can just add data manually to the database.

```

1 application=# INSERT INTO room(code, size, price, longitude, latitude) VALUES ('f853\
2 578c-fc0f-4e65-81b8-566c5dffa35a', 215, 39, -0.09998975, 51.75436293);
3 INSERT 0 1

```

You can verify that the table contains the new room with a SELECT

```

1 application=# SELECT * FROM room;
2   id |                               code | size | price | longitude | latitude \
3
4  ----+-----+-----+-----+-----+-----+
5  --
6    1 | f853578c-fc0f-4e65-81b8-566c5dffa35a | 215 | 39 | -0.09998975 | 51.75436293
7  (1 row)

```

and open or refresh <http://localhost:8080/rooms> with the browser to see the value returned by our use case.



Source code

<https://github.com/pycabook/rentomatic/tree/ed2-c08-s02>

This chapter concludes the overview of the clean architecture example. Starting from scratch, we created domain models, serializers, use cases, an in-memory storage system, a command-line interface and an HTTP endpoint. We then improved the whole system with a very generic request/response management code, that provides robust support for errors. Last, we implemented two new storage systems, using both a relational and a NoSQL database.

This is by no means a little achievement. Our architecture covers a very small use case, but is robust and fully tested. Whatever error we might find in the way we dealt with data, databases, requests, and so on, can be isolated and tamed much faster than in a system which doesn't have tests. Moreover, the decoupling philosophy not only allows us to provide support for multiple storage systems, but also to quickly implement new access protocols, or new serialisations for our objects.

Changelog



What's the last thing you do remember? Hmm?

Alien, 1979

I will track here changes between releases of the book, following [Semantic Versioning](#)³⁷. A change in the **major** number means an incompatible change, that is a big rewrite of the book, also known as 2nd edition, 3rd edition, and so on. A change in the **minor** number means that something important was added to the content, like a new section or chapter. A change in the **patch** number signals minor fixes like typos in the text or the code, rewording of sentences, and so on.

Current version: 2.1.0

Version 2.1.0 (2021-08-20)

- This version is written in Mau but converted into Markua to publish the PDF using Leanpub's processing chain.
- Chapter 8 has been improved with migrations that correctly create the tables in the production database.
- [Maxim](#)³⁸ corrected many bugs both in the book and in the code repository, and fixed several inconsistencies between the two. An impressive job, thank you so much for your help!
- GitHub user [robveijk](#)³⁹ spotted a mention to a file that is not included in the second edition. Thanks!
- GitHub user [mathisheeren](#)⁴⁰ corrected a typo. Thank you!
- GitHub user [4myhw](#)⁴¹ found a broken link and fixed the wrong use of `self` instead of `cls` in the code. Thanks!
- Several people, in particular [Jakob Waibel](#)⁴² spotted a typo in the name of Robert Martin. Thanks to all, and apologies to Mr. Martin.
- GitHub user [1110sillabo](#)⁴³ pointed out the PDF creation wasn't perfect with the toolchain based on AsciiDoctor, which was fixed going back to Lanpub's Markua.
- [Giovanni Natale](#)⁴⁴ found several issues both in the code and in the text and kindly submitted suggestions and fixes. Thanks!

³⁷<https://semver.org/>

³⁸<https://github.com/ivanovmg>

³⁹<https://github.com/robveijk>

⁴⁰<https://github.com/mathisheeren>

⁴¹<https://github.com/4myhw>

⁴²<https://github.com/JakWai01>

⁴³<https://github.com/1110sillabo>

⁴⁴<https://github.com/gnatale>

Version 2.0.1 (2021-02-14)

- GitHub users [1110sillabo](https://github.com/1110sillabo)⁴⁵ and the tireless [Faust Gertz](https://github.com/soulfulfaust)⁴⁶ kindly submitted some PRs to fix typos. Thanks!
- First version converted from Mau sources into AsciiDoctor

Version 2.0.0 (2020-12-30)

- Major rework of the structure of the book
- HTML version
- Introductory example with an overview of the components of the system
- Some nice figures
- Management script to orchestrate Docker
- Many typos added in random places

Version 1.0.12 (2020-04-13)

- GitHub user [Vlad Blazhko](https://github.com/VladBlazhko)⁴⁷ found a bug in the project `fileinfo` and added a fix and a test condition. As a result, I expanded the chapter on mocks with a small section describing what he did. Many thanks Vlad!

Version 1.0.11 (2020-02-15)

- GitHub user [lrfuentesw](https://github.com/lrfuentesw)⁴⁸ spotted an error in the memory repository. Price filters with a string value were not working because they were not converted into integers. Thank you!

Version 1.0.10 (2019-09-23)

- GitHub user [Ramces Chirino](https://github.com/RamcesChirino)⁴⁹ submitted a mega PR with many grammatical corrections. Thanks!

Version 1.0.9 (2019-04-12)

- GitHub user [plankington](https://github.com/plankington)⁵⁰ fixed some typos. Thank you!

⁴⁵<https://github.com/1110sillabo>

⁴⁶<https://github.com/soulfulfaust>

⁴⁷<https://github.com/pisarik>

⁴⁸<https://github.com/lrfuentesw>

⁴⁹<https://github.com/chirinosky>

⁵⁰<https://github.com/plankington>

Version 1.0.8 (2019-03-19)

- GitHub users [Faust Gertz](#)⁵¹ and [Michael “Irish” O’Neill](#)⁵² spotted a bug in the code of the example `calc`, chapter 1 of part 1. Thanks!
- GitHub user [Ahmed Ragab](#)⁵³ fixed some typos. Thank you so much!

Version 1.0.7 (2019-03-02)

- GitHub user [penguindustin](#)⁵⁴ suggested adding `pipenv` in the tools section as it is officially recommended by the Python packaging User Guide. Thanks!
- GitHub user [godiedelrio](#)⁵⁵ spotted an error in the file `rentomatic/rentomatic/repository/postgresrepo.py`. The code returned the result of the query without converting the single objects into domain entities. This was not spotted by tests as I haven’t introduced tests that check for the nature of the returned objects yet.

Version 1.0.6 (2019-02-06)

- The tireless [Eric Smith](#)⁵⁶ fixed typos and grammar in Part 2, Chapter 4. Thank you so much.

Version 1.0.5 (2019-01-28)

- [Eric Smith](#)⁵⁷ and [Faust Gertz](#)⁵⁸ fixed many typos in part 2. Thanks both for your help.

Version 1.0.4 (2019-01-22)

- [Grant Moore](#)⁵⁹ and [Hans Chen](#)⁶⁰ corrected two typos. Thank you!

Version 1.0.3 (2019-01-11)

- [Eric Smith](#)⁶¹ fixed more typos and corrected some phrasing in Chapter 3 of Part 1. Thanks Eric!

Version 1.0.2 (2019-01-09)

- [Max H. Gerlach](#)⁶² spotted and fixed more typos. Thanks again Max!

⁵¹<https://github.com/faustgertz>

⁵²<https://github.com/IrishPrime>

⁵³<https://github.com/Ragabov>

⁵⁴<https://github.com/penguindustin>

⁵⁵<https://github.com/godiedelrio>

⁵⁶<https://github.com/genericmoniker>

⁵⁷<https://github.com/genericmoniker>

⁵⁸<https://github.com/faustgertz>

⁵⁹<https://github.com/grantmoore3d>

⁶⁰<https://github.com/hanschen>

⁶¹<https://github.com/genericmoniker>

⁶²<https://github.com/maxhgerlach>

Version 1.0.1 (2019-01-01)

- [Max H. Gerlach](#)⁶³, [Paul Schwendenman](#)⁶⁴, and [Eric Smith](#)⁶⁵ kindly fixed many typos and grammar mistakes. Thank you very much!

Version 1.0.0 (2018-12-25)

- Initial release

⁶³<https://github.com/maxhgerlach>

⁶⁴<https://github.com/paul-schwendenman>

⁶⁵<https://github.com/genericmoniker>

Colophon

”

In conclusion, it should be noted...

Young Frankenstein, 1974

Writing a book is a giant effort.

I don't want this to sound like an excuse for the many errors that I forgot to fix or introduced. I merely want to state that what you read is the product of many many hours spent thinking how to explain a certain concept, debating with myself if a paragraph should come before another, and to endlessly trying to reconcile the code I wrote on the project and the Git commits with what is in the book. And since you read it until this point, I want to sincerely thank you for taking part in this adventure.

I didn't follow every advice I received, but I carefully read all the issues and messages that people sent me. I'm sorry I didn't manage to put in the book everything I wanted or everything you suggested. I have no commitment with an editor, but a certain point even a self-published book has to be called done. There will be more editions, I hope, so I just postponed the work. In the meanwhile, why don't you write a blog post on the clean architecture?

Thanks for reading my book!